BEHAVIORAL ANOMALY DETECTION SYSTEM

Complete Source Code & Technical Documentation Submission Bundle

Executive Benchmark & Architecture Summary

System: Multi-persona hybrid anomaly detection (Shared Autoencoder, Focal-Loss LSTM, Policy Engine, Signatures).
Benchmark Performance (Test Split Evaluation):

- Precision: 92.33% (0.923323) - Recall: 75.65% (0.756545) - F1-Score: 83.17% (0.831655)
- False Positive Rate: 0.16% (0.001642) - Steady-State P50: 13.45ms - Throughput: 10,704 events/sec
- Total Test Alerts: 313 (TN=14,592, FP=24, FN=93, TP=289) -- Assertion Passed (len(alerts.csv) == FP+TP)

Table of Contents / Included Source Files

Filename	Lines	Size (KB)	Description
README.md	161	7.5	System Documentation & Architecture Overview
config.py	554	15.4	Project Configuration & Hyperparameters
data_generator.py	907	31.2	Synthetic Log & Ground Truth Data Generator
feature_pipeline.py	553	17.6	Feature Engineering & Preprocessing Pipeline
models.py	1009	31.7	Autoencoder, LSTM & Risk Scorer Architectures
signature_rules.py	149	7.5	High-Precision Attack Signatures
alert_engine.py	411	15.8	Alert Generation & Attack Chain Reconstruction
drift_monitor.py	106	3.3	Concept Drift Monitoring (PSI)
inference_pipeline.py	609	27.0	End-to-End Inference & Captum IG Attribution
evaluation.py	523	19.9	Evaluation Metrics & Confusion Matrix Reports
pipeline_runner.py	431	16.8	Pipeline CLI Entrypoint & Stage Orchestrator
dashboard.py	717	25.0	SOC Interactive Streamlit Dashboard
scripts/validate_data.py	189	3.7	Dataset Integrity Validator
tests/conftest.py	9	0.2	Pytest Shared Fixtures
tests/test_smoke.py	187	7.2	Automated Verification Test Suite
TOTAL BUNDLE	**6515**	**229.9**	**16 Source Files Complete**

File: README.md

Description: System Documentation & Architecture Overview

```
# Behavioral Anomaly Detection Platform

AI-Powered Behavioral Anomaly Detection for IT/OT environments. Detects
insider threats, credential attacks, lateral movement, data exfiltration,
and device spoofing using a hybrid ML pipeline.

## Architecture

![System Architecture](Architecture%20Diagram.png)

### Hybrid Detection Pipeline

| Component | Purpose |
|-----------|---------|
| **SharedAutoencoder** | Reconstruction-based anomaly detection across all personas |
| **Adaptive Threshold** | Per-entity-type 95th percentile thresholds |
| **LSTMSequenceClassifier** | Temporal sequence classification with Focal Loss |
| **PolicyEngine** | Configurable rule-based scoring (10 rules) |
| **RiskScorer** | Budgeted weighted combination: `R = 0.10·AE + 0.80·LSTM + 0.10·Policy`; thresholds are global 0.37, corporate 0.73,
factory 0.37, service 0.42. |
| **Signature rules** | Precision-first impossible-travel, device-spoofing, and credential-stuffing signatures, unioned with the ML
threshold. |
| **Captum IG** | Integrated Gradients explainability |

## Configuration

Final calibration no longer uses the old hard 0.15 per-attack recall veto during threshold search. The shipped path uses a saved-model
calibration stage that tunes `w_ae`, `w_lstm`, and `w_policy` with a budget-constrained macro-F1 objective on the validation split, then
selects global and persona thresholds under the same alert-budget framing. Rare, high-precision classes are handled through the
signature-rule fast path instead of forcing one global ML threshold to satisfy incompatible objectives.

### Attack Types Detected

| Attack | MITRE Technique | Default Weight |
|--------|----------------|----------------|
| Brute Force | T1110 | 35% |
| Credential Stuffing | T1110.004 | 25% |
| Impossible Travel | T1078 | 15% |
| Device Spoofing | T1036 | 10% |
| Lateral Movement | T1021 | 8% |
| Low-and-Slow Exfiltration | T1048 | 5% |
| Insider Drift | T1074 | 2% |

### Persona Types

- **Corporate Employee** -- Office/VPN/Home ISP subnets, MFA/SSO/Password auth
- **Factory Operator** -- Plant/OT subnets, Badge/Password/Certificate auth
- **Service Account/IoT** -- Static infrastructure IPs, Certificate/API Key/Token auth

### Known Limitations

- Insider drift is detected reliably, but its attack-type label may be confused with lateral movement: on true insider-drift alerts the
prior softmax audit averaged 0.77 for lateral movement versus 0.17 for insider drift. Detection and classification should therefore be
interpreted separately.
- The impossible-travel signature intentionally favors sudden, externally sourced implausible jumps. It may miss slow, sustained
location drift; this is a deliberate precision/recall trade-off.
- Credential stuffing remains the disclosed detection gap: the synthetic attack generator rotates external source IPs, so a
precision-first "few source IPs across many accounts" signature has zero validation recall under its single-digit false-positive
constraint. The ML path detects 41.57% of test credential-stuffing events.
- `cold_start_demo_user` and `chain_demo_user` are clearly labelled test-only synthetic demo entities. `chain_demo_user` exists solely
to exercise genuine temporal chain reconstruction from correlated alert events; no chain ID is injected into the output.

### Alert Confidence Semantics

`classification_confidence` is the attack-type classifier's softmax probability only when the classifier selected the final attack label
for an `ml_threshold` alert. Signature-selected labels take precedence when a signature fires (including `both`); the dashboard
deliberately shows "Rule-based (validated)" or "Signature override (validated)" instead of attaching an unrelated classifier
probability. This is classification confidence, not a separate detection-confidence score.

### Concept Drift & Retraining Policy

The pipeline implements a two-tier Population Stability Index (PSI) drift monitor evaluating the distribution of Autoencoder
reconstruction errors for each persona.

- **Warning Tier (PSI >= 0.10)**: Moderate distribution shift detected. An alert is shown on the dashboard. No automated retraining is
triggered, but SOC teams should investigate for gradual behavioral changes.
```

- **Alert Tier (PSI >= 0.25)**: Significant distribution shift detected. Retraining should be triggered to update the baseline representations of normality.

Project Structure

```
behavioral_anomaly_detection/
??? data/
?   ??? raw/              # raw_logs.csv, ground_truth.csv
?   ??? processed/        # Feature arrays, scaler, encoder, metadata
??? models/               # Trained model checkpoints, thresholds
??? outputs/              # Alerts, evaluation reports
??? config.py             # All configuration (paths, seeds, hyperparameters)
??? data_generator.py     # Synthetic data generation with attack injection
??? feature_pipeline.py   # Point-in-time feature engineering
??? models.py             # PyTorch model definitions and training
??? evaluation.py         # Metrics computation and report generation
??? alert_engine.py       # Alert generation, MITRE mapping, chain reconstruction
??? inference_pipeline.py # End-to-end inference orchestration
??? pipeline_runner.py    # CLI entry point for all stages
??? dashboard.py          # Streamlit SOC dashboard
??? requirements.txt      # Python dependencies
??? README.md             # This file
```

Quick Start

1. Install Dependencies

```bash
cd behavioral_anomaly_detection
python -m venv venv
source venv/bin/activate  # or venv\Scripts\activate on Windows
pip install -r requirements.txt
```

2. Run the Full Pipeline

```bash
python pipeline_runner.py --stage all
```

This runs all 4 stages:
1. **generate** -- Creates 100k synthetic events with ~1-3% attacks
2. **features** -- Engineers 16 point-in-time features, splits, scales
3. **train** -- Trains Autoencoder (50 epochs) + LSTM (30 epochs)
4. **infer** -- Runs inference, generates alerts, evaluates, reports

3. Run Individual Stages

```bash
python pipeline_runner.py --stage generate
python pipeline_runner.py --stage features
python pipeline_runner.py --stage train
python pipeline_runner.py --stage infer --risk-threshold 0.6
```

4. Launch Dashboard

```bash
streamlit run dashboard.py
```

Configuration

All configurable values are in `config.py` as frozen dataclasses:

- **PathConfig** -- File/directory paths
- **DataGenConfig** -- Data generation parameters, persona configs, attack weights
- **FeatureConfig** -- Feature definitions, split ratios, scaler type
- **AutoencoderConfig** -- AE architecture (dims, dropout, activation)
- **LSTMConfig** -- LSTM architecture (hidden dims, layers, sequence length)
- **FocalLossConfig** -- Class imbalance loss (alpha, gamma)
- **RiskScorerConfig** -- Hybrid weights (w_ae, w_lstm, w_policy)
- **PolicyEngineConfig** -- Rule definitions and thresholds
- **TrainingConfig** -- Batch size, epochs, learning rates, early stopping

Evaluation Metrics

The platform computes and reports:
- Precision, Recall, F1-Score

```
- ROC-AUC, PR-AUC
- Confusion Matrix, False Positive Rate
- Per-attack-type performance
- Per-persona performance
- Reconstruction error and risk score distributions
- Inference latency statistics

## Coding Standards

- Python 3.11+
- Explicit type hints everywhere
- Google-style docstrings
- PEP8 compliant
- `logging` module only (no `print()`)
- Deterministic random seeds
- Vectorized NumPy/Pandas operations
- No TODOs, placeholders, or pseudocode

## License

Proprietary. Internal use only.
```

File: config.py

Description: Project Configuration & Hyperparameters

```
"""Central project configuration, path management, and system hyperparameters.

Defines all filesystem paths, persona types, attack classification labels,
feature definitions, and neural network training parameters.
"""
from __future__ import annotations

import logging
import os
from dataclasses import dataclass, field
from pathlib import Path
from typing import Dict, FrozenSet, List, Tuple

logger = logging.getLogger(__name__)

_PROJECT_ROOT: Path = Path(__file__).resolve().parent

# ============================================================================
# Project Path Configuration
# ============================================================================
@dataclass(frozen=True)
class PathConfig:

    project_root: Path = _PROJECT_ROOT
    raw_data_dir: Path = field(default_factory=lambda: _PROJECT_ROOT / "data" / "raw")
    processed_data_dir: Path = field(
        default_factory=lambda: _PROJECT_ROOT / "data" / "processed"
    )
    models_dir: Path = field(default_factory=lambda: _PROJECT_ROOT / "models")
    outputs_dir: Path = field(default_factory=lambda: _PROJECT_ROOT / "outputs")

    raw_logs_file: Path = field(
        default_factory=lambda: _PROJECT_ROOT / "data" / "raw" / "raw_logs.csv"
    )
    ground_truth_file: Path = field(
        default_factory=lambda: _PROJECT_ROOT / "data" / "raw" / "ground_truth.csv"
    )
    risk_weights_file: Path = field(
        default_factory=lambda: _PROJECT_ROOT / "models" / "risk_weights.json"
    )
    risk_thresholds_file: Path = field(
        default_factory=lambda: _PROJECT_ROOT / "models" / "risk_thresholds.json"
    )
    platt_scaler_file: Path = field(
        default_factory=lambda: _PROJECT_ROOT / "models" / "platt_scaler.pkl"
    )

    def ensure_dirs(self) -> None:
        for dir_path in (
            self.raw_data_dir,
            self.processed_data_dir,
            self.models_dir,
            self.outputs_dir,
        ):
            dir_path.mkdir(parents=True, exist_ok=True)
            logger.debug("Ensured directory exists: %s", dir_path)

RAW_LOG_COLUMNS: Tuple[str, ...] = (
    "entity_id",
    "entity_type",
    "timestamp",
    "source_ip",
    "geo_location",
    "resource_accessed",
    "auth_method",
    "session_duration",
    "command_sequence",
    "device_fingerprint",
)

GROUND_TRUTH_COLUMNS: Tuple[str, ...] = (
    "entity_id",
    "timestamp",
    "is_attack",
    "attack_type",
)
```

```
ENTITY_TYPES: Tuple[str, ...] = (
    "corporate_employee",
    "factory_operator",
    "service_account",
)

ENTITY_TYPE_TO_INDEX: Dict[str, int] = {
    etype: idx for idx, etype in enumerate(ENTITY_TYPES)
}

NUM_ENTITY_TYPES: int = len(ENTITY_TYPES)

ATTACK_TYPES: Tuple[str, ...] = (
    "brute_force",
    "credential_stuffing",
    "impossible_travel",
    "lateral_movement",
    "device_spoofing",
    "low_and_slow_exfiltration",
    "insider_drift",
)

NUM_ATTACK_TYPES: int = len(ATTACK_TYPES)

ATTACK_TYPE_LABELS: Dict[str, int] = {
    "benign": 0,
    **{atype: idx + 1 for idx, atype in enumerate(ATTACK_TYPES)},
}
ATTACK_TYPE_LABELS_INV: Dict[int, str] = {v: k for k, v in ATTACK_TYPE_LABELS.items()}
NUM_ATTACK_CLASSES: int = len(ATTACK_TYPE_LABELS)

@dataclass(frozen=True)
class DataGenConfig:

    num_entities: int = 500
    num_events: int = 100_000
    entity_type_distribution: Tuple[float, ...] = (0.50, 0.30, 0.20)
    attack_rate_min: float = 0.005
    attack_rate_max: float = 0.03
    simulation_days: int = 30
    random_seed: int = 42
    faker_seed: int = 42
    numpy_seed: int = 42

    geo_locations: Tuple[Tuple[float, float, str], ...] = (
        (37.7749, -122.4194, "San Francisco"),
        (40.7128, -74.0060, "New York"),
        (51.5074, -0.1278, "London"),
        (35.6762, 139.6503, "Tokyo"),
        (1.3521, 103.8198, "Singapore"),
        (48.8566, 2.3522, "Paris"),
        (55.7558, 37.6173, "Moscow"),
        (28.6139, 77.2090, "New Delhi"),
        (-33.8688, 151.2093, "Sydney"),
        (52.5200, 13.4050, "Berlin"),
    )

    auth_methods: Tuple[str, ...] = (
        "password",
        "mfa",
        "sso",
        "certificate",
        "api_key",
        "biometric",
    )

    corporate_auth_weights: Tuple[float, ...] = (
        0.25,
        0.25,
        0.25,
        0.03,
        0.02,
        0.20,
    )
    factory_auth_weights: Tuple[float, ...] = (
        0.35,
        0.25,
        0.05,
        0.20,
        0.05,
```

```
    0.10,
)
service_auth_weights: Tuple[float, ...] = (
    0.02,
    0.00,
    0.05,
    0.40,
    0.40,
    0.00,
)
service_extra_auth_methods: Tuple[str, ...] = ("token",)
service_extra_auth_weights: Tuple[float, ...] = (0.13,)

corporate_resources: Tuple[str, ...] = (
    "email_server",
    "crm_system",
    "hr_portal",
    "sharepoint",
    "vpn_gateway",
    "code_repository",
    "internal_wiki",
    "finance_dashboard",
)
factory_resources: Tuple[str, ...] = (
    "plc_controller",
    "scada_hmi",
    "historian_db",
    "safety_system",
    "robot_arm_interface",
    "sensor_gateway",
    "batch_scheduler",
    "maintenance_portal",
)
service_resources: Tuple[str, ...] = (
    "telemetry_endpoint",
    "config_server",
    "firmware_update_api",
    "certificate_authority",
    "ntp_server",
    "log_aggregator",
    "health_check_api",
    "ota_update_server",
)

corporate_commands: Tuple[str, ...] = (
    "ls", "cd", "cat", "grep", "git", "ssh", "scp", "curl",
    "docker", "kubectl", "python", "pip", "code", "outlook",
    "teams", "slack", "whoami", "chmod", "nano", "vim",
)
factory_commands: Tuple[str, ...] = (
    "read_register", "write_register", "set_point", "get_status",
    "start_batch", "stop_batch", "calibrate", "acknowledge_alarm",
    "download_firmware", "upload_config", "restart_plc",
    "read_sensor", "set_threshold", "toggle_valve", "run_diagnostic",
)
service_commands: Tuple[str, ...] = (
    "heartbeat", "send_telemetry", "fetch_config", "check_update",
    "rotate_cert", "sync_time", "report_status", "upload_logs",
    "ping", "ack", "register", "deregister", "self_test",
)

corporate_fingerprints: Tuple[str, ...] = (
    "Win11-Chrome-x64",
    "Win10-Edge-x64",
    "macOS-Safari-arm64",
    "macOS-Chrome-arm64",
    "Ubuntu-Firefox-x64",
)
factory_fingerprints: Tuple[str, ...] = (
    "WinCE-HMI-Panel",
    "Linux-SCADA-x86",
    "RTOS-PLC-arm",
    "VxWorks-DCS-mips",
)
service_fingerprints: Tuple[str, ...] = (
    "Linux-IoT-arm",
    "FreeRTOS-Sensor-cortex",
    "Zephyr-Gateway-riscv",
    "Embedded-Linux-mips",
)
```

```
    corporate_work_hour_start: int = 8
    corporate_work_hour_end: int = 18
    factory_shift_hours: Tuple[int, ...] = (6, 14, 22)
    service_interval_minutes_mean: float = 5.0
    service_interval_minutes_std: float = 2.0

    corporate_session_duration_range: Tuple[float, float] = (5.0, 480.0)
    factory_session_duration_range: Tuple[float, float] = (10.0, 720.0)
    service_session_duration_range: Tuple[float, float] = (0.1, 5.0)

    command_sequence_length_range: Tuple[int, int] = (1, 8)

    corporate_ip_subnets: Tuple[str, ...] = (
        "10.1.{}.{}",
        "10.200.{}.{}",
        "192.168.1.{}",
    )
    corporate_ip_subnet_weights: Tuple[float, ...] = (0.55, 0.30, 0.15)
    factory_ip_subnets: Tuple[str, ...] = (
        "10.10.{}.{}",
        "10.20.{}.{}",
    )
    factory_ip_subnet_weights: Tuple[float, ...] = (0.60, 0.40)
    service_ip_subnets: Tuple[str, ...] = (
        "10.100.0.{}",
        "10.100.1.{}",
    )
    service_ip_subnet_weights: Tuple[float, ...] = (0.50, 0.50)

    attack_type_weights: Tuple[float, ...] = (
        0.35,
        0.25,
        0.15,
        0.08,
        0.10,
        0.05,
        0.02,
    )

    spoofed_os_options: Tuple[str, ...] = (
        "Win11", "Win10", "macOS", "Ubuntu", "Android", "Kali",
    )
    spoofed_browser_options: Tuple[str, ...] = (
        "Chrome", "Firefox", "Edge", "Headless-Chrome", "Selenium",
        "Puppeteer", "curl",
    )
    spoofed_arch_options: Tuple[str, ...] = (
        "x64", "x86", "arm64", "arm", "mips",
    )
    spoofed_profile_options: Tuple[str, ...] = (
        "Desktop", "Mobile", "VM", "Emulator", "Container",
    )

    impossible_travel_max_speed_kmh: float = 900.0
    impossible_travel_min_distance_km: float = 5000.0
    impossible_travel_max_gap_seconds: int = 1800

@dataclass(frozen=True)
class FeatureConfig:

    behavioral_features: Tuple[str, ...] = (
        "login_hour",
        "day_of_week",
        "session_duration",
        "time_since_last_login",
    )
    network_features: Tuple[str, ...] = (
        "unique_ips_last_hour",
        "geo_distance_km",
        "failed_logins_last_10min",
        "travel_speed_kmh",
    )
    device_resource_features: Tuple[str, ...] = (
        "fingerprint_consistency",
        "resource_rarity",
        "sequence_entropy",
        "privilege_escalation_count",
        "os_mismatch",
        "browser_mismatch",
    )
    relationship_features: Tuple[str, ...] = (
```

```
        "degree_centrality",
        "newly_observed_neighbors",
    )

    unique_ip_window_seconds: int = 3600
    failed_login_window_seconds: int = 600

    geo_distance_earth_radius_km: float = 6371.0

    sequence_entropy_base: int = 2

    train_ratio: float = 0.7
    val_ratio: float = 0.15
    test_ratio: float = 0.15

    scaler_type: str = "standard"

    x_train_file: str = "X_train.npy"
    x_val_file: str = "X_val.npy"
    x_test_file: str = "X_test.npy"
    y_train_file: str = "y_train.npy"
    y_val_file: str = "y_val.npy"
    y_test_file: str = "y_test.npy"
    feature_metadata_file: str = "feature_metadata.json"
    scaler_file: str = "scaler.pkl"
    encoder_file: str = "encoder.pkl"

    @property
    def all_feature_names(self) -> Tuple[str, ...]:
        return (
            *self.behavioral_features,
            *self.network_features,
            *self.device_resource_features,
            *self.relationship_features,
        )

    @property
    def num_raw_features(self) -> int:
        return len(self.all_feature_names)

@dataclass(frozen=True)
class AutoencoderConfig:

    encoder_dims: Tuple[int, ...] = (128, 64)
    latent_dim: int = 32
    decoder_dims: Tuple[int, ...] = (64, 128)
    dropout_rate: float = 0.2
    activation: str = "relu"
    threshold_percentile: float = 95.0

@dataclass(frozen=True)
class LSTMConfig:

    hidden_dim: int = 128
    num_layers: int = 2
    bidirectional: bool = True
    dropout_rate: float = 0.3
    sequence_length: int = 10
    fc_dims: Tuple[int, ...] = (64,)

@dataclass(frozen=True)
class FocalLossConfig:

    alpha: float = 0.85
    gamma: float = 2.0

@dataclass(frozen=True)
class AttackClassifierConfig:

    fc_dims: Tuple[int, ...] = (128, 64)
    dropout_rate: float = 0.3
    num_classes: int = 8
    num_epochs: int = 20
    learning_rate: float = 1e-3
    focal_alpha: float = 0.75
    focal_gamma: float = 2.0

@dataclass(frozen=True)
class RiskScorerConfig:

    w_ae: float = 0.4
```

```
    w_lstm: float = 0.4
    w_policy: float = 0.2

@dataclass(frozen=True)
class PolicyRule:

    name: str
    feature: str
    operator: str
    threshold: float
    score: float

@dataclass(frozen=True)
class PolicyEngineConfig:

    rules: Tuple[PolicyRule, ...] = (
        PolicyRule(
            name="impossible_travel_speed",
            feature="travel_speed_kmh",
            operator="gt",
            threshold=900.0,
            score=1.5,
        ),
        PolicyRule(
            name="excessive_failed_logins",
            feature="failed_logins_last_10min",
            operator="gt",
            threshold=5.0,
            score=0.25,
        ),
        PolicyRule(
            name="unusual_login_hour",
            feature="login_hour",
            operator="gt",
            threshold=22.0,
            score=0.15,
        ),
        PolicyRule(
            name="high_sequence_entropy",
            feature="sequence_entropy",
            operator="gt",
            threshold=3.0,
            score=0.15,
        ),
        PolicyRule(
            name="low_fingerprint_consistency",
            feature="fingerprint_consistency",
            operator="lt",
            threshold=0.5,
            score=0.2,
        ),
        PolicyRule(
            name="os_mismatch",
            feature="os_mismatch",
            operator="eq",
            threshold=1.0,
            score=1.5,
        ),
        PolicyRule(
            name="browser_mismatch",
            feature="browser_mismatch",
            operator="eq",
            threshold=1.0,
            score=1.5,
        ),
        PolicyRule(
            name="high_unique_ips",
            feature="unique_ips_last_hour",
            operator="gt",
            threshold=10.0,
            score=0.15,
        ),
        PolicyRule(
            name="privilege_escalation",
            feature="privilege_escalation_count",
            operator="gt",
            threshold=2.0,
            score=0.25,
        ),
        PolicyRule(
            name="resource_rarity_spike",
```

```
            feature="resource_rarity",
            operator="gt",
            threshold=0.9,
            score=0.15,
        ),
    )
    max_score: float = 5.8

@dataclass(frozen=True)
class ModelConfig:

    autoencoder: AutoencoderConfig = field(default_factory=AutoencoderConfig)
    lstm: LSTMConfig = field(default_factory=LSTMConfig)
    focal_loss: FocalLossConfig = field(default_factory=FocalLossConfig)
    risk_scorer: RiskScorerConfig = field(default_factory=RiskScorerConfig)
    policy_engine: PolicyEngineConfig = field(default_factory=PolicyEngineConfig)
    attack_classifier: AttackClassifierConfig = field(default_factory=AttackClassifierConfig)

@dataclass(frozen=True)
class TrainingConfig:

    batch_size: int = 256
    num_epochs_ae: int = 50
    num_epochs_lstm: int = 30
    learning_rate_ae: float = 1e-3
    learning_rate_lstm: float = 5e-4
    weight_decay: float = 1e-5
    early_stopping_patience: int = 7
    device: str = "cpu"
    random_seed: int = 42
    num_workers: int = 0
    pin_memory: bool = False
    gradient_clip_max_norm: float = 1.0

    weight_search_step: float = 0.05

    threshold_search_start: float = 0.05
    threshold_search_end: float = 0.95
    threshold_search_step: float = 0.01
    threshold_optimisation_objective: str = "f1"
    threshold_optimisation_constraint: float = 0.01
    alert_budget: int = 350

    cold_start_min_history_days: Dict[str, float] = field(default_factory=lambda: {
        "corporate_employee": 7.0,
        "factory_operator": 3.0,
        "service_account": 1.0,
    })
    cold_start_w_ae: float = 0.05
    cold_start_w_lstm: float = 0.10
    cold_start_w_policy: float = 0.85

    drift_window_size: int = 1000
    drift_psi_warning: float = 0.1
    drift_psi_alert: float = 0.25
    drift_n_bins: int = 10

@dataclass(frozen=True)
class ProjectConfig:

    paths: PathConfig = field(default_factory=PathConfig)
    data_gen: DataGenConfig = field(default_factory=DataGenConfig)
    features: FeatureConfig = field(default_factory=FeatureConfig)
    model: ModelConfig = field(default_factory=ModelConfig)
    training: TrainingConfig = field(default_factory=TrainingConfig)

def get_project_config() -> ProjectConfig:
    config = ProjectConfig()
    config.paths.ensure_dirs()
    logger.info("ProjectConfig initialised. Root: %s", config.paths.project_root)
    return config

PROJECT_CONFIG: ProjectConfig = get_project_config()
```

File: data_generator.py

Description: Synthetic Log & Ground Truth Data Generator

```
"""Synthetic security log generator simulating realistic multi-persona behavioral events and injected attack patterns.

Simulates normal baseline activities and 7 MITRE attack types across 30 days, creating raw_logs.csv and ground_truth.csv.
"""

from __future__ import annotations

import logging
import math
import random
from dataclasses import dataclass, field
from datetime import datetime, timedelta, timezone
from pathlib import Path
from typing import Any, Dict, List, Optional, Tuple

import numpy as np
import pandas as pd
from faker import Faker

from config import (
    ATTACK_TYPES,
    ENTITY_TYPES,
    GROUND_TRUTH_COLUMNS,
    RAW_LOG_COLUMNS,
    DataGenConfig,
    PathConfig,
    ProjectConfig,
    get_project_config,
)

logger = logging.getLogger(__name__)

_GEO_COORDS: Dict[str, Tuple[float, float]] = {}

def _haversine_km(
    lat1: float, lon1: float, lat2: float, lon2: float,
    earth_radius_km: float = 6371.0,
) -> float:
    lat1_r, lat2_r = math.radians(lat1), math.radians(lat2)
    dlat = math.radians(lat2 - lat1)
    dlon = math.radians(lon2 - lon1)
    a = (math.sin(dlat / 2.0) ** 2
         + math.cos(lat1_r) * math.cos(lat2_r) * math.sin(dlon / 2.0) ** 2)
    c = 2.0 * math.atan2(math.sqrt(a), math.sqrt(1.0 - a))
    return earth_radius_km * c

@dataclass
class EntityProfile:

    entity_id: str
    entity_type: str
    home_geo: Tuple[float, float, str]
    primary_fingerprint: str
    primary_auth: str
    auth_methods: Tuple[str, ...]
    auth_weights: np.ndarray
    primary_resources: Tuple[str, ...]
    command_vocab: Tuple[str, ...]
    ip_subnets: Tuple[str, ...]
    ip_subnet_weights: np.ndarray
    primary_ip: str = ""
    last_login_ts: Optional[datetime] = None
    login_count: int = 0
    ip_history: List[str] = field(default_factory=list)

def _set_deterministic_seeds(cfg: DataGenConfig) -> None:
    random.seed(cfg.random_seed)
    np.random.seed(cfg.numpy_seed)
    logger.info(
        "Seeds set -- random: %d, numpy: %d, faker: %d",
        cfg.random_seed,
        cfg.numpy_seed,
        cfg.faker_seed,
    )

def _create_faker(cfg: DataGenConfig) -> Faker:
```

```
    fake = Faker()
    Faker.seed(cfg.faker_seed)
    return fake

def _normalise_weights(weights: Tuple[float, ...]) -> np.ndarray:
    arr = np.array(weights, dtype=np.float64)
    total = arr.sum()
    if total <= 0:
        return np.ones_like(arr) / len(arr)
    return arr / total

def _generate_persona_ip(
    subnets: Tuple[str, ...],
    subnet_weights: np.ndarray,
    rng: np.random.Generator,
) -> str:
    idx = int(rng.choice(len(subnets), p=subnet_weights))
    template = subnets[idx]
    placeholders = template.count("{}")
    parts = [int(rng.integers(1, 255)) for _ in range(placeholders)]
    return template.format(*parts)

def _generate_command_sequence(
    vocab: Tuple[str, ...],
    rng: np.random.Generator,
    length_range: Tuple[int, int],
) -> str:
    seq_len = int(rng.integers(length_range[0], length_range[1] + 1))
    indices = rng.integers(0, len(vocab), size=seq_len)
    return ";".join(vocab[i] for i in indices)

def _sample_session_duration(
    entity_type: str,
    cfg: DataGenConfig,
    rng: np.random.Generator,
) -> float:
    ranges = {
        "corporate_employee": cfg.corporate_session_duration_range,
        "factory_operator": cfg.factory_session_duration_range,
        "service_account": cfg.service_session_duration_range,
    }
    lo, hi = ranges[entity_type]
    mu = (math.log(lo) + math.log(hi)) / 2.0
    sigma = (math.log(hi) - math.log(lo)) / 4.0
    duration = float(rng.lognormal(mu, sigma))
    return round(max(lo, min(hi, duration)), 2)

def _sample_timestamp_for_persona(
    entity_type: str,
    base_date: datetime,
    cfg: DataGenConfig,
    rng: np.random.Generator,
) -> datetime:
    if entity_type == "corporate_employee":
        if rng.random() < 0.90:
            hour = int(
                rng.integers(cfg.corporate_work_hour_start, cfg.corporate_work_hour_end)
            )
        else:
            hour = int(rng.integers(0, 24))
    elif entity_type == "factory_operator":
        shift_start = int(rng.choice(cfg.factory_shift_hours))
        hour = (shift_start + int(rng.integers(0, 8))) % 24
    else:
        hour = int(rng.integers(0, 24))

    minute = int(rng.integers(0, 60))
    second = int(rng.integers(0, 60))
    return base_date.replace(
        hour=hour, minute=minute, second=second, tzinfo=timezone.utc
    )

def _sample_persona_auth(
    profile: EntityProfile,
    rng: np.random.Generator,
) -> str:
    if rng.random() < 0.80:
        return profile.primary_auth
    idx = int(rng.choice(len(profile.auth_methods), p=profile.auth_weights))
    return profile.auth_methods[idx]
```

```
def _create_entity_profiles(
    cfg: DataGenConfig,
    fake: Faker,
    rng: np.random.Generator,
) -> List[EntityProfile]:
    profiles: List[EntityProfile] = []

    type_counts = np.round(
        np.array(cfg.entity_type_distribution) * cfg.num_entities
    ).astype(int)
    type_counts[-1] = cfg.num_entities - type_counts[:-1].sum()

    entity_idx = 0
    for etype_idx, etype in enumerate(ENTITY_TYPES):
        count = int(type_counts[etype_idx])

        if etype == "corporate_employee":
            resources = cfg.corporate_resources
            commands = cfg.corporate_commands
            fingerprints = cfg.corporate_fingerprints
            auth_meths = cfg.auth_methods
            auth_wts = _normalise_weights(cfg.corporate_auth_weights)
            ip_subs = cfg.corporate_ip_subnets
            ip_wts = _normalise_weights(cfg.corporate_ip_subnet_weights)
        elif etype == "factory_operator":
            resources = cfg.factory_resources
            commands = cfg.factory_commands
            fingerprints = cfg.factory_fingerprints
            auth_meths = cfg.auth_methods
            auth_wts = _normalise_weights(cfg.factory_auth_weights)
            ip_subs = cfg.factory_ip_subnets
            ip_wts = _normalise_weights(cfg.factory_ip_subnet_weights)
        else:
            resources = cfg.service_resources
            commands = cfg.service_commands
            fingerprints = cfg.service_fingerprints
            auth_meths = cfg.auth_methods + cfg.service_extra_auth_methods
            base_wts = cfg.service_auth_weights + cfg.service_extra_auth_weights
            auth_wts = _normalise_weights(base_wts)
            ip_subs = cfg.service_ip_subnets
            ip_wts = _normalise_weights(cfg.service_ip_subnet_weights)

        for _ in range(count):
            entity_id = f"{etype[:3].upper()}-{entity_idx:05d}"
            home_geo = cfg.geo_locations[int(rng.integers(0, len(cfg.geo_locations)))]
            primary_fp = fingerprints[int(rng.integers(0, len(fingerprints)))]

            primary_auth_idx = int(rng.choice(len(auth_meths), p=auth_wts))
            primary_auth = auth_meths[primary_auth_idx]

            num_resources = max(2, int(rng.integers(2, len(resources) + 1)))
            resource_indices = rng.choice(
                len(resources), size=num_resources, replace=False
            )
            entity_resources = tuple(resources[i] for i in resource_indices)

            primary_ip = _generate_persona_ip(ip_subs, ip_wts, rng)

            profiles.append(
                EntityProfile(
                    entity_id=entity_id,
                    entity_type=etype,
                    home_geo=home_geo,
                    primary_fingerprint=primary_fp,
                    primary_auth=primary_auth,
                    auth_methods=auth_meths,
                    auth_weights=auth_wts,
                    primary_resources=entity_resources,
                    command_vocab=commands,
                    ip_subnets=ip_subs,
                    ip_subnet_weights=ip_wts,
                    primary_ip=primary_ip,
                )
            )
            entity_idx += 1

    logger.info(
        "Created %d entity profiles: %s",
        len(profiles),
        {etype: int(c) for etype, c in zip(ENTITY_TYPES, type_counts)},
    )
```

```
    return profiles

def _generate_normal_event(
    profile: EntityProfile,
    timestamp: datetime,
    cfg: DataGenConfig,
    rng: np.random.Generator,
) -> Dict[str, Any]:
    if profile.ip_history and rng.random() < 0.70:
        source_ip = profile.ip_history[int(rng.integers(0, len(profile.ip_history)))]
    elif rng.random() < 0.50:
        source_ip = profile.primary_ip
    else:
        source_ip = _generate_persona_ip(
            profile.ip_subnets, profile.ip_subnet_weights, rng
        )
    if source_ip not in profile.ip_history:
        profile.ip_history.append(source_ip)
        if len(profile.ip_history) > 5:
            profile.ip_history = profile.ip_history[-5:]

    if rng.random() < 0.85:
        geo_label = profile.home_geo[2]
    else:
        nearby = cfg.geo_locations[int(rng.integers(0, len(cfg.geo_locations)))]
        geo_label = nearby[2]

    resource = profile.primary_resources[
        int(rng.integers(0, len(profile.primary_resources)))
    ]

    auth = _sample_persona_auth(profile, rng)

    session_dur = _sample_session_duration(profile.entity_type, cfg, rng)

    cmd_seq = _generate_command_sequence(
        profile.command_vocab, rng, cfg.command_sequence_length_range
    )

    if rng.random() < 0.90:
        fingerprint = profile.primary_fingerprint
    else:
        if profile.entity_type == "corporate_employee":
            fps = cfg.corporate_fingerprints
        elif profile.entity_type == "factory_operator":
            fps = cfg.factory_fingerprints
        else:
            fps = cfg.service_fingerprints
        fingerprint = fps[int(rng.integers(0, len(fps)))]

    profile.last_login_ts = timestamp
    profile.login_count += 1

    return {
        "entity_id": profile.entity_id,
        "entity_type": profile.entity_type,
        "timestamp": timestamp.isoformat(),
        "source_ip": source_ip,
        "geo_location": geo_label,
        "resource_accessed": resource,
        "auth_method": auth,
        "session_duration": session_dur,
        "command_sequence": cmd_seq,
        "device_fingerprint": fingerprint,
    }

def _inject_brute_force(
    event: Dict[str, Any],
    cfg: DataGenConfig,
    rng: np.random.Generator,
) -> Dict[str, Any]:
    event["auth_method"] = "password"
    event["session_duration"] = round(float(rng.uniform(0.01, 0.5)), 2)
    event["command_sequence"] = ";".join(["login_attempt"] * int(rng.integers(5, 20)))
    if rng.random() < 0.40:
        event["source_ip"] = f"172.16.{rng.integers(0, 256)}.{rng.integers(1, 255)}"
    return event

def _inject_credential_stuffing(
    event: Dict[str, Any],
    cfg: DataGenConfig,
```

```
    rng: np.random.Generator,
) -> Dict[str, Any]:
    event["source_ip"] = f"172.{rng.integers(16, 32)}.{rng.integers(0, 256)}.{rng.integers(1, 255)}"
    event["auth_method"] = "password"
    event["session_duration"] = round(float(rng.uniform(0.01, 1.0)), 2)
    event["command_sequence"] = ";".join(
        ["auth_check", "login_attempt"] * int(rng.integers(2, 6))
    )
    return event

def _inject_impossible_travel(
    event: Dict[str, Any],
    cfg: DataGenConfig,
    rng: np.random.Generator,
) -> Dict[str, Any]:
    current_label = event["geo_location"]
    current_coords = _GEO_COORDS.get(current_label)

    min_dist = cfg.impossible_travel_min_distance_km
    far_candidates = []
    for loc in cfg.geo_locations:
        if loc[2] == current_label:
            continue
        if current_coords is not None:
            dist = _haversine_km(
                current_coords[0], current_coords[1], loc[0], loc[1]
            )
            if dist >= min_dist:
                far_candidates.append(loc)
        else:
            far_candidates.append(loc)

    if not far_candidates:
        far_candidates = [loc for loc in cfg.geo_locations if loc[2] != current_label]

    if far_candidates:
        far_loc = far_candidates[int(rng.integers(0, len(far_candidates)))]
        event["geo_location"] = far_loc[2]

    ts = datetime.fromisoformat(event["timestamp"])
    gap_seconds = int(rng.integers(60, cfg.impossible_travel_max_gap_seconds + 1))
    ts = ts - timedelta(seconds=gap_seconds)
    event["timestamp"] = ts.isoformat()

    event["source_ip"] = f"172.16.{rng.integers(0, 256)}.{rng.integers(1, 255)}"
    return event

def _inject_lateral_movement(
    event: Dict[str, Any],
    cfg: DataGenConfig,
    rng: np.random.Generator,
) -> Dict[str, Any]:
    entity_type = event["entity_type"]
    if entity_type == "corporate_employee":
        cross_resources = cfg.factory_resources + cfg.service_resources
    elif entity_type == "factory_operator":
        cross_resources = cfg.corporate_resources + cfg.service_resources
    else:
        cross_resources = cfg.corporate_resources + cfg.factory_resources

    event["resource_accessed"] = cross_resources[
        int(rng.integers(0, len(cross_resources)))
    ]
    lateral_cmds = [
        "net_scan", "port_scan", "whoami", "net_user", "tasklist",
        "reg_query", "psexec", "wmic", "mimikatz", "pass_the_hash",
    ]
    num_cmds = int(rng.integers(3, 8))
    event["command_sequence"] = ";".join(
        lateral_cmds[int(rng.integers(0, len(lateral_cmds)))]
        for _ in range(num_cmds)
    )
    return event

def _inject_device_spoofing(
    event: Dict[str, Any],
    cfg: DataGenConfig,
    rng: np.random.Generator,
) -> Dict[str, Any]:
    os_choice = cfg.spoofed_os_options[
        int(rng.integers(0, len(cfg.spoofed_os_options)))
```

```
    ]
    browser_choice = cfg.spoofed_browser_options[
        int(rng.integers(0, len(cfg.spoofed_browser_options)))
    ]
    arch_choice = cfg.spoofed_arch_options[
        int(rng.integers(0, len(cfg.spoofed_arch_options)))
    ]
    profile_choice = cfg.spoofed_profile_options[
        int(rng.integers(0, len(cfg.spoofed_profile_options)))
    ]
    event["device_fingerprint"] = f"{os_choice}-{browser_choice}-{arch_choice}-{profile_choice}"

    if rng.random() < 0.50:
        event["source_ip"] = f"172.16.{rng.integers(0, 256)}.{rng.integers(1, 255)}"
    return event

def _inject_low_and_slow_exfiltration(
    event: Dict[str, Any],
    cfg: DataGenConfig,
    rng: np.random.Generator,
) -> Dict[str, Any]:
    event["session_duration"] = round(float(rng.uniform(300.0, 720.0)), 2)
    exfil_cmds = [
        "scp", "rsync", "curl_upload", "compress", "encrypt",
        "split_file", "base64_encode", "dns_tunnel",
    ]
    num_cmds = int(rng.integers(4, 10))
    event["command_sequence"] = ";".join(
        exfil_cmds[int(rng.integers(0, len(exfil_cmds)))]
        for _ in range(num_cmds)
    )
    sensitive_resources = ["finance_dashboard", "hr_portal", "historian_db", "crm_system"]
    event["resource_accessed"] = sensitive_resources[
        int(rng.integers(0, len(sensitive_resources)))
    ]
    return event

def _inject_insider_drift(
    event: Dict[str, Any],
    cfg: DataGenConfig,
    rng: np.random.Generator,
) -> Dict[str, Any]:
    ts = datetime.fromisoformat(event["timestamp"])
    unusual_hour = int(rng.choice([0, 1, 2, 3, 4, 5, 23, 22, 21]))
    ts = ts.replace(hour=unusual_hour)
    event["timestamp"] = ts.isoformat()

    entity_type = event["entity_type"]
    if entity_type == "corporate_employee":
        cross_resources = cfg.factory_resources + cfg.service_resources
    elif entity_type == "factory_operator":
        cross_resources = cfg.corporate_resources + cfg.service_resources
    else:
        cross_resources = cfg.corporate_resources + cfg.factory_resources

    event["resource_accessed"] = cross_resources[
        int(rng.integers(0, len(cross_resources)))
    ]

    event["auth_method"] = cfg.auth_methods[
        int(rng.integers(0, len(cfg.auth_methods)))
    ]

    return event

_ATTACK_INJECTORS = {
    "brute_force": _inject_brute_force,
    "credential_stuffing": _inject_credential_stuffing,
    "impossible_travel": _inject_impossible_travel,
    "lateral_movement": _inject_lateral_movement,
    "device_spoofing": _inject_device_spoofing,
    "low_and_slow_exfiltration": _inject_low_and_slow_exfiltration,
    "insider_drift": _inject_insider_drift,
}

def _generate_all_events(
    profiles: List[EntityProfile],
    cfg: DataGenConfig,
    rng: np.random.Generator,
) -> Tuple[List[Dict[str, Any]], List[Dict[str, Any]]]:
    events: List[Dict[str, Any]] = []
```

```
    ground_truth: List[Dict[str, Any]] = []

    attack_rate = float(
        rng.uniform(cfg.attack_rate_min, cfg.attack_rate_max)
    )
    num_attacks = int(cfg.num_events * attack_rate)
    attack_indices = set(
        rng.choice(cfg.num_events, size=num_attacks, replace=False).tolist()
    )
    logger.info(
        "Attack budget: %d events (%.2f%% of %d)",
        num_attacks,
        attack_rate * 100,
        cfg.num_events,
    )

    attack_weights = _normalise_weights(cfg.attack_type_weights)

    start_date = datetime(2024, 1, 1, tzinfo=timezone.utc)
    day_offsets = rng.integers(0, cfg.simulation_days, size=cfg.num_events)

    for event_idx in range(cfg.num_events):
        profile = profiles[int(rng.integers(0, len(profiles)))]

        base_date = start_date + timedelta(days=int(day_offsets[event_idx]))

        timestamp = _sample_timestamp_for_persona(
            profile.entity_type, base_date, cfg, rng
        )

        event = _generate_normal_event(profile, timestamp, cfg, rng)

        is_attack = event_idx in attack_indices
        attack_type = "none"

        if is_attack:
            attack_type_idx = int(rng.choice(len(ATTACK_TYPES), p=attack_weights))
            attack_type = ATTACK_TYPES[attack_type_idx]
            injector = _ATTACK_INJECTORS[attack_type]
            event = injector(event, cfg, rng)

        events.append(event)
        ground_truth.append(
            {
                "entity_id": event["entity_id"],
                "timestamp": event["timestamp"],
                "is_attack": int(is_attack),
                "attack_type": attack_type,
            }
        )

        if (event_idx + 1) % 25_000 == 0:
            logger.info("Generated %d / %d events", event_idx + 1, cfg.num_events)

    return events, ground_truth

def _validate_dataset(
    df_logs: pd.DataFrame,
    df_truth: pd.DataFrame,
    cfg: DataGenConfig,
) -> None:
    errors: List[str] = []

    missing_log_cols = set(RAW_LOG_COLUMNS) - set(df_logs.columns)
    if missing_log_cols:
        errors.append(f"Missing columns in raw_logs: {missing_log_cols}")

    missing_gt_cols = set(GROUND_TRUTH_COLUMNS) - set(df_truth.columns)
    if missing_gt_cols:
        errors.append(f"Missing columns in ground_truth: {missing_gt_cols}")

    if len(df_logs) != len(df_truth):
        errors.append(
            f"Row count mismatch: logs={len(df_logs)}, truth={len(df_truth)}"
        )

    null_counts = df_logs.isnull().sum()
    cols_with_nulls = null_counts[null_counts > 0]
    if not cols_with_nulls.empty:
        errors.append(f"Null values detected in raw_logs:\n{cols_with_nulls}")
```

```
    attack_rate = df_truth["is_attack"].mean()
    tolerance = 0.005
    if attack_rate < (cfg.attack_rate_min - tolerance) or attack_rate > (
        cfg.attack_rate_max + tolerance
    ):
        errors.append(
            f"Attack rate {attack_rate:.4f} outside bounds "
            f"[{cfg.attack_rate_min}, {cfg.attack_rate_max}]"
        )

    valid_types = set(ENTITY_TYPES)
    invalid_types = set(df_logs["entity_type"].unique()) - valid_types
    if invalid_types:
        errors.append(f"Invalid entity types found: {invalid_types}")

    for etype in ENTITY_TYPES:
        mask = df_logs["entity_type"] == etype
        if mask.sum() == 0:
            continue
        auth_dist = df_logs.loc[mask, "auth_method"].value_counts(normalize=True)
        if etype == "service_account":
            biometric_rate = auth_dist.get("biometric", 0.0)
            if biometric_rate > 0.01:
                errors.append(
                    f"Service accounts have biometric auth rate={biometric_rate:.3f} "
                    f"(expected ~0)"
                )
        if etype == "corporate_employee":
            mfa_sso_rate = auth_dist.get("mfa", 0.0) + auth_dist.get("sso", 0.0)
            if mfa_sso_rate < 0.10:
                errors.append(
                    f"Corporate employees have low MFA+SSO rate={mfa_sso_rate:.3f}"
                )

    logger.info("Auth distribution validation complete.")

    corporate_res_set = set(cfg.corporate_resources)
    factory_res_set = set(cfg.factory_resources)
    service_res_set = set(cfg.service_resources)

    for etype, expected_res in [
        ("corporate_employee", corporate_res_set),
        ("factory_operator", factory_res_set),
        ("service_account", service_res_set),
    ]:
        mask = df_logs["entity_type"] == etype
        if mask.sum() == 0:
            continue
        non_attack_mask = mask & (df_truth["is_attack"] == 0)
        if non_attack_mask.sum() == 0:
            continue
        resources_used = set(df_logs.loc[non_attack_mask, "resource_accessed"].unique())
        cross_domain = resources_used - expected_res
        cross_domain_rate = (
            df_logs.loc[non_attack_mask, "resource_accessed"]
            .isin(cross_domain)
            .mean()
        )
        if cross_domain_rate > 0.05:
            errors.append(
                f"{etype} has {cross_domain_rate:.3f} cross-domain resource "
                f"access rate in normal events (expected <0.05)"
            )

    logger.info("Resource distribution validation complete.")

    attack_events = df_truth[df_truth["is_attack"] == 1]
    if len(attack_events) > 0:
        attack_dist = (
            attack_events["attack_type"]
            .value_counts(normalize=True)
            .to_dict()
        )
        expected_weights = _normalise_weights(cfg.attack_type_weights)
        for i, atype in enumerate(ATTACK_TYPES):
            actual = attack_dist.get(atype, 0.0)
            expected = float(expected_weights[i])
            if expected > 0.05 and actual < expected * 0.20:
                errors.append(
                    f"Attack type '{atype}' has rate={actual:.3f}, "
                    f"expected ~{expected:.3f} (too low)"
```

```
            )

    logger.info("Attack distribution validation complete.")

    for etype, expected_prefixes in [
        ("corporate_employee", ("10.1.", "10.200.", "192.168.1.")),
        ("factory_operator", ("10.10.", "10.20.")),
        ("service_account", ("10.100.0.", "10.100.1.")),
    ]:
        mask = df_logs["entity_type"] == etype
        non_attack_mask = mask & (df_truth["is_attack"] == 0)
        if non_attack_mask.sum() == 0:
            continue
        ips = df_logs.loc[non_attack_mask, "source_ip"]
        in_range = ips.apply(
            lambda ip: any(ip.startswith(p) for p in expected_prefixes)
        )
        in_range_rate = in_range.mean()
        if in_range_rate < 0.80:
            errors.append(
                f"{etype} has {in_range_rate:.3f} in-subnet IP rate "
                f"(expected >=0.80, prefixes={expected_prefixes})"
            )

    logger.info("IP range validation complete.")

    valid_geos = {loc[2] for loc in cfg.geo_locations}
    invalid_geos = set(df_logs["geo_location"].unique()) - valid_geos
    if invalid_geos:
        errors.append(f"Invalid geo_locations found: {invalid_geos}")

    all_valid_fps = (
        set(cfg.corporate_fingerprints)
        | set(cfg.factory_fingerprints)
        | set(cfg.service_fingerprints)
    )
    non_attack_mask = df_truth["is_attack"] == 0
    normal_fps = set(df_logs.loc[non_attack_mask, "device_fingerprint"].unique())
    invalid_fps = normal_fps - all_valid_fps
    if invalid_fps:
        errors.append(
            f"Non-attack events contain invalid fingerprints: {invalid_fps}"
        )

    if errors:
        error_msg = "Dataset validation FAILED:\n" + "\n".join(
            f"  [{i+1}] {e}" for i, e in enumerate(errors)
        )
        logger.error(error_msg)
        raise ValueError(error_msg)

    logger.info(
        "Validation PASSED -- %d events, attack rate: %.4f, %d checks OK",
        len(df_logs),
        attack_rate,
        12,
    )

def generate_dataset(config: Optional[ProjectConfig] = None) -> Tuple[pd.DataFrame, pd.DataFrame]:
    if config is None:
        config = get_project_config()

    cfg = config.data_gen
    paths = config.paths

    logger.info("Starting synthetic data generation...")
    _set_deterministic_seeds(cfg)
    rng = np.random.default_rng(cfg.numpy_seed)
    fake = _create_faker(cfg)

    global _GEO_COORDS
    _GEO_COORDS = {
        label: (lat, lon)
        for lat, lon, label in cfg.geo_locations
    }

    profiles = _create_entity_profiles(cfg, fake, rng)

    events, ground_truth = _generate_all_events(profiles, cfg, rng)

    test_window_start = datetime(2024, 1, 1, tzinfo=timezone.utc) + timedelta(days=28)
```

```
    demo_events = []
    demo_truth = []
    current_time = test_window_start

    for i in range(20):
        is_attack = 1 if i == 19 else 0
        attack_type = "impossible_travel" if is_attack else "none"

        event = {
            "timestamp": current_time.isoformat(),
            "entity_id": "cold_start_demo_user",
            "entity_type": "corporate_employee",
            "source_ip": "10.0.0.99" if not is_attack else "10.0.99.99",
            "geo_location": "New York" if not is_attack else "Moscow",
            "auth_method": "mfa" if not is_attack else "password",
            "action": "login",
            "status": "success",
            "device_fingerprint": cfg.corporate_fingerprints[0] if not is_attack else "Windows 95",
            "resource_accessed": cfg.corporate_resources[0],
            "command_sequence": "whoami",
            "session_duration": 60.0
        }
        truth = {
            "timestamp": current_time.isoformat(),
            "entity_id": "cold_start_demo_user",
            "is_attack": is_attack,
            "attack_type": attack_type
        }
        events.append(event)
        ground_truth.append(truth)
        current_time += timedelta(minutes=15)

    chain_entity_id = "chain_demo_user"
    chain_start = test_window_start + timedelta(days=8)
    baseline_event = {
        "entity_id": chain_entity_id,
        "entity_type": "corporate_employee",
        "source_ip": "10.0.0.77",
        "geo_location": "New York",
        "resource_accessed": cfg.corporate_resources[0],
        "auth_method": "mfa",
        "session_duration": 60.0,
        "command_sequence": "outlook;teams;whoami",
        "device_fingerprint": cfg.corporate_fingerprints[0],
    }
    for i in range(8):
        timestamp = chain_start + timedelta(days=i)
        event = {**baseline_event, "timestamp": timestamp.isoformat()}
        events.append(event)
        ground_truth.append({
            "timestamp": timestamp.isoformat(),
            "entity_id": chain_entity_id,
            "is_attack": 0,
            "attack_type": "none",
        })

    stage_start = chain_start + timedelta(days=7, hours=3)
    credential_attempt_fields = {
        "source_ip": "10.0.0.77",
        "geo_location": "New York",
        "resource_accessed": "vpn_gateway",
        "auth_method": "password",
        "session_duration": 0.1,
        "command_sequence": "auth_check;login_attempt",
        "device_fingerprint": cfg.corporate_fingerprints[0],
    }
    for attempt_index in range(5):
        timestamp = stage_start - timedelta(minutes=5 - attempt_index)
        events.append({
            "entity_id": chain_entity_id,
            "entity_type": "corporate_employee",
            "timestamp": timestamp.isoformat(),
            **credential_attempt_fields,
        })
        ground_truth.append({
            "timestamp": timestamp.isoformat(),
            "entity_id": chain_entity_id,
            "is_attack": 1,
            "attack_type": "credential_stuffing",
        })
```

```
chain_stages = [
    (
        "credential_stuffing",
        {
            "source_ip": "10.0.0.77",
            "geo_location": "New York",
            "resource_accessed": "vpn_gateway",
            "auth_method": "password",
            "session_duration": 0.1,
            "command_sequence": "auth_check;login_attempt;auth_check;login_attempt;auth_check;login_attempt",
            "device_fingerprint": cfg.corporate_fingerprints[0],
        },
    ),
    (
        "lateral_movement",
        {
            "source_ip": "10.0.0.77",
            "geo_location": "New York",
            "resource_accessed": "plc_controller",
            "auth_method": "password",
            "session_duration": 180.0,
            "command_sequence": "net_scan;port_scan;psexec;mimikatz;pass_the_hash;net_user",
            "device_fingerprint": cfg.corporate_fingerprints[0],
        },
    ),
    (
        "device_spoofing",
        {
            "source_ip": "10.0.0.77",
            "geo_location": "New York",
            "resource_accessed": "certificate_authority",
            "auth_method": "password",
            "session_duration": 240.0,
            "command_sequence": "sudo;runas;chmod;reg_query;wmic;whoami",
            "device_fingerprint": "Kali-Firefox-x64",
        },
    ),
    (
        "low_and_slow_exfiltration",
        {
            "source_ip": "10.0.0.77",
            "geo_location": "New York",
            "resource_accessed": "finance_dashboard",
            "auth_method": "password",
            "session_duration": 1440.0,
            "command_sequence": "compress;encrypt;split_file;scp;curl_upload;dns_tunnel;rsync;base64_encode;archive;stage_data",
            "device_fingerprint": cfg.corporate_fingerprints[0],
        },
    ),
]
for stage_index, (attack_type, stage_fields) in enumerate(chain_stages):
    timestamp = stage_start + timedelta(minutes=10 * stage_index)
    event = {
        "entity_id": chain_entity_id,
        "entity_type": "corporate_employee",
        "timestamp": timestamp.isoformat(),
        **stage_fields,
    }
    events.append(event)
    ground_truth.append({
        "timestamp": timestamp.isoformat(),
        "entity_id": chain_entity_id,
        "is_attack": 1,
        "attack_type": attack_type,
    })

df_logs = pd.DataFrame(events, columns=list(RAW_LOG_COLUMNS))
df_truth = pd.DataFrame(ground_truth, columns=list(GROUND_TRUTH_COLUMNS))

df_logs["timestamp"] = pd.to_datetime(df_logs["timestamp"], utc=True)
df_truth["timestamp"] = pd.to_datetime(df_truth["timestamp"], utc=True)

sort_idx = df_logs["timestamp"].argsort()
df_logs = df_logs.iloc[sort_idx].reset_index(drop=True)
df_truth = df_truth.iloc[sort_idx].reset_index(drop=True)

_validate_dataset(df_logs, df_truth, cfg)

paths.ensure_dirs()
df_logs.to_csv(paths.raw_logs_file, index=False)
df_truth.to_csv(paths.ground_truth_file, index=False)
```

```
    logger.info("Wrote raw_logs.csv (%d rows) to %s", len(df_logs), paths.raw_logs_file)
    logger.info(
        "Wrote ground_truth.csv (%d rows) to %s",
        len(df_truth),
        paths.ground_truth_file,
    )

    attack_counts = df_truth[df_truth["is_attack"] == 1]["attack_type"].value_counts()
    logger.info("Attack distribution:\n%s", attack_counts.to_string())
    entity_type_counts = df_logs["entity_type"].value_counts()
    logger.info("Entity type distribution:\n%s", entity_type_counts.to_string())

    return df_logs, df_truth

if __name__ == "__main__":
    logging.basicConfig(
        level=logging.INFO,
        format="%(asctime)s | %(name)s | %(levelname)s | %(message)s",
    )
    generate_dataset()
```

File: feature_pipeline.py

Description: Feature Engineering & Preprocessing Pipeline

```
"""Feature engineering and preprocessing pipeline extracting 19 behavioral, network, device, and relationship features.

Computes sliding window metrics (failed logins, travel speed, resource rarity, sequence entropy, privilege escalation count)
and scales numerical attributes using StandardScaler.
"""

from __future__ import annotations

import json
import logging
import math
import pickle
from collections import Counter
from pathlib import Path
from typing import Any, Dict, List, Optional, Tuple

import networkx as nx
import numpy as np
import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler, MinMaxScaler

from config import (
    ENTITY_TYPES,
    ENTITY_TYPE_TO_INDEX,
    GROUND_TRUTH_COLUMNS,
    NUM_ENTITY_TYPES,
    RAW_LOG_COLUMNS,
    FeatureConfig,
    PathConfig,
    ProjectConfig,
    get_project_config,
)

logger = logging.getLogger(__name__)

_GEO_COORDS: Dict[str, Tuple[float, float]] = {}

def _init_geo_coords(config: ProjectConfig) -> None:
    global _GEO_COORDS
    _GEO_COORDS = {
        label: (lat, lon)
        for lat, lon, label in config.data_gen.geo_locations
    }
    logger.debug("Geo-coordinate lookup initialised with %d locations.", len(_GEO_COORDS))

def _haversine_km(
    lat1: np.ndarray,
    lon1: np.ndarray,
    lat2: np.ndarray,
    lon2: np.ndarray,
    earth_radius_km: float,
) -> np.ndarray:
    lat1_r = np.radians(lat1)
    lat2_r = np.radians(lat2)
    dlat = np.radians(lat2 - lat1)
    dlon = np.radians(lon2 - lon1)

    a = np.sin(dlat / 2.0) ** 2 + np.cos(lat1_r) * np.cos(lat2_r) * np.sin(dlon / 2.0) ** 2
    c = 2.0 * np.arctan2(np.sqrt(a), np.sqrt(1.0 - a))
    return earth_radius_km * c

def _compute_behavioral_features(df: pd.DataFrame) -> pd.DataFrame:
    result = pd.DataFrame(index=df.index)
    result["login_hour"] = df["timestamp"].dt.hour.astype(np.float64)
    result["day_of_week"] = df["timestamp"].dt.dayofweek.astype(np.float64)
    result["session_duration"] = df["session_duration"].astype(np.float64)

    result["time_since_last_login"] = (
        df.groupby("entity_id")["timestamp"]
        .diff()
        .dt.total_seconds()
        .fillna(0.0)
        .astype(np.float64)
    )

    logger.info("Computed behavioural features: %s", list(result.columns))
```

```
    return result

def _compute_network_features(
    df: pd.DataFrame,
    feature_cfg: FeatureConfig,
) -> pd.DataFrame:
    n = len(df)
    unique_ips = np.zeros(n, dtype=np.float64)
    geo_dist = np.zeros(n, dtype=np.float64)
    failed_logins = np.zeros(n, dtype=np.float64)
    travel_speed = np.zeros(n, dtype=np.float64)

    ip_window_s = feature_cfg.unique_ip_window_seconds
    fail_window_s = feature_cfg.failed_login_window_seconds
    earth_r = feature_cfg.geo_distance_earth_radius_km

    ts_epoch = df["timestamp"].values.astype("datetime64[s]").astype(np.int64)

    failed_mask = (df["session_duration"].values < 0.5).astype(np.float64)

    for entity_id, group_idx in df.groupby("entity_id").groups.items():
        idx_arr = group_idx.values if hasattr(group_idx, "values") else np.array(group_idx)
        idx_arr = np.sort(idx_arr)

        entity_ts = ts_epoch[idx_arr]
        entity_ips = df["source_ip"].values[idx_arr]
        entity_geo = df["geo_location"].values[idx_arr]

        for j_pos, j in enumerate(idx_arr):
            current_ts = entity_ts[j_pos]

            window_start_ts = current_ts - ip_window_s
            past_mask = (entity_ts[:j_pos + 1] >= window_start_ts)
            past_ips = entity_ips[:j_pos + 1][past_mask]
            unique_ips[j] = float(len(set(past_ips)))

            if j_pos > 0:
                prev_geo_label = entity_geo[j_pos - 1]
                curr_geo_label = entity_geo[j_pos]
                prev_coords = _GEO_COORDS.get(prev_geo_label)
                curr_coords = _GEO_COORDS.get(curr_geo_label)
                if prev_coords is not None and curr_coords is not None:
                    dist = float(
                        _haversine_km(
                            np.array([prev_coords[0]]),
                            np.array([prev_coords[1]]),
                            np.array([curr_coords[0]]),
                            np.array([curr_coords[1]]),
                            earth_r,
                        )[0]
                    )
                    geo_dist[j] = dist
                    time_delta_sec = current_ts - entity_ts[j_pos - 1]
                    time_delta_hours = time_delta_sec / 3600.0
                    travel_speed[j] = dist / max(time_delta_hours, 1e-5)

            fail_window_start = current_ts - fail_window_s
            fail_past_mask = (entity_ts[:j_pos + 1] >= fail_window_start)
            fail_subset_indices = idx_arr[:j_pos + 1][fail_past_mask]
            failed_logins[j] = float(failed_mask[fail_subset_indices].sum())

    result = pd.DataFrame(
        {
            "unique_ips_last_hour": unique_ips,
            "geo_distance_km": geo_dist,
            "failed_logins_last_10min": failed_logins,
            "travel_speed_kmh": travel_speed,
        },
        index=df.index,
    )

    logger.info("Computed network features: %s", list(result.columns))
    return result

def _compute_shannon_entropy(sequence_str: str, base: int) -> float:
    if not sequence_str or not isinstance(sequence_str, str):
        return 0.0
    tokens = sequence_str.split(";")
    if len(tokens) <= 1:
        return 0.0
    counts = Counter(tokens)
```

```
    total = len(tokens)
    entropy = 0.0
    log_base = math.log(base) if base != math.e else 1.0
    for count in counts.values():
        p = count / total
        if p > 0:
            entropy -= p * (math.log(p) / log_base)
    return entropy

def _compute_device_resource_features(
    df: pd.DataFrame,
    feature_cfg: FeatureConfig,
) -> pd.DataFrame:
    n = len(df)
    fp_consistency = np.zeros(n, dtype=np.float64)
    resource_rarity = np.zeros(n, dtype=np.float64)
    seq_entropy = np.zeros(n, dtype=np.float64)
    priv_esc_count = np.zeros(n, dtype=np.float64)
    os_mismatch = np.zeros(n, dtype=np.float64)
    browser_mismatch = np.zeros(n, dtype=np.float64)

    entropy_base = feature_cfg.sequence_entropy_base

    priv_esc_keywords = frozenset({
        "chmod", "sudo", "su", "runas", "psexec", "mimikatz",
        "pass_the_hash", "net_user", "reg_query", "wmic",
    })

    seq_entropy = (
        df["command_sequence"]
        .apply(lambda s: _compute_shannon_entropy(s, entropy_base))
        .values.astype(np.float64)
    )

    resource_counter: Counter = Counter()
    total_events_so_far: int = 0

    entity_fp_history: Dict[str, List[str]] = {}
    entity_priv_esc: Dict[str, int] = {}
    entity_seen_os: Dict[str, set] = {}
    entity_seen_browser: Dict[str, set] = {}

    for i in range(n):
        entity_id = df.iloc[i]["entity_id"]
        fingerprint = df.iloc[i]["device_fingerprint"]
        resource = df.iloc[i]["resource_accessed"]
        cmd_seq = df.iloc[i]["command_sequence"]

        if entity_id not in entity_fp_history:
            entity_fp_history[entity_id] = []
            entity_seen_os[entity_id] = set()
            entity_seen_browser[entity_id] = set()

        history = entity_fp_history[entity_id]
        if len(history) > 0:
            match_count = sum(1 for fp in history if fp == fingerprint)
            fp_consistency[i] = match_count / len(history)
        else:
            fp_consistency[i] = 1.0

        history.append(fingerprint)

        parts = str(fingerprint).split("-")
        current_os = parts[0] if len(parts) > 0 else "unknown"
        current_browser = parts[1] if len(parts) > 1 else "unknown"

        if len(entity_seen_os[entity_id]) > 0:
            if current_os not in entity_seen_os[entity_id]:
                os_mismatch[i] = 1.0
            if current_browser not in entity_seen_browser[entity_id]:
                browser_mismatch[i] = 1.0

        entity_seen_os[entity_id].add(current_os)
        entity_seen_browser[entity_id].add(current_browser)

        total_events_so_far += 1
        resource_counter[resource] += 1
        resource_freq = resource_counter[resource] / total_events_so_far
        resource_rarity[i] = 1.0 - resource_freq

        if entity_id not in entity_priv_esc:
```

```
            entity_priv_esc[entity_id] = 0
        if isinstance(cmd_seq, str):
            tokens = cmd_seq.split(";")
            esc_count = sum(1 for t in tokens if t.strip() in priv_esc_keywords)
            entity_priv_esc[entity_id] += esc_count
        priv_esc_count[i] = float(entity_priv_esc[entity_id])

    result = pd.DataFrame(
        {
            "fingerprint_consistency": fp_consistency,
            "resource_rarity": resource_rarity,
            "sequence_entropy": seq_entropy,
            "privilege_escalation_count": priv_esc_count,
            "os_mismatch": os_mismatch,
            "browser_mismatch": browser_mismatch,
        },
        index=df.index,
    )

    logger.info("Computed device & resource features: %s", list(result.columns))
    return result

def _compute_relationship_features(df: pd.DataFrame) -> pd.DataFrame:
    n = len(df)
    degree_centrality = np.zeros(n, dtype=np.float64)
    newly_observed = np.zeros(n, dtype=np.float64)

    graph = nx.Graph()
    entity_neighbors: Dict[str, set] = {}

    for i in range(n):
        entity_id = df.iloc[i]["entity_id"]
        resource = df.iloc[i]["resource_accessed"]

        if entity_id not in entity_neighbors:
            entity_neighbors[entity_id] = set()

        if resource not in entity_neighbors[entity_id]:
            newly_observed[i] = 1.0
            entity_neighbors[entity_id].add(resource)

        graph.add_edge(entity_id, resource)

        num_nodes = graph.number_of_nodes()
        if num_nodes > 1:
            entity_degree = graph.degree(entity_id)
            degree_centrality[i] = entity_degree / (num_nodes - 1)
        else:
            degree_centrality[i] = 0.0

    result = pd.DataFrame(
        {
            "degree_centrality": degree_centrality,
            "newly_observed_neighbors": newly_observed,
        },
        index=df.index,
    )

    logger.info("Computed relationship features: %s", list(result.columns))
    return result

def _encode_entity_type(
    df: pd.DataFrame,
) -> Tuple[np.ndarray, LabelEncoder]:
    le = LabelEncoder()
    le.fit(list(ENTITY_TYPES))
    encoded = le.transform(df["entity_type"].values)

    one_hot = np.zeros((len(encoded), NUM_ENTITY_TYPES), dtype=np.float64)
    one_hot[np.arange(len(encoded)), encoded] = 1.0

    logger.info(
        "Encoded entity_type -- classes: %s, one-hot shape: %s",
        list(le.classes_),
        one_hot.shape,
    )
    return one_hot, le

def _assemble_features(
    entity_type_onehot: np.ndarray,
    behavioral_df: pd.DataFrame,
```

```
    network_df: pd.DataFrame,
    device_resource_df: pd.DataFrame,
    relationship_df: pd.DataFrame,
) -> Tuple[np.ndarray, List[str]]:
    onehot_names = [f"entity_type_{etype}" for etype in ENTITY_TYPES]

    feature_names = (
        onehot_names
        + list(behavioral_df.columns)
        + list(network_df.columns)
        + list(device_resource_df.columns)
        + list(relationship_df.columns)
    )

    feature_matrix = np.hstack(
        [
            entity_type_onehot,
            behavioral_df.values.astype(np.float64),
            network_df.values.astype(np.float64),
            device_resource_df.values.astype(np.float64),
            relationship_df.values.astype(np.float64),
        ]
    )

    feature_matrix = np.nan_to_num(feature_matrix, nan=0.0, posinf=0.0, neginf=0.0)

    logger.info(
        "Assembled feature matrix: shape=%s, features=%d",
        feature_matrix.shape,
        len(feature_names),
    )
    return feature_matrix, feature_names

def _temporal_split(
    X: np.ndarray,
    y: np.ndarray,
    feature_cfg: FeatureConfig,
) -> Tuple[
    np.ndarray, np.ndarray, np.ndarray,
    np.ndarray, np.ndarray, np.ndarray,
]:
    n = len(X)
    train_end = int(n * feature_cfg.train_ratio)
    val_end = train_end + int(n * feature_cfg.val_ratio)

    X_train = X[:train_end]
    X_val = X[train_end:val_end]
    X_test = X[val_end:]

    y_train = y[:train_end]
    y_val = y[train_end:val_end]
    y_test = y[val_end:]

    logger.info(
        "Temporal split -- train: %d, val: %d, test: %d",
        len(X_train),
        len(X_val),
        len(X_test),
    )
    return X_train, X_val, X_test, y_train, y_val, y_test

def _fit_and_transform_scaler(
    X_train: np.ndarray,
    X_val: np.ndarray,
    X_test: np.ndarray,
    scaler_type: str,
) -> Tuple[np.ndarray, np.ndarray, np.ndarray, Any]:
    if scaler_type == "standard":
        scaler = StandardScaler()
    elif scaler_type == "minmax":
        scaler = MinMaxScaler()
    else:
        raise ValueError(f"Unsupported scaler type: {scaler_type}")

    X_train_scaled = scaler.fit_transform(X_train)
    X_val_scaled = scaler.transform(X_val)
    X_test_scaled = scaler.transform(X_test)

    logger.info("Fitted and applied %s scaler.", scaler_type)
    return X_train_scaled, X_val_scaled, X_test_scaled, scaler
```

```
def _save_artefacts(
    X_train: np.ndarray,
    X_val: np.ndarray,
    X_test: np.ndarray,
    y_train: np.ndarray,
    y_val: np.ndarray,
    y_test: np.ndarray,
    feature_names: List[str],
    scaler: Any,
    encoder: LabelEncoder,
    paths: PathConfig,
    feature_cfg: FeatureConfig,
) -> None:
    out_dir = paths.processed_data_dir
    out_dir.mkdir(parents=True, exist_ok=True)

    np.save(out_dir / feature_cfg.x_train_file, X_train)
    np.save(out_dir / feature_cfg.x_val_file, X_val)
    np.save(out_dir / feature_cfg.x_test_file, X_test)

    np.save(out_dir / feature_cfg.y_train_file, y_train)
    np.save(out_dir / feature_cfg.y_val_file, y_val)
    np.save(out_dir / feature_cfg.y_test_file, y_test)

    metadata = {
        "feature_names": feature_names,
        "num_features": len(feature_names),
        "num_entity_type_features": NUM_ENTITY_TYPES,
        "train_samples": int(X_train.shape[0]),
        "val_samples": int(X_val.shape[0]),
        "test_samples": int(X_test.shape[0]),
        "scaler_type": feature_cfg.scaler_type,
        "feature_groups": {
            "entity_type_onehot": [
                f"entity_type_{etype}" for etype in ENTITY_TYPES
            ],
            "behavioral": list(feature_cfg.behavioral_features),
            "network": list(feature_cfg.network_features),
            "device_resource": list(feature_cfg.device_resource_features),
            "relationship": list(feature_cfg.relationship_features),
        },
    }
    metadata_path = out_dir / feature_cfg.feature_metadata_file
    with open(metadata_path, "w", encoding="utf-8") as f:
        json.dump(metadata, f, indent=2)

    scaler_path = out_dir / feature_cfg.scaler_file
    with open(scaler_path, "wb") as f:
        pickle.dump(scaler, f)

    encoder_path = out_dir / feature_cfg.encoder_file
    with open(encoder_path, "wb") as f:
        pickle.dump(encoder, f)

    logger.info("Saved all artefacts to %s", out_dir)
    logger.info(
        "  Feature arrays: X_train=%s, X_val=%s, X_test=%s",
        X_train.shape,
        X_val.shape,
        X_test.shape,
    )
    logger.info(
        "  Label arrays: y_train=%s, y_val=%s, y_test=%s",
        y_train.shape,
        y_val.shape,
        y_test.shape,
    )

def run_feature_pipeline(
    config: Optional[ProjectConfig] = None,
) -> Dict[str, Any]:
    if config is None:
        config = get_project_config()

    paths = config.paths
    feature_cfg = config.features

    logger.info("Starting feature engineering pipeline...")

    df_logs = pd.read_csv(paths.raw_logs_file, parse_dates=["timestamp"])
    df_truth = pd.read_csv(paths.ground_truth_file, parse_dates=["timestamp"])
```

```
    logger.info("Loaded raw_logs: %d rows, ground_truth: %d rows", len(df_logs), len(df_truth))

    df_logs = df_logs.sort_values("timestamp").reset_index(drop=True)
    df_truth = df_truth.sort_values("timestamp").reset_index(drop=True)

    if df_logs["timestamp"].dt.tz is None:
        df_logs["timestamp"] = df_logs["timestamp"].dt.tz_localize("UTC")
    if df_truth["timestamp"].dt.tz is None:
        df_truth["timestamp"] = df_truth["timestamp"].dt.tz_localize("UTC")

    _init_geo_coords(config)

    logger.info("Computing behavioural features...")
    behavioral_df = _compute_behavioral_features(df_logs)

    logger.info("Computing network features...")
    network_df = _compute_network_features(df_logs, feature_cfg)

    logger.info("Computing device & resource features...")
    device_resource_df = _compute_device_resource_features(df_logs, feature_cfg)

    logger.info("Computing relationship features...")
    relationship_df = _compute_relationship_features(df_logs)

    entity_type_onehot, encoder = _encode_entity_type(df_logs)

    X, feature_names = _assemble_features(
        entity_type_onehot,
        behavioral_df,
        network_df,
        device_resource_df,
        relationship_df,
    )

    y = df_truth["is_attack"].values.astype(np.float64)

    X_train, X_val, X_test, y_train, y_val, y_test = _temporal_split(
        X, y, feature_cfg
    )

    X_train, X_val, X_test, scaler = _fit_and_transform_scaler(
        X_train, X_val, X_test, feature_cfg.scaler_type
    )

    _save_artefacts(
        X_train, X_val, X_test,
        y_train, y_val, y_test,
        feature_names, scaler, encoder,
        paths, feature_cfg,
    )

    logger.info("Feature pipeline completed successfully.")

    return {
        "X_train": X_train,
        "X_val": X_val,
        "X_test": X_test,
        "y_train": y_train,
        "y_val": y_val,
        "y_test": y_test,
        "feature_names": feature_names,
        "scaler": scaler,
        "encoder": encoder,
    }

if __name__ == "__main__":
    logging.basicConfig(
        level=logging.INFO,
        format="%(asctime)s | %(name)s | %(levelname)s | %(message)s",
    )
    run_feature_pipeline()
```

File: models.py

Description: Autoencoder, LSTM & Risk Scorer Architectures

```
"""Core neural network models, Policy Engine, Platt Scaler, and Hybrid Risk Scorer.

Contains:
- SharedAutoencoder: Deep reconstruction neural network for unsupervised anomaly detection.
- LSTMSequenceClassifier: Recurrent neural network with Focal Loss for temporal sequence classification.
- PolicyEngine: Domain-specific security rule scoring engine.
- HybridRiskScorer: Weighted risk score aggregator (R = w_ae*AE + w_lstm*LSTM + w_policy*Policy).
"""

from __future__ import annotations

import json
import logging
from pathlib import Path
from typing import Any, Dict, List, Optional, Tuple, Union

import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from captum.attr import IntegratedGradients
from torch.utils.data import DataLoader, TensorDataset

from config import (
    ENTITY_TYPES,
    AutoencoderConfig,
    FocalLossConfig,
    AttackClassifierConfig,
    ModelConfig,
    PolicyEngineConfig,
    PolicyRule,
    ProjectConfig,
    RiskScorerConfig,
    TrainingConfig,
    get_project_config,
)

logger = logging.getLogger(__name__)

def set_torch_seed(seed: int) -> None:
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False
    logger.debug("PyTorch seeds set to %d", seed)

def _get_device(device_str: str) -> torch.device:
    if device_str == "cuda" and not torch.cuda.is_available():
        logger.warning("CUDA requested but not available. Falling back to CPU.")
        return torch.device("cpu")
    if device_str == "mps" and not (
        hasattr(torch.backends, "mps") and torch.backends.mps.is_available()
    ):
        logger.warning("MPS requested but not available. Falling back to CPU.")
        return torch.device("cpu")
    return torch.device(device_str)

class FocalLoss(nn.Module):

    def __init__(self, alpha: float = 0.75, gamma: float = 2.0) -> None:
        super().__init__()
        self.alpha: float = alpha
        self.gamma: float = gamma

    def forward(
        self, logits: torch.Tensor, targets: torch.Tensor
    ) -> torch.Tensor:
        probs = torch.sigmoid(logits)
        probs = torch.clamp(probs, min=1e-7, max=1.0 - 1e-7)

        bce_loss = F.binary_cross_entropy_with_logits(logits, targets, reduction="none")
        pt = torch.exp(-bce_loss)
        focal_loss = self.alpha * (1 - pt) ** self.gamma * bce_loss

        return focal_loss.mean()
```

```
    @classmethod
    def from_config(cls, cfg: FocalLossConfig) -> "FocalLoss":
        return cls(alpha=cfg.alpha, gamma=cfg.gamma)

class MultiClassFocalLoss(nn.Module):

    def __init__(
        self,
        alpha: float = 0.75,
        gamma: float = 2.0,
        reduction: str = "mean",
        class_weights: Optional[torch.Tensor] = None,
    ) -> None:
        super().__init__()
        self.alpha = alpha
        self.gamma = gamma
        self.reduction = reduction
        if class_weights is not None:
            self.register_buffer("class_weights", class_weights)
        else:
            self.class_weights = None

    def forward(self, inputs: torch.Tensor, targets: torch.Tensor) -> torch.Tensor:
        ce_loss = nn.functional.cross_entropy(
            inputs, targets, weight=self.class_weights, reduction="none"
        )
        pt = torch.exp(-ce_loss)
        focal_loss = self.alpha * (1 - pt) ** self.gamma * ce_loss

        if self.reduction == "mean":
            return focal_loss.mean()
        elif self.reduction == "sum":
            return focal_loss.sum()
        return focal_loss

    @staticmethod
    def compute_class_weights(
        labels: np.ndarray,
        num_classes: int,
        max_ratio: float = 10.0,
    ) -> torch.Tensor:
        counts = np.bincount(labels.astype(int), minlength=num_classes).astype(np.float64)
        counts = np.maximum(counts, 1.0)
        inv_sqrt = 1.0 / np.sqrt(counts)
        inv_sqrt /= inv_sqrt.min()
        inv_sqrt = np.clip(inv_sqrt, 1.0, max_ratio)
        return torch.tensor(inv_sqrt, dtype=torch.float32)

    @classmethod
    def from_config(cls, cfg: AttackClassifierConfig) -> "MultiClassFocalLoss":
        return cls(alpha=cfg.focal_alpha, gamma=cfg.focal_gamma)

class SharedAutoencoder(nn.Module):

    def __init__(
        self,
        input_dim: int,
        cfg: AutoencoderConfig,
    ) -> None:
        super().__init__()
        self.input_dim: int = input_dim
        self.cfg: AutoencoderConfig = cfg

        encoder_layers: List[nn.Module] = []
        prev_dim = input_dim
        for dim in cfg.encoder_dims:
            encoder_layers.append(nn.Linear(prev_dim, dim))
            encoder_layers.append(nn.BatchNorm1d(dim))
            encoder_layers.append(self._get_activation(cfg.activation))
            encoder_layers.append(nn.Dropout(cfg.dropout_rate))
            prev_dim = dim
        encoder_layers.append(nn.Linear(prev_dim, cfg.latent_dim))
        self.encoder = nn.Sequential(*encoder_layers)

        decoder_layers: List[nn.Module] = []
        prev_dim = cfg.latent_dim
        for dim in cfg.decoder_dims:
            decoder_layers.append(nn.Linear(prev_dim, dim))
            decoder_layers.append(nn.BatchNorm1d(dim))
            decoder_layers.append(self._get_activation(cfg.activation))
```

```
            decoder_layers.append(nn.Dropout(cfg.dropout_rate))
            prev_dim = dim
        decoder_layers.append(nn.Linear(prev_dim, input_dim))
        self.decoder = nn.Sequential(*decoder_layers)

        self.thresholds: Dict[str, float] = {}

    @staticmethod
    def _get_activation(name: str) -> nn.Module:
        activations = {
            "relu": nn.ReLU(),
            "elu": nn.ELU(),
            "leaky_relu": nn.LeakyReLU(),
            "tanh": nn.Tanh(),
        }
        if name not in activations:
            raise ValueError(
                f"Unsupported activation: {name}. Choose from {list(activations.keys())}"
            )
        return activations[name]

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        latent = self.encoder(x)
        reconstructed = self.decoder(latent)
        return reconstructed

    def encode(self, x: torch.Tensor) -> torch.Tensor:
        return self.encoder(x)

    def compute_reconstruction_error(self, x: torch.Tensor) -> torch.Tensor:
        reconstructed = self.forward(x)
        mse = ((x - reconstructed) ** 2).mean(dim=1)
        return mse

    def compute_adaptive_thresholds(
        self,
        X: np.ndarray,
        entity_types: np.ndarray,
        percentile: float,
        device: torch.device,
    ) -> Dict[str, float]:
        self.eval()
        X_tensor = torch.tensor(X, dtype=torch.float32, device=device)

        with torch.no_grad():
            errors = self.compute_reconstruction_error(X_tensor).cpu().numpy()

        thresholds: Dict[str, float] = {}
        for etype in ENTITY_TYPES:
            mask = entity_types == etype
            if mask.sum() > 0:
                threshold = float(np.percentile(errors[mask], percentile))
                thresholds[etype] = threshold
                logger.info(
                    "Threshold for %s: %.6f (p%.0f, n=%d)",
                    etype,
                    threshold,
                    percentile,
                    mask.sum(),
                )
            else:
                thresholds[etype] = float(np.percentile(errors, percentile))
                logger.warning(
                    "No samples for %s; using global threshold %.6f",
                    etype,
                    thresholds[etype],
                )

        self.thresholds = thresholds
        return thresholds

    @classmethod
    def from_config(cls, input_dim: int, cfg: AutoencoderConfig) -> "SharedAutoencoder":
        return cls(input_dim=input_dim, cfg=cfg)

class LSTMSequenceClassifier(nn.Module):

    def __init__(
        self,
        input_dim: int,
        cfg: LSTMConfig,
```

```
    ) -> None:
        super().__init__()
        self.input_dim: int = input_dim
        self.cfg: LSTMConfig = cfg

        self.lstm = nn.LSTM(
            input_size=input_dim,
            hidden_size=cfg.hidden_dim,
            num_layers=cfg.num_layers,
            batch_first=True,
            bidirectional=cfg.bidirectional,
            dropout=cfg.dropout_rate if cfg.num_layers > 1 else 0.0,
        )

        lstm_output_dim = cfg.hidden_dim * (2 if cfg.bidirectional else 1)

        fc_layers: List[nn.Module] = []
        prev_dim = lstm_output_dim
        for dim in cfg.fc_dims:
            fc_layers.append(nn.Linear(prev_dim, dim))
            fc_layers.append(nn.BatchNorm1d(dim))
            fc_layers.append(nn.ReLU())
            fc_layers.append(nn.Dropout(cfg.dropout_rate))
            prev_dim = dim
        self.fc_layers = nn.Sequential(*fc_layers)

        self.output_layer = nn.Linear(prev_dim, 1)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        lstm_out, _ = self.lstm(x)

        last_hidden = lstm_out[:, -1, :]

        fc_out = self.fc_layers(last_hidden)

        logits = self.output_layer(fc_out).squeeze(-1)
        return logits

    def predict_proba(self, x: torch.Tensor) -> torch.Tensor:
        logits = self.forward(x)
        return torch.sigmoid(logits)

    def extract_hidden(self, x: torch.Tensor) -> torch.Tensor:
        lstm_out, _ = self.lstm(x)
        return lstm_out[:, -1, :]

    @classmethod
    def from_config(cls, input_dim: int, cfg: LSTMConfig) -> "LSTMSequenceClassifier":
        return cls(input_dim=input_dim, cfg=cfg)

class AttackTypeClassifier(nn.Module):

    def __init__(self, input_dim: int, cfg: AttackClassifierConfig) -> None:
        super().__init__()
        self.input_dim = input_dim
        self.cfg = cfg

        layers: List[nn.Module] = []
        prev_dim = input_dim
        for dim in cfg.fc_dims:
            layers.append(nn.Linear(prev_dim, dim))
            layers.append(nn.BatchNorm1d(dim))
            layers.append(nn.ReLU())
            layers.append(nn.Dropout(cfg.dropout_rate))
            prev_dim = dim

        layers.append(nn.Linear(prev_dim, cfg.num_classes))
        self.network = nn.Sequential(*layers)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.network(x)

    @classmethod
    def from_config(cls, input_dim: int, cfg: AttackClassifierConfig) -> "AttackTypeClassifier":
        return cls(input_dim=input_dim, cfg=cfg)

class PolicyEngine:

    _OPERATORS = {
        "gt": lambda x, t: x > t,
        "lt": lambda x, t: x < t,
```

```
        "ge": lambda x, t: x >= t,
        "le": lambda x, t: x <= t,
        "eq": lambda x, t: np.isclose(x, t),
        "ne": lambda x, t: ~np.isclose(x, t),
    }

    def __init__(self, cfg: PolicyEngineConfig) -> None:
        self.rules: Tuple[PolicyRule, ...] = cfg.rules
        self.max_score: float = cfg.max_score
        logger.info(
            "PolicyEngine initialised with %d rules, max_score=%.2f",
            len(self.rules),
            self.max_score,
        )

    def evaluate(
        self,
        features: Dict[str, np.ndarray],
    ) -> np.ndarray:
        n_samples = next(iter(features.values())).shape[0]
        raw_scores = np.zeros(n_samples, dtype=np.float64)

        for rule in self.rules:
            if rule.feature not in features:
                logger.warning(
                    "Feature '%s' not found for rule '%s'. Skipping.",
                    rule.feature,
                    rule.name,
                )
                continue

            feature_values = features[rule.feature]
            operator_fn = self._OPERATORS.get(rule.operator)
            if operator_fn is None:
                logger.warning(
                    "Unknown operator '%s' in rule '%s'. Skipping.",
                    rule.operator,
                    rule.name,
                )
                continue

            mask = operator_fn(feature_values, rule.threshold)
            raw_scores[mask] += rule.score

        normalised = np.clip(raw_scores / self.max_score, 0.0, 1.0)
        return normalised

    def evaluate_single(self, features: Dict[str, float]) -> float:
        raw_score = 0.0
        for rule in self.rules:
            if rule.feature not in features:
                continue
            val = features[rule.feature]
            operator_fn = self._OPERATORS.get(rule.operator)
            if operator_fn is None:
                continue
            if operator_fn(np.float64(val), rule.threshold):
                raw_score += rule.score
        return min(raw_score / self.max_score, 1.0)

    @classmethod
    def from_config(cls, cfg: PolicyEngineConfig) -> "PolicyEngine":
        return cls(cfg=cfg)

class RiskScorer:

    def __init__(self, cfg: RiskScorerConfig) -> None:
        self.w_ae: float = cfg.w_ae
        self.w_lstm: float = cfg.w_lstm
        self.w_policy: float = cfg.w_policy

        total = self.w_ae + self.w_lstm + self.w_policy
        if not np.isclose(total, 1.0):
            logger.warning(
                "Risk scorer weights sum to %.4f, not 1.0. "
                "Scores will still be computed but may exceed [0, 1].",
                total,
            )

    def compute(
        self,
```

```
        ae_errors: np.ndarray,
        lstm_probs: np.ndarray,
        policy_scores: np.ndarray,
        ae_min: float = 0.0,
        ae_max: float = 1.0,
    ) -> np.ndarray:
        if ae_max > ae_min:
            normalised_ae = (ae_errors - ae_min) / (ae_max - ae_min)
        else:
            normalised_ae = np.zeros_like(ae_errors)
        normalised_ae = np.clip(normalised_ae, 0.0, 1.0)

        normalised_lstm = np.clip(lstm_probs, 0.0, 1.0)

        normalised_policy = np.clip(policy_scores, 0.0, 1.0)

        risk = (
            self.w_ae * normalised_ae
            + self.w_lstm * normalised_lstm
            + self.w_policy * normalised_policy
        )

        return np.clip(risk, 0.0, 1.0)

    def compute_single(
        self,
        ae_error: float,
        lstm_prob: float,
        policy_score: float,
        ae_min: float,
        ae_max: float,
    ) -> float:
        if ae_max > ae_min:
            norm_ae = (ae_error - ae_min) / (ae_max - ae_min)
        else:
            norm_ae = 0.0
        norm_ae = max(0.0, min(1.0, norm_ae))
        norm_lstm = max(0.0, min(1.0, lstm_prob))
        norm_policy = max(0.0, min(1.0, policy_score))

        risk = self.w_ae * norm_ae + self.w_lstm * norm_lstm + self.w_policy * norm_policy
        return max(0.0, min(1.0, risk))

    def update_weights(self, w_ae: float, w_lstm: float, w_policy: float) -> None:
        self.w_ae = w_ae
        self.w_lstm = w_lstm
        self.w_policy = w_policy

    @classmethod
    def from_config(cls, cfg: RiskScorerConfig) -> "RiskScorer":
        return cls(cfg=cfg)

def create_sequences(
    X: np.ndarray,
    y: np.ndarray,
    sequence_length: int,
) -> Tuple[np.ndarray, np.ndarray]:
    n_samples = len(X)
    if n_samples < sequence_length:
        logger.warning(
            "Not enough samples (%d) for sequence_length (%d). "
            "Returning empty arrays.",
            n_samples,
            sequence_length,
        )
        return (
            np.empty((0, sequence_length, X.shape[1]), dtype=np.float64),
            np.empty(0, dtype=np.float64),
        )

    n_sequences = n_samples - sequence_length + 1
    X_seq = np.zeros(
        (n_sequences, sequence_length, X.shape[1]), dtype=np.float64
    )
    y_seq = np.zeros(n_sequences, dtype=np.float64)

    for i in range(n_sequences):
        X_seq[i] = X[i : i + sequence_length]
        y_seq[i] = y[i + sequence_length - 1]

    logger.info(
```

```
        "Created %d sequences of length %d from %d samples.",
        n_sequences,
        sequence_length,
        n_samples,
    )
    return X_seq, y_seq

def train_autoencoder(
    model: SharedAutoencoder,
    X_train: np.ndarray,
    X_val: np.ndarray,
    training_cfg: TrainingConfig,
) -> Dict[str, List[float]]:
    device = _get_device(training_cfg.device)
    model = model.to(device)

    train_tensor = torch.tensor(X_train, dtype=torch.float32)
    val_tensor = torch.tensor(X_val, dtype=torch.float32)

    train_dataset = TensorDataset(train_tensor, train_tensor)
    val_dataset = TensorDataset(val_tensor, val_tensor)

    train_loader = DataLoader(
        train_dataset,
        batch_size=training_cfg.batch_size,
        shuffle=True,
        num_workers=training_cfg.num_workers,
        pin_memory=training_cfg.pin_memory,
    )
    val_loader = DataLoader(
        val_dataset,
        batch_size=training_cfg.batch_size,
        shuffle=False,
        num_workers=training_cfg.num_workers,
        pin_memory=training_cfg.pin_memory,
    )

    optimizer = torch.optim.Adam(
        model.parameters(),
        lr=training_cfg.learning_rate_ae,
        weight_decay=training_cfg.weight_decay,
    )
    criterion = nn.MSELoss()

    history: Dict[str, List[float]] = {"train_losses": [], "val_losses": []}
    best_val_loss = float("inf")
    patience_counter = 0
    best_state_dict = None

    logger.info(
        "Training SharedAutoencoder -- epochs: %d, batch_size: %d, device: %s",
        training_cfg.num_epochs_ae,
        training_cfg.batch_size,
        device,
    )

    for epoch in range(training_cfg.num_epochs_ae):
        model.train()
        train_loss_sum = 0.0
        train_batches = 0
        for batch_x, batch_target in train_loader:
            batch_x = batch_x.to(device)
            batch_target = batch_target.to(device)

            optimizer.zero_grad()
            reconstructed = model(batch_x)
            loss = criterion(reconstructed, batch_target)
            loss.backward()
            torch.nn.utils.clip_grad_norm_(
                model.parameters(), training_cfg.gradient_clip_max_norm
            )
            optimizer.step()

            train_loss_sum += loss.item()
            train_batches += 1

        avg_train_loss = train_loss_sum / max(train_batches, 1)

        model.eval()
        val_loss_sum = 0.0
        val_batches = 0
```

```
        with torch.no_grad():
            for batch_x, batch_target in val_loader:
                batch_x = batch_x.to(device)
                batch_target = batch_target.to(device)
                reconstructed = model(batch_x)
                loss = criterion(reconstructed, batch_target)
                val_loss_sum += loss.item()
                val_batches += 1

        avg_val_loss = val_loss_sum / max(val_batches, 1)

        history["train_losses"].append(avg_train_loss)
        history["val_losses"].append(avg_val_loss)

        logger.info(
            "Epoch %d/%d -- train_loss: %.6f, val_loss: %.6f",
            epoch + 1,
            training_cfg.num_epochs_ae,
            avg_train_loss,
            avg_val_loss,
        )

        if avg_val_loss < best_val_loss:
            best_val_loss = avg_val_loss
            patience_counter = 0
            best_state_dict = {k: v.cpu().clone() for k, v in model.state_dict().items()}
        else:
            patience_counter += 1
            if patience_counter >= training_cfg.early_stopping_patience:
                logger.info(
                    "Early stopping at epoch %d (patience=%d)",
                    epoch + 1,
                    training_cfg.early_stopping_patience,
                )
                break

    if best_state_dict is not None:
        model.load_state_dict(best_state_dict)
        model = model.to(device)
        logger.info("Restored best model weights (val_loss=%.6f)", best_val_loss)

    return history

def train_lstm(
    model: LSTMSequenceClassifier,
    X_train_seq: np.ndarray,
    y_train_seq: np.ndarray,
    X_val_seq: np.ndarray,
    y_val_seq: np.ndarray,
    focal_loss: FocalLoss,
    training_cfg: TrainingConfig,
) -> Dict[str, List[float]]:
    device = _get_device(training_cfg.device)
    model = model.to(device)
    focal_loss = focal_loss.to(device)

    train_dataset = TensorDataset(
        torch.tensor(X_train_seq, dtype=torch.float32),
        torch.tensor(y_train_seq, dtype=torch.float32),
    )
    val_dataset = TensorDataset(
        torch.tensor(X_val_seq, dtype=torch.float32),
        torch.tensor(y_val_seq, dtype=torch.float32),
    )

    train_loader = DataLoader(
        train_dataset,
        batch_size=training_cfg.batch_size,
        shuffle=True,
        num_workers=training_cfg.num_workers,
        pin_memory=training_cfg.pin_memory,
    )
    val_loader = DataLoader(
        val_dataset,
        batch_size=training_cfg.batch_size,
        shuffle=False,
        num_workers=training_cfg.num_workers,
        pin_memory=training_cfg.pin_memory,
    )

    optimizer = torch.optim.Adam(
```

```
        model.parameters(),
        lr=training_cfg.learning_rate_lstm,
        weight_decay=training_cfg.weight_decay,
    )

    history: Dict[str, List[float]] = {"train_losses": [], "val_losses": []}
    best_val_loss = float("inf")
    patience_counter = 0
    best_state_dict = None

    logger.info(
        "Training LSTMSequenceClassifier -- epochs: %d, batch_size: %d, device: %s",
        training_cfg.num_epochs_lstm,
        training_cfg.batch_size,
        device,
    )

    for epoch in range(training_cfg.num_epochs_lstm):
        model.train()
        train_loss_sum = 0.0
        train_batches = 0
        for batch_x, batch_y in train_loader:
            batch_x = batch_x.to(device)
            batch_y = batch_y.to(device)

            optimizer.zero_grad()
            logits = model(batch_x)
            loss = focal_loss(logits, batch_y)
            loss.backward()
            torch.nn.utils.clip_grad_norm_(
                model.parameters(), training_cfg.gradient_clip_max_norm
            )
            optimizer.step()

            train_loss_sum += loss.item()
            train_batches += 1

        avg_train_loss = train_loss_sum / max(train_batches, 1)

        model.eval()
        val_loss_sum = 0.0
        val_batches = 0
        with torch.no_grad():
            for batch_x, batch_y in val_loader:
                batch_x = batch_x.to(device)
                batch_y = batch_y.to(device)
                logits = model(batch_x)
                loss = focal_loss(logits, batch_y)
                val_loss_sum += loss.item()
                val_batches += 1

        avg_val_loss = val_loss_sum / max(val_batches, 1)

        history["train_losses"].append(avg_train_loss)
        history["val_losses"].append(avg_val_loss)

        logger.info(
            "Epoch %d/%d -- train_loss: %.6f, val_loss: %.6f",
            epoch + 1,
            training_cfg.num_epochs_lstm,
            avg_train_loss,
            avg_val_loss,
        )

        if avg_val_loss < best_val_loss:
            best_val_loss = avg_val_loss
            patience_counter = 0
            best_state_dict = {k: v.cpu().clone() for k, v in model.state_dict().items()}
        else:
            patience_counter += 1
            if patience_counter >= training_cfg.early_stopping_patience:
                logger.info(
                    "Early stopping at epoch %d (patience=%d)",
                    epoch + 1,
                    training_cfg.early_stopping_patience,
                )
                break

    if best_state_dict is not None:
        model.load_state_dict(best_state_dict)
        model = model.to(device)
```

```
        logger.info("Restored best model weights (val_loss=%.6f)", best_val_loss)

    return history

def train_attack_classifier(
    model: AttackTypeClassifier,
    X_train_hidden: np.ndarray,
    y_train_attack: np.ndarray,
    X_val_hidden: np.ndarray,
    y_val_attack: np.ndarray,
    focal_loss: MultiClassFocalLoss,
    training_cfg: TrainingConfig,
) -> Dict[str, List[float]]:
    device = _get_device(training_cfg.device)
    model = model.to(device)
    focal_loss = focal_loss.to(device)

    train_dataset = TensorDataset(
        torch.tensor(X_train_hidden, dtype=torch.float32),
        torch.tensor(y_train_attack, dtype=torch.long),
    )
    val_dataset = TensorDataset(
        torch.tensor(X_val_hidden, dtype=torch.float32),
        torch.tensor(y_val_attack, dtype=torch.long),
    )

    train_loader = DataLoader(
        train_dataset,
        batch_size=training_cfg.batch_size,
        shuffle=True,
    )
    val_loader = DataLoader(
        val_dataset,
        batch_size=training_cfg.batch_size,
        shuffle=False,
    )

    optimizer = torch.optim.Adam(
        model.parameters(),
        lr=model.cfg.learning_rate,
        weight_decay=training_cfg.weight_decay,
    )

    history: Dict[str, List[float]] = {"train_losses": [], "val_losses": []}
    best_val_loss = float("inf")
    patience_counter = 0
    best_state_dict = None

    logger.info("Training AttackTypeClassifier -- epochs: %d, device: %s", model.cfg.num_epochs, device)

    for epoch in range(model.cfg.num_epochs):
        model.train()
        train_loss_sum = 0.0
        train_batches = 0
        for batch_x, batch_y in train_loader:
            batch_x, batch_y = batch_x.to(device), batch_y.to(device)
            optimizer.zero_grad()
            logits = model(batch_x)
            loss = focal_loss(logits, batch_y)
            loss.backward()
            optimizer.step()
            train_loss_sum += loss.item()
            train_batches += 1

        avg_train_loss = train_loss_sum / max(train_batches, 1)

        model.eval()
        val_loss_sum = 0.0
        val_batches = 0
        with torch.no_grad():
            for batch_x, batch_y in val_loader:
                batch_x, batch_y = batch_x.to(device), batch_y.to(device)
                logits = model(batch_x)
                loss = focal_loss(logits, batch_y)
                val_loss_sum += loss.item()
                val_batches += 1

        avg_val_loss = val_loss_sum / max(val_batches, 1)
        history["train_losses"].append(avg_train_loss)
        history["val_losses"].append(avg_val_loss)
```

```
        if avg_val_loss < best_val_loss:
            best_val_loss = avg_val_loss
            patience_counter = 0
            best_state_dict = {k: v.cpu().clone() for k, v in model.state_dict().items()}
        else:
            patience_counter += 1
            if patience_counter >= training_cfg.early_stopping_patience:
                break

    if best_state_dict is not None:
        model.load_state_dict(best_state_dict)
        model = model.to(device)

    return history

def infer_autoencoder(
    model: SharedAutoencoder,
    X: np.ndarray,
    device: torch.device,
) -> np.ndarray:
    model.eval()
    model = model.to(device)
    X_tensor = torch.tensor(X, dtype=torch.float32, device=device)

    errors: List[np.ndarray] = []
    batch_size = 1024
    with torch.no_grad():
        for i in range(0, len(X_tensor), batch_size):
            batch = X_tensor[i : i + batch_size]
            batch_errors = model.compute_reconstruction_error(batch)
            errors.append(batch_errors.cpu().numpy())

    return np.concatenate(errors, axis=0)

def infer_lstm(
    model: LSTMSequenceClassifier,
    X_seq: np.ndarray,
    device: torch.device,
) -> np.ndarray:
    model.eval()
    model = model.to(device)
    X_tensor = torch.tensor(X_seq, dtype=torch.float32, device=device)

    probs: List[np.ndarray] = []
    batch_size = 1024
    with torch.no_grad():
        for i in range(0, len(X_tensor), batch_size):
            batch = X_tensor[i : i + batch_size]
            batch_probs = model.predict_proba(batch)
            probs.append(batch_probs.cpu().numpy())

    return np.concatenate(probs, axis=0)

def explain_autoencoder(
    model: SharedAutoencoder,
    X: np.ndarray,
    feature_names: List[str],
    device: torch.device,
    n_steps: int = 50,
) -> Dict[str, np.ndarray]:
    model.eval()
    model = model.to(device)

    def ae_forward_for_ig(x: torch.Tensor) -> torch.Tensor:
        reconstructed = model(x)
        error = ((x - reconstructed) ** 2).mean(dim=1)
        return error

    X_tensor = torch.tensor(X, dtype=torch.float32, device=device)
    X_tensor.requires_grad_(True)

    baseline = torch.zeros_like(X_tensor, device=device)

    ig = IntegratedGradients(ae_forward_for_ig)

    batch_size = 256
    all_attributions: List[np.ndarray] = []

    for i in range(0, len(X_tensor), batch_size):
        batch_input = X_tensor[i : i + batch_size]
        batch_baseline = baseline[i : i + batch_size]
```

```
        attrs = ig.attribute(
            batch_input,
            baselines=batch_baseline,
            n_steps=n_steps,
            return_convergence_delta=False,
        )
        all_attributions.append(attrs.detach().cpu().numpy())

    attributions = np.concatenate(all_attributions, axis=0)
    mean_abs_attr = np.mean(np.abs(attributions), axis=0)

    sorted_indices = np.argsort(mean_abs_attr)[::-1]
    logger.info("Top 5 features by mean |IG attribution|:")
    for rank, idx in enumerate(sorted_indices[:5]):
        logger.info(
            "  %d. %s: %.6f", rank + 1, feature_names[idx], mean_abs_attr[idx]
        )

    return {
        "attributions": attributions,
        "feature_names": feature_names,
        "mean_attributions": mean_abs_attr,
    }

def save_model(
    model: nn.Module,
    path: Path,
    metadata: Optional[Dict[str, Any]] = None,
) -> None:
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)

    save_dict: Dict[str, Any] = {"state_dict": model.state_dict()}
    if metadata is not None:
        save_dict["metadata"] = metadata

    torch.save(save_dict, path)
    logger.info("Saved model to %s", path)

def load_model(
    model: nn.Module,
    path: Path,
    device: Optional[torch.device] = None,
) -> Dict[str, Any]:
    path = Path(path)
    map_location = device if device is not None else torch.device("cpu")
    checkpoint = torch.load(path, map_location=map_location, weights_only=False)

    model.load_state_dict(checkpoint["state_dict"])
    if device is not None:
        model.to(device)

    logger.info("Loaded model from %s", path)
    return checkpoint.get("metadata", {})

def save_thresholds(thresholds: Dict[str, float], path: Path) -> None:
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    with open(path, "w", encoding="utf-8") as f:
        json.dump(thresholds, f, indent=2)
    logger.info("Saved thresholds to %s", path)

def load_thresholds(path: Path) -> Dict[str, float]:
    with open(path, "r", encoding="utf-8") as f:
        thresholds = json.load(f)
    logger.info("Loaded thresholds from %s", path)
    return thresholds

def run_hybrid_inference(
    autoencoder: SharedAutoencoder,
    lstm_classifier: LSTMSequenceClassifier,
    policy_engine: PolicyEngine,
    risk_scorer: RiskScorer,
    X: np.ndarray,
    feature_names: List[str],
    sequence_length: int,
    device: torch.device,
) -> Dict[str, np.ndarray]:
    ae_errors = infer_autoencoder(autoencoder, X, device)
```

```
    dummy_y = np.zeros(len(X), dtype=np.float64)
    X_seq, _ = create_sequences(X, dummy_y, sequence_length)
    lstm_probs = infer_lstm(lstm_classifier, X_seq, device)

    offset = sequence_length - 1
    aligned_X = X[offset:]
    feature_dict: Dict[str, np.ndarray] = {}
    for i, name in enumerate(feature_names):
        feature_dict[name] = aligned_X[:, i]
    policy_scores = policy_engine.evaluate(feature_dict)

    ae_errors_aligned = ae_errors[offset:]

    min_len = min(len(ae_errors_aligned), len(lstm_probs), len(policy_scores))
    ae_errors_aligned = ae_errors_aligned[:min_len]
    lstm_probs = lstm_probs[:min_len]
    policy_scores = policy_scores[:min_len]

    risk_scores = risk_scorer.compute(ae_errors_aligned, lstm_probs, policy_scores)

    logger.info(
        "Hybrid inference complete -- %d samples scored. "
        "Risk score stats: mean=%.4f, std=%.4f, max=%.4f",
        min_len,
        risk_scores.mean(),
        risk_scores.std(),
        risk_scores.max(),
    )

    return {
        "ae_errors": ae_errors_aligned,
        "lstm_probs": lstm_probs,
        "policy_scores": policy_scores,
        "risk_scores": risk_scores,
    }

if __name__ == "__main__":
    logging.basicConfig(
        level=logging.INFO,
        format="%(asctime)s | %(name)s | %(levelname)s | %(message)s",
    )
    logger.info("models.py loaded. Use the public API to train and infer.")
```

File: signature_rules.py

Description: High-Precision Attack Signatures

```
"""High-precision, validation-tuned behavioral attack signatures.

Fast-path rules for impossible travel (speed > 600 km/h + external IP), device spoofing (OS/browser mismatch),
and credential stuffing, unioned with the ML detection threshold.
"""

from __future__ import annotations

import json
from dataclasses import dataclass
from pathlib import Path
from typing import Any, Callable, Dict, Tuple

import numpy as np
import pandas as pd

@dataclass(frozen=True)
class SignatureRule:
    name: str
    condition: Callable[[pd.DataFrame, Dict[str, float]], np.ndarray]
    mitre_technique_id: str
    confidence: float

def impossible_travel_mask(features: pd.DataFrame, thresholds: Dict[str, float]) -> np.ndarray:
    speed = features["travel_speed_kmh"].to_numpy() > thresholds["speed_threshold_kmh"]
    if "source_ip" in features:
        external_ip = features["source_ip"].astype(str).str.startswith("172.16.").to_numpy()
        return speed & external_ip
    return speed

def device_spoofing_mask(features: pd.DataFrame, thresholds: Dict[str, float]) -> np.ndarray:
    mismatch = (features["os_mismatch"].to_numpy() == 1) | (features["browser_mismatch"].to_numpy() == 1)
    return mismatch & (features["fingerprint_consistency"].to_numpy() < thresholds["fingerprint_consistency_threshold"])

SIGNATURE_RULES = (
    SignatureRule("impossible_travel_signature", impossible_travel_mask, "T1078", 0.99),
    SignatureRule("device_spoofing_signature", device_spoofing_mask, "T1036", 0.98),
)

def credential_stuffing_mask(features: pd.DataFrame, thresholds: Dict[str, float]) -> np.ndarray:
    required = {"timestamp", "entity_id", "source_ip", "session_duration"}
    threshold_keys = {"credential_window_seconds", "credential_entity_count", "credential_source_ip_count", "credential_failure_rate"}
    if not required.issubset(features.columns) or not threshold_keys.issubset(thresholds):
        return np.zeros(len(features), dtype=bool)
    ordered = features.copy()
    ordered["_original_index"] = np.arange(len(ordered))
    ordered["timestamp"] = pd.to_datetime(ordered["timestamp"], utc=True)
    ordered = ordered.sort_values("timestamp")
    times = ordered["timestamp"].astype("int64").to_numpy() // 1_000_000_000
    failed = ordered["session_duration"].to_numpy(dtype=float) < 0.5
    entity_values = ordered["entity_id"].astype(str).to_numpy()
    ip_values = ordered["source_ip"].astype(str).to_numpy()
    out = np.zeros(len(ordered), dtype=bool)
    start = 0
    window_seconds = int(thresholds["credential_window_seconds"])
    entity_counts: Dict[str, int] = {}
    ip_counts: Dict[str, int] = {}
    failure_count = 0
    for end in range(len(ordered)):
        entity_counts[entity_values[end]] = entity_counts.get(entity_values[end], 0) + 1
        ip_counts[ip_values[end]] = ip_counts.get(ip_values[end], 0) + 1
        failure_count += int(failed[end])
        while times[end] - times[start] > window_seconds:
            entity_counts[entity_values[start]] -= 1
            if entity_counts[entity_values[start]] == 0:
                del entity_counts[entity_values[start]]
            ip_counts[ip_values[start]] -= 1
            if ip_counts[ip_values[start]] == 0:
                del ip_counts[ip_values[start]]
            failure_count -= int(failed[start])
            start += 1
        failure_rate = failure_count / (end - start + 1)
        out[end] = (
            len(entity_counts) >= int(thresholds["credential_entity_count"])
            and len(ip_counts) <= int(thresholds["credential_source_ip_count"])
            and failure_rate >= float(thresholds["credential_failure_rate"])
```

```
        )
    result = np.zeros(len(features), dtype=bool)
    result[ordered["_original_index"].to_numpy()] = out
    return result

SIGNATURE_RULES = SIGNATURE_RULES + (
    SignatureRule("credential_stuffing_signature", credential_stuffing_mask, "T1110.004", 0.97),
)

def tune_signatures(features: pd.DataFrame, attack_types: np.ndarray) -> Tuple[Dict[str, float], Dict[str, Any]]:
    benign = attack_types == "none"
    impossible = attack_types == "impossible_travel"
    device = attack_types == "device_spoofing"
    evidence: Dict[str, float] = {}

    for speed in range(600, 1501, 50):
        fires = impossible_travel_mask(features, {"speed_threshold_kmh": float(speed)})
        fp = int((fires & benign).sum())
        if fp <= 3:
            speed_threshold = float(speed)
            break
    else:
        speed_threshold = 1500.0
    speed_fires = impossible_travel_mask(features, {"speed_threshold_kmh": speed_threshold})
    evidence.update({
        "impossible_travel_validation_fp": int((speed_fires & benign).sum()),
        "impossible_travel_validation_recall": float(speed_fires[impossible].mean()) if impossible.any() else 0.0,
    })
    credential_candidates = []
    for window in (300, 600, 900):
        for entity_count in (3, 5):
            for source_count in (1, 3):
                for failure_rate in (0.5, 0.9):
                    candidate = {
                        "credential_window_seconds": window,
                        "credential_entity_count": entity_count,
                        "credential_source_ip_count": source_count,
                        "credential_failure_rate": failure_rate,
                    }
                    fires = credential_stuffing_mask(features, candidate)
                    credential = attack_types == "credential_stuffing"
                          credential_candidates.append({**candidate, "validation_fp": int((fires & benign).sum()), "validation_recall":
float(fires[credential].mean()) if credential.any() else 0.0})
    feasible = [x for x in credential_candidates if x["validation_fp"] <= 9 and x["validation_recall"] > 0]
      selected = max(feasible, key=lambda x: (x["validation_recall"], x["credential_entity_count"], -x["credential_source_ip_count"],
x["credential_failure_rate"])) if feasible else None
    if selected:
        evidence["credential_stuffing_validation"] = {"selected": selected, "candidates": credential_candidates}
    else:
        evidence["credential_stuffing_validation"] = {"selected": None, "candidates": credential_candidates}

    candidates = []
    for threshold in np.arange(0.05, 1.0, 0.05):
        fires = device_spoofing_mask(features, {"fingerprint_consistency_threshold": float(threshold)})
        recall = float(fires[device].mean()) if device.any() else 0.0
        if recall >= 0.60:
            candidates.append((int((fires & benign).sum()), -float(threshold), float(threshold), recall))
    _, _, fingerprint_threshold, device_recall = min(candidates) if candidates else (0, 0, 0.5, 0.0)
    device_fires = device_spoofing_mask(features, {"fingerprint_consistency_threshold": fingerprint_threshold})
    evidence.update({
        "device_spoofing_validation_fp": int((device_fires & benign).sum()),
        "device_spoofing_validation_recall": device_recall,
    })
    thresholds = {
        "speed_threshold_kmh": speed_threshold,
        "fingerprint_consistency_threshold": fingerprint_threshold,
    }
    if selected:
                      thresholds.update({key: selected[key] for key in ("credential_window_seconds", "credential_entity_count",
"credential_source_ip_count", "credential_failure_rate")})
    else:
              thresholds.update({"credential_window_seconds": 300, "credential_entity_count": 999999, "credential_source_ip_count": 0,
"credential_failure_rate": 1.0})
    return thresholds, evidence

def evaluate_signatures(features: pd.DataFrame, thresholds: Dict[str, float]) -> Dict[str, np.ndarray]:
    return {rule.name: rule.condition(features, thresholds) for rule in SIGNATURE_RULES}
```

File: alert_engine.py

Description: Alert Generation & Attack Chain Reconstruction

```
"""Alert generation engine and temporal attack chain reconstruction.

Translates hybrid risk scores and signature matches into structured SOC alerts with MITRE ATT&CK mapping,
and reconstructs multi-stage attack chains by correlating alerts per entity within a 1-hour correlation window.
"""

from __future__ import annotations

import logging
from dataclasses import dataclass, field
from datetime import datetime, timedelta
from typing import Any, Dict, List, Optional, Tuple

import numpy as np
import pandas as pd

from config import ATTACK_TYPES, ENTITY_TYPES, ProjectConfig, get_project_config

logger = logging.getLogger(__name__)

MITRE_ATTACK_MAPPING: Dict[str, Dict[str, Any]] = {
    "brute_force": {
        "tactic": "Credential Access",
        "technique_id": "T1110",
        "technique_name": "Brute Force",
        "sub_techniques": ["T1110.001", "T1110.003"],
        "severity_base": 0.7,
    },
    "credential_stuffing": {
        "tactic": "Credential Access",
        "technique_id": "T1110.004",
        "technique_name": "Credential Stuffing",
        "sub_techniques": [],
        "severity_base": 0.75,
    },
    "impossible_travel": {
        "tactic": "Initial Access",
        "technique_id": "T1078",
        "technique_name": "Valid Accounts (Impossible Travel)",
        "sub_techniques": ["T1078.004"],
        "severity_base": 0.8,
    },
    "lateral_movement": {
        "tactic": "Lateral Movement",
        "technique_id": "T1021",
        "technique_name": "Remote Services",
        "sub_techniques": ["T1021.002", "T1021.006"],
        "severity_base": 0.85,
    },
    "device_spoofing": {
        "tactic": "Defense Evasion",
        "technique_id": "T1036",
        "technique_name": "Masquerading",
        "sub_techniques": ["T1036.005"],
        "severity_base": 0.65,
    },
    "low_and_slow_exfiltration": {
        "tactic": "Exfiltration",
        "technique_id": "T1048",
        "technique_name": "Exfiltration Over Alternative Protocol",
        "sub_techniques": ["T1048.002"],
        "severity_base": 0.9,
    },
    "insider_drift": {
        "tactic": "Collection",
        "technique_id": "T1074",
        "technique_name": "Data Staged (Insider Threat)",
        "sub_techniques": ["T1074.001"],
        "severity_base": 0.6,
    },
    "unknown": {
        "tactic": "Unknown",
        "technique_id": "N/A",
        "technique_name": "Anomalous Behaviour",
        "sub_techniques": [],
        "severity_base": 0.5,
```

```
    },
}

SEVERITY_LEVELS: Dict[str, Tuple[float, float]] = {
    "critical": (0.85, 1.0),
    "high": (0.65, 0.85),
    "medium": (0.45, 0.65),
    "low": (0.25, 0.45),
    "info": (0.0, 0.25),
}

def _classify_severity(risk_score: float) -> str:
    for level, (lo, hi) in SEVERITY_LEVELS.items():
        if lo <= risk_score < hi:
            return level
    return "critical" if risk_score >= 0.85 else "info"

@dataclass
class Alert:

    alert_id: str
    entity_id: str
    entity_type: str
    timestamp: str
    risk_score: float
    severity: str
    ae_error: float
    lstm_prob: float
    policy_score: float
    predicted_attack_type: str
    mitre_tactic: str
    mitre_technique_id: str
    mitre_technique_name: str
    mitre_sub_techniques: List[str]
    source_ip: str = ""
    geo_location: str = ""
    resource_accessed: str = ""
    attack_chain_id: Optional[str] = None
    cold_start: bool = False
    detection_source: str = "ml_threshold"
    classification_confidence: Optional[float] = None
    contributing_factors: List[str] = field(default_factory=list)

    def to_dict(self) -> Dict[str, Any]:
        return {
            "alert_id": self.alert_id,
            "entity_id": self.entity_id,
            "entity_type": self.entity_type,
            "timestamp": self.timestamp,
            "risk_score": round(self.risk_score, 6),
            "severity": self.severity,
            "ae_error": round(self.ae_error, 6),
            "lstm_prob": round(self.lstm_prob, 6),
            "policy_score": round(self.policy_score, 6),
            "predicted_attack_type": self.predicted_attack_type,
            "mitre_tactic": self.mitre_tactic,
            "mitre_technique_id": self.mitre_technique_id,
            "mitre_technique_name": self.mitre_technique_name,
            "mitre_sub_techniques": ";".join(self.mitre_sub_techniques),
            "source_ip": self.source_ip,
            "geo_location": self.geo_location,
            "resource_accessed": self.resource_accessed,
            "attack_chain_id": self.attack_chain_id or "",
            "cold_start": self.cold_start,
            "detection_source": self.detection_source,
            "classification_confidence": (
                round(float(self.classification_confidence), 6)
                if self.classification_confidence is not None else None
            ),
            "contributing_factors": ";".join(self.contributing_factors) if self.contributing_factors else "",
        }

def _predict_attack_type(
    ae_error: float,
    lstm_prob: float,
    policy_score: float,
    features: Optional[Dict[str, float]] = None,
) -> str:
    if features is None:
        features = {}
```

```
    travel_speed = features.get("travel_speed_kmh", 0.0)
    geo_dist = features.get("geo_distance_km", 0.0)
    failed_logins = features.get("failed_logins_last_10min", 0.0)
    fp_consistency = features.get("fingerprint_consistency", 1.0)
    os_mismatch = features.get("os_mismatch", 0.0)
    browser_mismatch = features.get("browser_mismatch", 0.0)
    seq_entropy = features.get("sequence_entropy", 0.0)
    priv_esc = features.get("privilege_escalation_count", 0.0)
    resource_rarity = features.get("resource_rarity", 0.0)
    session_dur = features.get("session_duration", 0.0)
    login_hour = features.get("login_hour", 12.0)

    if failed_logins > 5:
        if geo_dist > 1000:
            return "credential_stuffing"
        return "brute_force"

    if travel_speed > 900.0:
        return "impossible_travel"

    if fp_consistency < 0.5 or os_mismatch == 1.0 or browser_mismatch == 1.0:
        return "device_spoofing"

    if priv_esc > 2 and seq_entropy > 2.0:
        return "lateral_movement"

    if session_dur > 300 and resource_rarity > 0.8:
        return "low_and_slow_exfiltration"

    if (login_hour >= 22 or login_hour <= 5) and lstm_prob > 0.3:
        return "insider_drift"

    if lstm_prob > 0.5:
        return "brute_force"

    return "unknown"

class AlertEngine:

    def __init__(
        self,
        risk_threshold: float = 0.5,
        chain_window_seconds: int = 3600,
        chain_min_alerts: int = 3,
    ) -> None:
        self.risk_threshold: float = risk_threshold
        self.chain_window_seconds: int = chain_window_seconds
        self.chain_min_alerts: int = chain_min_alerts
        logger.info(
            "AlertEngine initialised -- threshold=%.2f, chain_window=%ds, chain_min=%d",
            risk_threshold, chain_window_seconds, chain_min_alerts,
        )

    def generate_alerts(
        self,
        risk_scores: np.ndarray,
        ae_errors: np.ndarray,
        lstm_probs: np.ndarray,
        policy_scores: np.ndarray,
        entity_ids: np.ndarray,
        entity_types: np.ndarray,
        timestamps: np.ndarray,
        source_ips: Optional[np.ndarray] = None,
        geo_locations: Optional[np.ndarray] = None,
        resources: Optional[np.ndarray] = None,
        feature_matrix: Optional[np.ndarray] = None,
        feature_names: Optional[List[str]] = None,
        risk_thresholds: Optional[Dict[str, float] | float] = None,
        cold_starts: Optional[np.ndarray] = None,
        contributing_factors: Optional[List[List[str]]] = None,
        predicted_attack_types: Optional[np.ndarray] = None,
        attack_probabilities: Optional[np.ndarray] = None,
        attack_type_to_index: Optional[Dict[str, int]] = None,
        signature_matches: Optional[Dict[str, np.ndarray]] = None,
        signature_factors: Optional[List[List[str]]] = None,
    ) -> pd.DataFrame:
        alerts: List[Dict[str, Any]] = []
        alert_counter = 0

        for i in range(len(risk_scores)):
            etype = str(entity_types[i])
```

```
            if isinstance(risk_thresholds, dict):
                t = risk_thresholds.get(etype, risk_thresholds.get("global", self.risk_threshold))
            elif isinstance(risk_thresholds, (float, int)):
                t = float(risk_thresholds)
            else:
                t = self.risk_threshold

            ml_triggered = risk_scores[i] >= t
            matching_signatures = [name for name, mask in (signature_matches or {}).items() if bool(mask[i])]
            signature_triggered = bool(matching_signatures)
            if not ml_triggered and not signature_triggered:
                continue

            alert_counter += 1
            alert_id = f"ALR-{alert_counter:06d}"

            feat_dict: Optional[Dict[str, float]] = None
            if feature_matrix is not None and feature_names is not None:
                feat_dict = {
                    name: float(feature_matrix[i, j])
                    for j, name in enumerate(feature_names)
                }

            prediction_from_signature = bool(matching_signatures)
            prediction_from_classifier = False
            if "impossible_travel_signature" in matching_signatures:
                predicted_type = "impossible_travel"
            elif "device_spoofing_signature" in matching_signatures:
                predicted_type = "device_spoofing"
            elif "credential_stuffing_signature" in matching_signatures:
                predicted_type = "credential_stuffing"
             elif predicted_attack_types is not None and predicted_attack_types[i] is not None and str(predicted_attack_types[i]) not in
("unknown", "benign", "None"):
                predicted_type = str(predicted_attack_types[i])
                prediction_from_classifier = True
            else:
                predicted_type = _predict_attack_type(
                    ae_errors[i], lstm_probs[i], policy_scores[i], feat_dict
                )

            mitre = MITRE_ATTACK_MAPPING.get(
                predicted_type, MITRE_ATTACK_MAPPING["unknown"]
            )
            confidence: Optional[float] = None
            if (
                prediction_from_classifier
                and not prediction_from_signature
                and attack_probabilities is not None
                and attack_type_to_index is not None
            ):
                class_index = attack_type_to_index.get(predicted_type)
                if class_index is not None and i < len(attack_probabilities):
                    confidence = float(attack_probabilities[i, class_index])

            severity = _classify_severity(risk_scores[i])

            alert = Alert(
                alert_id=alert_id,
                entity_id=str(entity_ids[i]),
                entity_type=str(entity_types[i]),
                timestamp=str(timestamps[i]),
                risk_score=float(risk_scores[i]),
                severity=severity,
                ae_error=float(ae_errors[i]),
                lstm_prob=float(lstm_probs[i]),
                policy_score=float(policy_scores[i]),
                predicted_attack_type=predicted_type,
                mitre_tactic=mitre["tactic"],
                mitre_technique_id=mitre["technique_id"],
                mitre_technique_name=mitre["technique_name"],
                mitre_sub_techniques=mitre["sub_techniques"],
                source_ip=str(source_ips[i]) if source_ips is not None else "",
                geo_location=str(geo_locations[i]) if geo_locations is not None else "",
                resource_accessed=str(resources[i]) if resources is not None else "",
                cold_start=bool(cold_starts[i]) if cold_starts is not None else False,
                    detection_source="both" if ml_triggered and signature_triggered else ("signature_rule" if signature_triggered else
"ml_threshold"),
                classification_confidence=confidence,
                            contributing_factors=(signature_factors[i] if signature_triggered and signature_factors is not None else
contributing_factors[i]) if contributing_factors is not None and len(contributing_factors) > i else [],
            )
```

```
            alerts.append(alert.to_dict())

        if not alerts:
            logger.info("No alerts generated (all risk scores below threshold).")
            return pd.DataFrame(columns=[
                "alert_id", "entity_id", "entity_type", "timestamp",
                "risk_score", "severity", "ae_error", "lstm_prob",
                "policy_score", "predicted_attack_type", "mitre_tactic",
                "mitre_technique_id", "mitre_technique_name",
                "mitre_sub_techniques", "source_ip", "geo_location",
                "resource_accessed", "attack_chain_id", "cold_start", "detection_source",
                "classification_confidence", "contributing_factors",
            ])

        alerts_df = pd.DataFrame(alerts)
        alerts_df = alerts_df.sort_values("risk_score", ascending=False).reset_index(drop=True)

        alerts_df = self._reconstruct_chains(alerts_df)

        logger.info(
            "Generated %d alerts (%d critical, %d high, %d medium, %d low, %d info)",
            len(alerts_df),
            (alerts_df["severity"] == "critical").sum(),
            (alerts_df["severity"] == "high").sum(),
            (alerts_df["severity"] == "medium").sum(),
            (alerts_df["severity"] == "low").sum(),
            (alerts_df["severity"] == "info").sum(),
        )

        return alerts_df

    def _reconstruct_chains(self, alerts_df: pd.DataFrame) -> pd.DataFrame:
        if alerts_df.empty:
            return alerts_df

        alerts_df = alerts_df.copy()
        alerts_df["_ts_parsed"] = pd.to_datetime(alerts_df["timestamp"], utc=True)
        alerts_df = alerts_df.sort_values(["entity_id", "_ts_parsed"])

        chain_counter = 0
        chain_ids: List[Optional[str]] = [None] * len(alerts_df)
        window = timedelta(seconds=self.chain_window_seconds)

        for entity_id, group in alerts_df.groupby("entity_id"):
            if len(group) < self.chain_min_alerts:
                continue

            group_sorted = group.sort_values("_ts_parsed")
            chain_start_idx = 0
            current_chain: List[int] = [group_sorted.index[0]]

            for j in range(1, len(group_sorted)):
                curr_ts = group_sorted.iloc[j]["_ts_parsed"]
                prev_ts = group_sorted.iloc[j - 1]["_ts_parsed"]

                if (curr_ts - prev_ts) <= window:
                    current_chain.append(group_sorted.index[j])
                else:
                    if len(current_chain) >= self.chain_min_alerts:
                        chain_counter += 1
                        chain_id = f"CHAIN-{chain_counter:04d}"
                        for idx in current_chain:
                            chain_ids[alerts_df.index.get_loc(idx)] = chain_id
                    current_chain = [group_sorted.index[j]]

            if len(current_chain) >= self.chain_min_alerts:
                chain_counter += 1
                chain_id = f"CHAIN-{chain_counter:04d}"
                for idx in current_chain:
                    chain_ids[alerts_df.index.get_loc(idx)] = chain_id

        alerts_df["attack_chain_id"] = chain_ids
        alerts_df = alerts_df.drop(columns=["_ts_parsed"])

        logger.info(
            "Reconstructed %d attack chains from %d alerts.",
            chain_counter, len(alerts_df),
        )
        return alerts_df

    @classmethod
```

```
    def from_config(
        cls,
        config: Optional[ProjectConfig] = None,
        risk_threshold: float = 0.5,
    ) -> "AlertEngine":
        return cls(risk_threshold=risk_threshold)

def get_mitre_mapping() -> Dict[str, Dict[str, Any]]:
    return MITRE_ATTACK_MAPPING.copy()

def get_severity_distribution(alerts_df: pd.DataFrame) -> Dict[str, int]:
    if alerts_df.empty:
        return {level: 0 for level in SEVERITY_LEVELS}
    return alerts_df["severity"].value_counts().to_dict()
```

File: drift_monitor.py

Description: Concept Drift Monitoring (PSI)

```
"""Concept drift monitoring using Population Stability Index (PSI).

Monitors distribution shift in Autoencoder reconstruction errors over time per persona against the baseline training set.
"""

from __future__ import annotations

import logging
from dataclasses import dataclass
from typing import Dict, List, Tuple

import numpy as np

from config import ENTITY_TYPES, ProjectConfig

logger = logging.getLogger(__name__)

@dataclass
class DriftResult:
    persona: str
    psi: float
    level: str

class DriftMonitor:

    def __init__(self, config: ProjectConfig) -> None:
        self.config = config
        self.psi_warning = config.training.drift_psi_warning
        self.psi_alert = config.training.drift_psi_alert
        self.n_bins = config.training.drift_n_bins

    def compute_psi(
        self,
        reference: np.ndarray,
        current: np.ndarray,
    ) -> float:
        if len(reference) == 0 or len(current) == 0:
            return 0.0

        clip_val = max(
            float(np.percentile(reference, 99.9)),
            float(np.percentile(current, 99.9)),
        )

        reference = np.clip(reference, a_min=None, a_max=clip_val)
        current = np.clip(current, a_min=None, a_max=clip_val)

        quantiles = np.linspace(0, 100, self.n_bins + 1)
        bins = np.percentile(reference, quantiles)

        bins = np.unique(bins)

        if len(bins) < 2:
            bins = np.linspace(reference.min(), reference.max(), self.n_bins + 1)

        bins[0] -= 1e-5
        bins[-1] += 1e-5

        expected_counts, _ = np.histogram(reference, bins=bins)
        actual_counts, _ = np.histogram(current, bins=bins)

        expected_pct = expected_counts / len(reference)
        actual_pct = actual_counts / len(current)

        eps = 1e-6
        expected_pct = np.clip(expected_pct, eps, None)
        actual_pct = np.clip(actual_pct, eps, None)

        psi_values = (actual_pct - expected_pct) * np.log(actual_pct / expected_pct)
        psi = np.sum(psi_values)

        return float(psi)

    def check_drift(
        self,
        current_errors: np.ndarray,
```

```
        current_entity_types: np.ndarray,
        reference_errors: np.ndarray,
        reference_entity_types: np.ndarray,
    ) -> Dict[str, DriftResult]:
        results: Dict[str, DriftResult] = {}

        for etype in ENTITY_TYPES:
            curr_mask = current_entity_types == etype
            ref_mask = reference_entity_types == etype

            curr_vals = current_errors[curr_mask]
            ref_vals = reference_errors[ref_mask]

            if len(curr_vals) == 0 or len(ref_vals) == 0:
                results[etype] = DriftResult(etype, 0.0, "normal")
                continue

            psi = self.compute_psi(ref_vals, curr_vals)

            level = "normal"
            if psi >= self.psi_alert:
                level = "alert"
                logger.error("ALERT: Significant concept drift detected for %s (PSI=%.4f >= %.4f)", etype, psi, self.psi_alert)
            elif psi >= self.psi_warning:
                level = "warning"
                logger.warning("WARNING: Moderate concept drift detected for %s (PSI=%.4f >= %.4f)", etype, psi, self.psi_warning)

            results[etype] = DriftResult(etype, psi, level)

        return results
```

File: inference_pipeline.py

Description: End-to-End Inference & Captum IG Attribution

```
"""End-to-end inference orchestrator and Captum Integrated Gradients explainability pipeline.

Loads trained models, executes AE/LSTM/Policy inference, evaluates signature rules, generates alerts,
computes feature attribution via Captum IG, and enforces alert count consistency assertions.
"""

from __future__ import annotations

import json
import logging
import pickle
import time
from pathlib import Path
from typing import Any, Dict, List, Optional, Tuple

import numpy as np
import pandas as pd
import torch

from config import (
    ENTITY_TYPES,
    NUM_ENTITY_TYPES,
    ATTACK_TYPE_LABELS,
    ProjectConfig,
    get_project_config,
)
from models import (
    FocalLoss,
    LSTMSequenceClassifier,
    PolicyEngine,
    RiskScorer,
    SharedAutoencoder,
    AttackTypeClassifier,
    create_sequences,
    infer_autoencoder,
    infer_lstm,
    load_model,
    load_thresholds,
    set_torch_seed,
    explain_autoencoder,
)
from models import _get_device
from alert_engine import AlertEngine
from evaluation import (
    EvaluationResult,
    LatencyTracker,
    evaluate_predictions,
    find_optimal_threshold,
    generate_report,
)
from drift_monitor import DriftMonitor
from signature_rules import evaluate_signatures

logger = logging.getLogger(__name__)

class InferencePipeline:

    def __init__(
        self,
        autoencoder: SharedAutoencoder,
        lstm_classifier: LSTMSequenceClassifier,
        policy_engine: PolicyEngine,
        risk_scorer: RiskScorer,
        alert_engine: AlertEngine,
        thresholds: Dict[str, float],
        scaler: Any,
        encoder: Any,
        feature_names: List[str],
        device: torch.device,
        sequence_length: int,
        config: ProjectConfig,
        risk_thresholds: Optional[Dict[str, float]] = None,
        platt_scaler: Optional[Any] = None,
        ae_min: float = 0.0,
        ae_max: float = 1.0,
    ) -> None:
```

```
        self.autoencoder = autoencoder
        self.lstm_classifier = lstm_classifier
        self.policy_engine = policy_engine
        self.risk_scorer = risk_scorer
        self.alert_engine = alert_engine
        self.thresholds = thresholds
        self.scaler = scaler
        self.encoder = encoder
        self.feature_names = feature_names
        self.device = device
        self.sequence_length = sequence_length
        self.config = config
        self.risk_thresholds = risk_thresholds
        self.platt_scaler = platt_scaler
        self.ae_min = ae_min
        self.ae_max = ae_max
        self.attack_classifier: Optional[AttackTypeClassifier] = None

    def run(
        self,
        X: Optional[np.ndarray] = None,
        y_true: Optional[np.ndarray] = None,
        entity_types: Optional[np.ndarray] = None,
        entity_ids: Optional[np.ndarray] = None,
        timestamps: Optional[np.ndarray] = None,
        source_ips: Optional[np.ndarray] = None,
        geo_locations: Optional[np.ndarray] = None,
        resources: Optional[np.ndarray] = None,
        attack_types: Optional[np.ndarray] = None,
        risk_threshold: float = 0.5,
        split: str = "test",
    ) -> Dict[str, Any]:
        paths = self.config.paths
        feature_cfg = self.config.features
        latency_tracker = LatencyTracker()

        if X is None:
            X, y_true, entity_types, entity_ids, timestamps, source_ips, \
                geo_locations, resources, attack_types, cold_starts = self._load_split_data(
                    split, paths, feature_cfg
                )

        n_samples = len(X)
        logger.info("Running inference on %d samples (split=%s)...", n_samples, split)

        with latency_tracker:
            ae_errors = infer_autoencoder(self.autoencoder, X, self.device)
        logger.info("AE inference complete. Mean error: %.6f", ae_errors.mean())

        drift_results = None
        ref_path = paths.models_dir / "ae_reference_stats.npz"
        if ref_path.exists() and entity_types is not None:
            ref_data = np.load(ref_path, allow_pickle=True)
            ref_errors = ref_data["errors"]
            ref_entity_types = ref_data["entity_types"]

            monitor = DriftMonitor(self.config)
            drift_results = monitor.check_drift(ae_errors, entity_types, ref_errors, ref_entity_types)

            drift_dict = {k: v.__dict__ for k, v in drift_results.items()}
            with open(paths.outputs_dir / "drift_report.json", "w") as f:
                json.dump(drift_dict, f, indent=2)
            logger.info("Drift check complete. Results saved to drift_report.json")

        dummy_y = np.zeros(n_samples, dtype=np.float64)
        X_seq, y_seq = create_sequences(X, dummy_y, self.sequence_length)

        with latency_tracker:
            self.lstm_classifier.eval()
            X_seq_tensor = torch.tensor(X_seq, dtype=torch.float32, device=self.device)
            with torch.no_grad():
                logits = self.lstm_classifier(X_seq_tensor).cpu().numpy().reshape(-1)

            if hasattr(self, "platt_scaler") and self.platt_scaler is not None:
                lstm_probs = self.platt_scaler.predict_proba(logits.reshape(-1, 1))[:, 1]
            else:
                lstm_probs = 1.0 / (1.0 + np.exp(-logits))

        logger.info("LSTM inference complete. Mean prob: %.6f", lstm_probs.mean())

        predicted_attack_types = None
```

```
        attack_probs: Optional[np.ndarray] = None
        attack_type_to_index: Optional[Dict[str, int]] = None
        if getattr(self, "attack_classifier", None) is not None:
            with latency_tracker:
                self.attack_classifier.eval()
                self.lstm_classifier.eval()
                with torch.no_grad():
                    hidden_states = self.lstm_classifier.extract_hidden(X_seq_tensor)
                    attack_logits = self.attack_classifier(hidden_states)
                    attack_probs = torch.softmax(attack_logits, dim=1).cpu().numpy()

                idx_to_attack = {v: k for k, v in ATTACK_TYPE_LABELS.items()}
                attack_type_to_index = {attack: index for index, attack in idx_to_attack.items()}
                hard_classes = {"impossible_travel", "device_spoofing", "insider_drift", "low_and_slow_exfiltration"}

                predicted_attack_types = []
                for i in range(len(attack_probs)):
                    pred_idx = int(np.argmax(attack_probs[i, 1:])) + 1
                    prob = float(attack_probs[i, pred_idx])
                    atype = idx_to_attack.get(pred_idx, "unknown")
                    threshold = 0.20 if atype in hard_classes else 0.50

                    if prob < threshold:
                        predicted_attack_types.append(None)
                    else:
                        predicted_attack_types.append(atype)

                predicted_attack_types = np.array(predicted_attack_types)
            logger.info("AttackTypeClassifier inference complete.")

        offset = self.sequence_length - 1
        aligned_X = X[offset:]

        unscaled_X_all = self.scaler.inverse_transform(aligned_X)

        with latency_tracker:
            feature_dict: Dict[str, np.ndarray] = {}
            for j, name in enumerate(self.feature_names):
                feature_dict[name] = unscaled_X_all[:, j]
            policy_scores = self.policy_engine.evaluate(feature_dict)
        logger.info("Policy engine complete. Mean score: %.6f", policy_scores.mean())

        ae_errors_aligned = ae_errors[offset:]
        min_len = min(len(ae_errors_aligned), len(lstm_probs), len(policy_scores))
        ae_errors_aligned = ae_errors_aligned[:min_len]
        lstm_probs = lstm_probs[:min_len]
        policy_scores = policy_scores[:min_len]
        unscaled_X_aligned = unscaled_X_all[:min_len]
        aligned_cold_starts = cold_starts[offset:offset + min_len] if cold_starts is not None else np.zeros(min_len, dtype=bool)

        with latency_tracker:
            risk_scores = self.risk_scorer.compute(
                ae_errors_aligned, lstm_probs, policy_scores,
                ae_min=getattr(self, "ae_min", 0.0),
                ae_max=getattr(self, "ae_max", 1.0)
            )

            if aligned_cold_starts.any():
                cold_idx = np.where(aligned_cold_starts)[0]
                t_cfg = self.config.training
                orig_weights = (self.risk_scorer.w_ae, self.risk_scorer.w_lstm, self.risk_scorer.w_policy)
                self.risk_scorer.w_ae = t_cfg.cold_start_w_ae
                self.risk_scorer.w_lstm = t_cfg.cold_start_w_lstm
                self.risk_scorer.w_policy = t_cfg.cold_start_w_policy

                cold_scores = self.risk_scorer.compute(
                    ae_errors_aligned[cold_idx],
                    lstm_probs[cold_idx],
                    policy_scores[cold_idx],
                    ae_min=getattr(self, "ae_min", 0.0),
                    ae_max=getattr(self, "ae_max", 1.0)
                )
                risk_scores[cold_idx] = cold_scores

                self.risk_scorer.w_ae, self.risk_scorer.w_lstm, self.risk_scorer.w_policy = orig_weights

        logger.info(
            "Risk scoring complete. Mean: %.4f, Max: %.4f (Cold-starts: %d)",
            risk_scores.mean(), risk_scores.max(), aligned_cold_starts.sum()
        )
```

```
        aligned_entity_ids = entity_ids[offset:offset + min_len] if entity_ids is not None else np.array(["unknown"] * min_len)
        aligned_entity_types = entity_types[offset:offset + min_len] if entity_types is not None else np.array(["unknown"] * min_len)
        aligned_timestamps = timestamps[offset:offset + min_len] if timestamps is not None else np.array([""] * min_len)
        aligned_source_ips = source_ips[offset:offset + min_len] if source_ips is not None else None
        aligned_geo_locs = geo_locations[offset:offset + min_len] if geo_locations is not None else None
        aligned_resources = resources[offset:offset + min_len] if resources is not None else None

        signature_thresholds_path = paths.models_dir / "signature_thresholds.json"
        signature_matches: Dict[str, np.ndarray] = {}
        if signature_thresholds_path.exists():
            with open(signature_thresholds_path, "r") as f:
                signature_thresholds = json.load(f)
            signature_features = pd.DataFrame(unscaled_X_aligned, columns=self.feature_names)
            if aligned_source_ips is not None:
                signature_features["source_ip"] = aligned_source_ips
            signature_features["entity_id"] = aligned_entity_ids
            signature_features["timestamp"] = aligned_timestamps
            signature_matches = evaluate_signatures(signature_features, signature_thresholds)
        use_thresholds = getattr(self, "risk_thresholds", None) or risk_threshold
        if isinstance(use_thresholds, dict):
            ml_alert_mask = np.fromiter(
                 (risk_scores[i] >= use_thresholds.get(str(aligned_entity_types[i]), use_thresholds.get("global", risk_threshold)) for i
in range(min_len)),
                dtype=bool, count=min_len,
            )
        else:
            ml_alert_mask = risk_scores >= float(use_thresholds)
        signature_alert_mask = np.zeros(min_len, dtype=bool)
        for mask in signature_matches.values():
            signature_alert_mask |= mask
        combined_alert_mask = ml_alert_mask | signature_alert_mask
        signature_factors = [[] for _ in range(min_len)]
        for i in np.where(signature_alert_mask)[0]:
            parts = []
            if signature_matches.get("impossible_travel_signature", np.zeros(min_len, dtype=bool))[i]:
                       parts.append(f"travel_speed_kmh = {unscaled_X_aligned[i, self.feature_names.index('travel_speed_kmh')]:.0f} km/h
(physically implausible)")
            if signature_matches.get("device_spoofing_signature", np.zeros(min_len, dtype=bool))[i]:
                parts.append("device fingerprint mismatch with OS/browser change")
            if signature_matches.get("credential_stuffing_signature", np.zeros(min_len, dtype=bool))[i]:
                parts.append("high failed-login rate across multiple accounts from few source IPs")
            signature_factors[i] = parts

        contributing_factors = [[] for _ in range(min_len)]
        with latency_tracker:
            top_indices = np.where(ml_alert_mask)[0]
            top_k = len(top_indices)

            X_top = aligned_X[:min_len][top_indices]

            try:
                ig_results = explain_autoencoder(
                    self.autoencoder, X_top, self.feature_names, self.device, n_steps=20
                )
                attrs = ig_results["attributions"]

                feature_desc = {
                    "geo_distance_km": ("unusual geographic jump", "low geo distance"),
                    "travel_speed_kmh": ("impossible travel speed", "normal travel speed"),
                    "failed_logins_last_10min": ("high failed logins", "normal failed logins"),
                    "fingerprint_consistency": ("normal fingerprint consistency", "device fingerprint mismatch"),
                    "os_mismatch": ("OS mismatch", "OS matches baseline"),
                    "browser_mismatch": ("browser mismatch", "browser matches baseline"),
                    "sequence_entropy": ("highly anomalous sequence", "normal sequence entropy"),
                    "privilege_escalation_count": ("privilege escalation detected", "normal privilege usage"),
                    "resource_rarity": ("access to rare resource", "access to common resource"),
                    "session_duration": ("unusually long session", "unusually short session"),
                    "login_hour": ("unusual login hour", "typical login hour"),
                }

                for i, original_idx in enumerate(top_indices):
                    abs_attrs = np.abs(attrs[i])
                    sorted_indices = np.argsort(abs_attrs)[::-1]

                    filtered_top_3 = []
                    for f_idx in sorted_indices:
                        feat_name = self.feature_names[f_idx]
                        if feat_name.startswith("entity_type_"):
                            continue

                        is_high = X_top[i, f_idx] > 0
```

```
                if feat_name in feature_desc:
                    phrase = feature_desc[feat_name][0] if is_high else feature_desc[feat_name][1]
                else:
                    phrase = f"high {feat_name}" if is_high else f"low {feat_name}"

                filtered_top_3.append(phrase)
                if len(filtered_top_3) == 3:
                    break

            contributing_factors[original_idx] = filtered_top_3

        logger.info("Computed Captum IG attributions for top %d samples.", top_k)
    except Exception as e:
        logger.error("Failed to compute IG attributions: %s", e)

with latency_tracker:
    alerts_df = self.alert_engine.generate_alerts(
        risk_scores=risk_scores,
        ae_errors=ae_errors_aligned,
        lstm_probs=lstm_probs,
        policy_scores=policy_scores,
        entity_ids=aligned_entity_ids,
        entity_types=aligned_entity_types,
        timestamps=aligned_timestamps,
        source_ips=aligned_source_ips,
        geo_locations=aligned_geo_locs,
        resources=aligned_resources,
        feature_matrix=unscaled_X_aligned,
        feature_names=self.feature_names,
        risk_thresholds=use_thresholds,
        cold_starts=aligned_cold_starts,
        contributing_factors=contributing_factors,
        predicted_attack_types=predicted_attack_types[:min_len] if predicted_attack_types is not None else None,
        attack_probabilities=attack_probs[:min_len] if attack_probs is not None else None,
        attack_type_to_index=attack_type_to_index,
        signature_matches=signature_matches,
        signature_factors=signature_factors,
    )
logger.info("Alert generation complete. %d alerts generated.", len(alerts_df))

if attack_probs is not None:
    if isinstance(use_thresholds, dict):
        alert_mask = np.fromiter(
            (risk_scores[i] >= use_thresholds.get(str(aligned_entity_types[i]), use_thresholds.get("global", risk_threshold))
             for i in range(min_len)),
            dtype=bool,
            count=min_len,
        )
    else:
        alert_mask = risk_scores >= float(use_thresholds)
    probability_columns = [f"prob_{idx_to_attack[i]}" for i in range(attack_probs.shape[1])]
    probability_report = pd.DataFrame(attack_probs[:min_len], columns=probability_columns)
    probability_report["true_attack_type"] = (
        attack_types[offset:offset + min_len] if attack_types is not None else "unknown"
    )
    probability_report["predicted_attack_type"] = predicted_attack_types[:min_len]
    probability_report = probability_report.loc[alert_mask].reset_index(drop=True)
    probability_report.to_csv(paths.outputs_dir / "classifier_softmax_alerts.csv", index=False)
    probability_summary = probability_report.groupby("true_attack_type", dropna=False)[probability_columns].mean()
    probability_summary.insert(0, "alert_count", probability_report.groupby("true_attack_type", dropna=False).size())
    probability_summary.to_csv(paths.outputs_dir / "classifier_softmax_summary.csv")

evaluation: Optional[EvaluationResult] = None
total_inference_time = latency_tracker.get_latencies().sum() / 1000.0
if y_true is not None:
    y_true_aligned = y_true[offset:offset + min_len]
    aligned_attack_types = attack_types[offset:offset + min_len] if attack_types is not None else None
    evaluation = evaluate_predictions(
        y_true=y_true_aligned,
        y_scores=risk_scores,
        threshold=use_thresholds,
        predictions=combined_alert_mask.astype(int),
        reconstruction_errors=ae_errors_aligned,
        risk_scores=risk_scores,
        attack_types=aligned_attack_types,
        entity_types=aligned_entity_types,
        latencies_ms=latency_tracker.get_latencies(),
        warmup_calls=3,
        total_inference_time_sec=total_inference_time,
        total_samples_inferred=min_len,
```

```
            )
            report_name = "evaluation_report" if split == "test" else f"{split}_evaluation_report"
            generate_report(evaluation, paths.outputs_dir, report_name=report_name)
            logger.info("Evaluation report saved to %s (report_name: %s)", paths.outputs_dir, report_name)

            cm = evaluation.confusion_mat
            tn, fp, fn, tp = cm[0][0], cm[0][1], cm[1][0], cm[1][1]
            eval_alert_count = fp + tp
            actual_alert_count = len(alerts_df)
            assert actual_alert_count == eval_alert_count, (
                f"LOUD FAILURE: len(alerts.csv) ({actual_alert_count}) "
                f"does not match evaluation confusion matrix FP+TP ({fp}+{tp} = {eval_alert_count})!"
            )
            logger.info("ASSERTION PASSED: len(alerts.csv) (%d) == FP+TP (%d)", actual_alert_count, eval_alert_count)

        return {
            "ae_errors": ae_errors_aligned,
            "lstm_probs": lstm_probs,
            "policy_scores": policy_scores,
            "risk_scores": risk_scores,
            "alerts_df": alerts_df,
            "evaluation": evaluation,
            "latency_tracker": latency_tracker,
            "attack_probabilities": attack_probs[:min_len] if attack_probs is not None else None,
        }

    def _load_split_data(
        self,
        split: str,
        paths: Any,
        feature_cfg: Any,
    ) -> Tuple[
        np.ndarray,
        Optional[np.ndarray],
        Optional[np.ndarray],
        Optional[np.ndarray],
        Optional[np.ndarray],
        Optional[np.ndarray],
        Optional[np.ndarray],
        Optional[np.ndarray],
        Optional[np.ndarray],
        Optional[np.ndarray],
        Optional[np.ndarray],
    ]:
        proc_dir = paths.processed_data_dir
        file_map = {
            "train": (feature_cfg.x_train_file, feature_cfg.y_train_file),
            "val": (feature_cfg.x_val_file, feature_cfg.y_val_file),
            "test": (feature_cfg.x_test_file, feature_cfg.y_test_file),
        }
        x_file, y_file = file_map[split]
        X = np.load(proc_dir / x_file)
        y = np.load(proc_dir / y_file)

        logger.info("Loaded %s split: X=%s, y=%s", split, X.shape, y.shape)

        df_logs = pd.read_csv(paths.raw_logs_file, parse_dates=["timestamp"])
        df_truth = pd.read_csv(paths.ground_truth_file, parse_dates=["timestamp"])
        df_logs = df_logs.sort_values("timestamp").reset_index(drop=True)
        df_truth = df_truth.sort_values("timestamp").reset_index(drop=True)

        n = len(df_logs)
        train_end = int(n * feature_cfg.train_ratio)
        val_end = train_end + int(n * feature_cfg.val_ratio)

        if split == "train":
            df_slice = df_logs.iloc[:train_end]
            truth_slice = df_truth.iloc[:train_end]
        elif split == "val":
            df_slice = df_logs.iloc[train_end:val_end]
            truth_slice = df_truth.iloc[train_end:val_end]
        else:
            df_slice = df_logs.iloc[val_end:]
            truth_slice = df_truth.iloc[val_end:]

        entity_ids = df_slice["entity_id"].values
        entity_types = df_slice["entity_type"].values
        timestamps = df_slice["timestamp"].astype(str).values
        source_ips = df_slice["source_ip"].values
        geo_locs = df_slice["geo_location"].values
        resources = df_slice["resource_accessed"].values
```

```
        attack_types_arr = truth_slice["attack_type"].values

        first_seen = df_logs.groupby("entity_id")["timestamp"].transform("min")
        days_since_first = (df_logs["timestamp"] - first_seen).dt.total_seconds() / 86400.0
        min_days_map = self.config.training.cold_start_min_history_days
        min_days_series = df_logs["entity_type"].map(min_days_map).fillna(0.0)
        df_logs["cold_start"] = days_since_first < min_days_series

        if split == "train":
            cold_starts_slice = df_logs.iloc[:train_end]["cold_start"].values
        elif split == "val":
            cold_starts_slice = df_logs.iloc[train_end:val_end]["cold_start"].values
        else:
            cold_starts_slice = df_logs.iloc[val_end:]["cold_start"].values

        return X, y, entity_types, entity_ids, timestamps, source_ips, geo_locs, resources, attack_types_arr, cold_starts_slice

    @classmethod
    def from_saved_models(
        cls,
        config: Optional[ProjectConfig] = None,
        risk_threshold: float = 0.5,
    ) -> "InferencePipeline":
        if config is None:
            config = get_project_config()

        paths = config.paths
        proc_dir = paths.processed_data_dir
        models_dir = paths.models_dir
        model_cfg = config.model
        training_cfg = config.training

        device = _get_device(training_cfg.device)
        set_torch_seed(training_cfg.random_seed)

        metadata_path = proc_dir / config.features.feature_metadata_file
        with open(metadata_path, "r", encoding="utf-8") as f:
            metadata = json.load(f)
        feature_names = metadata["feature_names"]
        num_features = metadata["num_features"]

        logger.info("Loaded feature metadata: %d features.", num_features)

        with open(proc_dir / config.features.scaler_file, "rb") as f:
            scaler = pickle.load(f)
        with open(proc_dir / config.features.encoder_file, "rb") as f:
            encoder = pickle.load(f)

        autoencoder = SharedAutoencoder.from_config(
            input_dim=num_features, cfg=model_cfg.autoencoder
        )
        ae_path = models_dir / "autoencoder.pt"
        if ae_path.exists():
            load_model(autoencoder, ae_path, device)
        else:
            logger.warning("Autoencoder model file not found at %s", ae_path)
        autoencoder = autoencoder.to(device)

        lstm = LSTMSequenceClassifier.from_config(
            input_dim=num_features, cfg=model_cfg.lstm
        )
        lstm_path = models_dir / "lstm_classifier.pt"
        if lstm_path.exists():
            load_model(lstm, lstm_path, device)
        else:
            logger.warning("LSTM model file not found at %s", lstm_path)
        lstm = lstm.to(device)

        thresholds_path = models_dir / "thresholds.json"
        if thresholds_path.exists():
            thresholds = load_thresholds(thresholds_path)
        else:
            logger.warning("Thresholds not found. Using default 0.5 for all types.")
            thresholds = {etype: 0.5 for etype in ENTITY_TYPES}

        policy_engine = PolicyEngine.from_config(model_cfg.policy_engine)
        risk_scorer = RiskScorer.from_config(model_cfg.risk_scorer)
        alert_engine = AlertEngine.from_config(config, risk_threshold)

        rw_path = models_dir / "risk_weights.json"
        ae_min, ae_max = 0.0, 1.0
```

```
    if rw_path.exists():
        with open(rw_path, "r") as f:
            rw = json.load(f)
        risk_scorer.update_weights(rw["w_ae"], rw["w_lstm"], rw["w_policy"])
        ae_min = rw.get("ae_min", 0.0)
        ae_max = rw.get("ae_max", 1.0)

    rt_path = models_dir / "risk_thresholds.json"
    risk_thresholds = None
    if rt_path.exists():
        with open(rt_path, "r") as f:
            risk_thresholds = json.load(f)

    ps_path = models_dir / "platt_scaler.pkl"
    platt_scaler = None
    if ps_path.exists():
        with open(ps_path, "rb") as f:
            platt_scaler = pickle.load(f)

    pipeline = cls(
        autoencoder=autoencoder,
        lstm_classifier=lstm,
        policy_engine=policy_engine,
        risk_scorer=risk_scorer,
        alert_engine=alert_engine,
        thresholds=thresholds,
        scaler=scaler,
        encoder=encoder,
        feature_names=feature_names,
        device=device,
        sequence_length=model_cfg.lstm.sequence_length,
        config=config,
        risk_thresholds=risk_thresholds,
        platt_scaler=platt_scaler,
        ae_min=ae_min,
        ae_max=ae_max,
    )

    ac_path = models_dir / "attack_classifier.pt"
    if ac_path.exists():
        attack_classifier = AttackTypeClassifier.from_config(
            input_dim=model_cfg.lstm.hidden_dim * (2 if model_cfg.lstm.bidirectional else 1),
            cfg=model_cfg.attack_classifier
        )
        load_model(attack_classifier, ac_path, device)
        pipeline.attack_classifier = attack_classifier.to(device)

    logger.info("InferencePipeline initialised from saved models.")
    return pipeline
```

File: evaluation.py

Description: Evaluation Metrics & Confusion Matrix Reports

```
"""Comprehensive evaluation suite computing precision, recall, F1, ROC-AUC, PR-AUC, FPR, and latency metrics.

Generates evaluation reports with full attack-wise and persona-wise performance breakdowns.
"""

from __future__ import annotations

import json
import logging
import time
from dataclasses import dataclass, field
from pathlib import Path
from typing import Any, Callable, Dict, List, Optional, Tuple, Union

import numpy as np
from sklearn.metrics import (
    auc,
    average_precision_score,
    balanced_accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
    precision_recall_curve,
    precision_score,
    recall_score,
    roc_auc_score,
    roc_curve,
)

from config import ATTACK_TYPES, ENTITY_TYPES, ProjectConfig, get_project_config

logger = logging.getLogger(__name__)

@dataclass
class DistributionStats:

    mean: float
    std: float
    min_val: float
    max_val: float
    p25: float
    p50: float
    p75: float
    p90: float
    p95: float
    p99: float

    def to_dict(self) -> Dict[str, float]:
        return {
            "mean": self.mean,
            "std": self.std,
            "min": self.min_val,
            "max": self.max_val,
            "p25": self.p25,
            "p50": self.p50,
            "p75": self.p75,
            "p90": self.p90,
            "p95": self.p95,
            "p99": self.p99,
        }

@dataclass
class ClassMetrics:

    precision: float
    recall: float
    f1: float
    support: int

    def to_dict(self) -> Dict[str, Any]:
        return {
            "precision": round(self.precision, 6),
            "recall": round(self.recall, 6),
            "f1": round(self.f1, 6),
            "support": self.support,
        }
```

```
@dataclass
class EvaluationResult:

    precision: float
    recall: float
    f1: float
    pr_auc: float
    roc_auc: float
    confusion_mat: List[List[int]]
    false_positive_rate: float
    reconstruction_error_stats: Optional[DistributionStats] = None
    risk_score_stats: Optional[DistributionStats] = None
    benign_risk_stats: Optional[DistributionStats] = None
    attack_risk_stats: Optional[DistributionStats] = None
    attack_wise_metrics: Dict[str, ClassMetrics] = field(default_factory=dict)
    persona_wise_metrics: Dict[str, ClassMetrics] = field(default_factory=dict)
    persona_confusion_matrices: Dict[str, List[List[int]]] = field(default_factory=dict)
    latency_stats: Optional[DistributionStats] = None
    warmup_latency_stats: Optional[DistributionStats] = None
    steady_state_latency_stats: Optional[DistributionStats] = None
    throughput_events_per_sec: Optional[float] = None
    threshold: float = 0.5
    thresholds_dict: Optional[Dict[str, float]] = None
    total_samples: int = 0
    total_positives: int = 0
    total_negatives: int = 0
    roc_curve_data: Optional[Dict[str, List[float]]] = None
    pr_curve_data: Optional[Dict[str, List[float]]] = None
    threshold_metrics: Optional[Dict[str, List[float]]] = None

    def to_dict(self) -> Dict[str, Any]:
        result: Dict[str, Any] = {
            "precision": round(self.precision, 6),
            "recall": round(self.recall, 6),
            "f1": round(self.f1, 6),
            "pr_auc": round(self.pr_auc, 6),
            "roc_auc": round(self.roc_auc, 6),
            "confusion_matrix": self.confusion_mat,
            "false_positive_rate": round(self.false_positive_rate, 6),
            "threshold": self.threshold,
            "total_samples": self.total_samples,
            "total_positives": self.total_positives,
            "total_negatives": self.total_negatives,
        }
        if self.thresholds_dict:
            result["thresholds_dict"] = self.thresholds_dict
        if self.reconstruction_error_stats is not None:
            result["reconstruction_error_distribution"] = self.reconstruction_error_stats.to_dict()
        if self.risk_score_stats is not None:
            result["risk_score_distribution"] = self.risk_score_stats.to_dict()
        if self.benign_risk_stats is not None:
            result["benign_risk_distribution"] = self.benign_risk_stats.to_dict()
        if self.attack_risk_stats is not None:
            result["attack_risk_distribution"] = self.attack_risk_stats.to_dict()
        if self.attack_wise_metrics:
            result["attack_wise_metrics"] = {k: v.to_dict() for k, v in self.attack_wise_metrics.items()}
        if self.persona_wise_metrics:
            result["persona_wise_metrics"] = {k: v.to_dict() for k, v in self.persona_wise_metrics.items()}
        if self.persona_confusion_matrices:
            result["persona_confusion_matrices"] = self.persona_confusion_matrices
        if self.latency_stats is not None:
            result["latency_stats_ms"] = self.latency_stats.to_dict()
        if self.warmup_latency_stats is not None:
            result["warmup_latency_stats_ms"] = self.warmup_latency_stats.to_dict()
        if self.steady_state_latency_stats is not None:
            result["steady_state_latency_stats_ms"] = self.steady_state_latency_stats.to_dict()
        if self.throughput_events_per_sec is not None:
            result["throughput_events_per_sec"] = round(self.throughput_events_per_sec, 2)
        if self.roc_curve_data is not None:
            result["roc_curve_data"] = self.roc_curve_data
        if self.pr_curve_data is not None:
            result["pr_curve_data"] = self.pr_curve_data
        if self.threshold_metrics is not None:
            result["threshold_metrics"] = self.threshold_metrics
        return result

def compute_distribution_stats(values: np.ndarray) -> DistributionStats:
    if len(values) == 0:
        return DistributionStats(
            mean=0.0, std=0.0, min_val=0.0, max_val=0.0,
```

```
            p25=0.0, p50=0.0, p75=0.0, p90=0.0, p95=0.0, p99=0.0,
        )
    return DistributionStats(
        mean=float(np.mean(values)),
        std=float(np.std(values)),
        min_val=float(np.min(values)),
        max_val=float(np.max(values)),
        p25=float(np.percentile(values, 25)),
        p50=float(np.percentile(values, 50)),
        p75=float(np.percentile(values, 75)),
        p90=float(np.percentile(values, 90)),
        p95=float(np.percentile(values, 95)),
        p99=float(np.percentile(values, 99)),
    )

def _downsample_curve(x: np.ndarray, y: np.ndarray, t: np.ndarray, n_points: int = 100) -> Tuple[List[float], List[float], List[float]]:
    if len(x) <= n_points:
        return x.tolist(), y.tolist(), t.tolist()
    indices = np.linspace(0, len(x) - 1, n_points, dtype=int)
    return x[indices].tolist(), y[indices].tolist(), t[indices].tolist()

def compute_fpr(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    cm = confusion_matrix(y_true, y_pred, labels=[0, 1])
    tn, fp = cm[0, 0], cm[0, 1]
    if (fp + tn) == 0:
        return 0.0
    return float(fp / (fp + tn))

def evaluate_predictions(
    y_true: np.ndarray,
    y_scores: np.ndarray,
    threshold: Union[float, Dict[str, float]] = 0.5,
    reconstruction_errors: Optional[np.ndarray] = None,
    risk_scores: Optional[np.ndarray] = None,
    attack_types: Optional[np.ndarray] = None,
    entity_types: Optional[np.ndarray] = None,
    latencies_ms: Optional[np.ndarray] = None,
    warmup_calls: int = 3,
    total_inference_time_sec: Optional[float] = None,
    total_samples_inferred: Optional[int] = None,
    predictions: Optional[np.ndarray] = None,
) -> EvaluationResult:

    thresholds_dict = None
    if predictions is not None:
        y_pred = np.asarray(predictions, dtype=int)
        threshold_val = 0.0
        thresholds_dict = threshold if isinstance(threshold, dict) else None
    elif isinstance(threshold, dict) and entity_types is not None:
        y_pred = np.zeros_like(y_scores, dtype=int)
        for i, etype in enumerate(entity_types):
            t = threshold.get(etype, 0.5)
            if y_scores[i] >= t:
                y_pred[i] = 1
        threshold_val = sum(threshold.values()) / len(threshold) if threshold else 0.5
        thresholds_dict = threshold
    else:
        t = threshold if isinstance(threshold, float) else 0.5
        y_pred = (y_scores >= t).astype(int)
        threshold_val = t

    prec = float(precision_score(y_true, y_pred, zero_division=0))
    rec = float(recall_score(y_true, y_pred, zero_division=0))
    f1_val = float(f1_score(y_true, y_pred, zero_division=0))

    try:
        roc_auc_val = float(roc_auc_score(y_true, y_scores))
        fpr_c, tpr_c, thresh_roc = roc_curve(y_true, y_scores)
        fpr_ds, tpr_ds, thresh_roc_ds = _downsample_curve(fpr_c, tpr_c, thresh_roc)
        roc_curve_data = {"fpr": fpr_ds, "tpr": tpr_ds, "thresholds": thresh_roc_ds}
    except ValueError:
        roc_auc_val = 0.0
        roc_curve_data = None
        logger.warning("ROC-AUC could not be computed (single class present).")

    try:
        pr_auc_val = float(average_precision_score(y_true, y_scores))
        p_c, r_c, thresh_pr = precision_recall_curve(y_true, y_scores)
        thresh_pr = np.append(thresh_pr, 1.0)
        p_ds, r_ds, thresh_pr_ds = _downsample_curve(p_c, r_c, thresh_pr)
        pr_curve_data = {"precision": p_ds, "recall": r_ds, "thresholds": thresh_pr_ds}
```

```
        threshold_metrics = {
            "thresholds": thresh_pr_ds,
            "precision": p_ds,
            "recall": r_ds,
            "f1": [2 * p * r / (p + r) if (p+r) > 0 else 0.0 for p, r in zip(p_ds, r_ds)]
        }
    except ValueError:
        pr_auc_val = 0.0
        pr_curve_data = None
        threshold_metrics = None
        logger.warning("PR-AUC could not be computed.")

    cm = confusion_matrix(y_true, y_pred, labels=[0, 1])
    cm_list = cm.tolist()
    fpr = compute_fpr(y_true, y_pred)

    recon_stats = compute_distribution_stats(reconstruction_errors) if reconstruction_errors is not None else None
    risk_stats = compute_distribution_stats(risk_scores) if risk_scores is not None else None

    benign_risk_stats = None
    attack_risk_stats = None
    if risk_scores is not None:
        benign_risk_stats = compute_distribution_stats(risk_scores[y_true == 0])
        attack_risk_stats = compute_distribution_stats(risk_scores[y_true == 1])

    lat_stats = compute_distribution_stats(latencies_ms) if latencies_ms is not None else None
    warmup_lat_stats = None
    steady_lat_stats = None
    throughput = None
    if latencies_ms is not None and len(latencies_ms) > warmup_calls:
        warmup_lat_stats = compute_distribution_stats(latencies_ms[:warmup_calls])
        steady_lat_stats = compute_distribution_stats(latencies_ms[warmup_calls:])
        logger.info(
            "Latency split -- warm-up P50=%.2fms (n=%d), steady-state P50=%.2fms P95=%.2fms P99=%.2fms (n=%d)",
            warmup_lat_stats.p50, warmup_calls,
            steady_lat_stats.p50, steady_lat_stats.p95, steady_lat_stats.p99,
            len(latencies_ms) - warmup_calls,
        )
    if total_inference_time_sec is not None and total_samples_inferred is not None and total_inference_time_sec > 0:
        throughput = total_samples_inferred / total_inference_time_sec
        logger.info("Throughput: %.2f events/sec", throughput)

    atk_metrics: Dict[str, ClassMetrics] = {}
    if attack_types is not None:
        for atype in ATTACK_TYPES:
            mask = attack_types == atype
            if mask.sum() == 0:
                continue
            atk_metrics[atype] = ClassMetrics(
                precision=0.0,
                recall=float(y_pred[mask].mean()),
                f1=0.0,
                support=int(mask.sum()),
            )

    persona_metrics: Dict[str, ClassMetrics] = {}
    persona_cms: Dict[str, List[List[int]]] = {}
    if entity_types is not None:
        for etype in ENTITY_TYPES:
            mask = entity_types == etype
            if mask.sum() == 0:
                continue
            p_true = y_true[mask]
            p_pred = y_pred[mask]
            persona_metrics[etype] = ClassMetrics(
                precision=float(precision_score(p_true, p_pred, zero_division=0)),
                recall=float(recall_score(p_true, p_pred, zero_division=0)),
                f1=float(f1_score(p_true, p_pred, zero_division=0)),
                support=int(mask.sum()),
            )
            persona_cms[etype] = confusion_matrix(p_true, p_pred, labels=[0, 1]).tolist()

    total_pos = int(y_true.sum())
    total_neg = int(len(y_true) - total_pos)

    result = EvaluationResult(
        precision=prec,
        recall=rec,
        f1=f1_val,
        pr_auc=pr_auc_val,
```

```
        roc_auc=roc_auc_val,
        confusion_mat=cm_list,
        false_positive_rate=fpr,
        reconstruction_error_stats=recon_stats,
        risk_score_stats=risk_stats,
        benign_risk_stats=benign_risk_stats,
        attack_risk_stats=attack_risk_stats,
        attack_wise_metrics=atk_metrics,
        persona_wise_metrics=persona_metrics,
        persona_confusion_matrices=persona_cms,
        latency_stats=lat_stats,
        warmup_latency_stats=warmup_lat_stats,
        steady_state_latency_stats=steady_lat_stats,
        throughput_events_per_sec=throughput,
        threshold=threshold_val,
        thresholds_dict=thresholds_dict,
        total_samples=len(y_true),
        total_positives=total_pos,
        total_negatives=total_neg,
        roc_curve_data=roc_curve_data,
        pr_curve_data=pr_curve_data,
        threshold_metrics=threshold_metrics,
    )

    logger.info(
        "Evaluation complete -- P=%.4f, R=%.4f, F1=%.4f, ROC-AUC=%.4f, PR-AUC=%.4f, FPR=%.4f",
        prec, rec, f1_val, roc_auc_val, pr_auc_val, fpr,
    )

    return result

def generate_report(
    result: EvaluationResult,
    output_dir: Path,
    report_name: str = "evaluation_report",
) -> Path:
    output_dir = Path(output_dir)
    output_dir.mkdir(parents=True, exist_ok=True)

    result_dict = result.to_dict()

    json_path = output_dir / f"{report_name}.json"
    with open(json_path, "w", encoding="utf-8") as f:
        json.dump(result_dict, f, indent=2)

    txt_path = output_dir / f"{report_name}.txt"
    with open(txt_path, "w", encoding="utf-8") as f:
        f.write("=" * 80 + "\n")
        f.write("BEHAVIORAL ANOMALY DETECTION -- EVALUATION REPORT\n")
        f.write("=" * 80 + "\n\n")

        f.write("OVERALL METRICS\n")
        f.write("-" * 40 + "\n")
        f.write(f"  Precision     : {result.precision:.6f}\n")
        f.write(f"  Recall        : {result.recall:.6f}\n")
        f.write(f"  F1-Score      : {result.f1:.6f}\n")
        f.write(f"  ROC-AUC       : {result.roc_auc:.6f}\n")
        f.write(f"  PR-AUC        : {result.pr_auc:.6f}\n")
        f.write(f"  FPR           : {result.false_positive_rate:.6f}\n")

        if result.thresholds_dict:
            f.write(f"  Thresholds    : {json.dumps(result.thresholds_dict)}\n")
        else:
            f.write(f"  Threshold     : {result.threshold:.4f}\n")

        f.write(f"  Total Samples : {result.total_samples}\n")
        f.write(f"  Positives     : {result.total_positives}\n")
        f.write(f"  Negatives     : {result.total_negatives}\n\n")

        f.write("CONFUSION MATRIX\n")
        f.write("-" * 40 + "\n")
        cm = result.confusion_mat
        f.write(f"  TN={cm[0][0]:>7}  FP={cm[0][1]:>7}\n")
        f.write(f"  FN={cm[1][0]:>7}  TP={cm[1][1]:>7}\n\n")

        if result.risk_score_stats is not None:
            f.write("HYBRID RISK SCORE DISTRIBUTION\n")
            f.write("-" * 40 + "\n")
            stats = result.risk_score_stats
            f.write(f"  Mean : {stats.mean:.6f}  Std : {stats.std:.6f}\n")
            f.write(f"  Min  : {stats.min_val:.6f}  Max : {stats.max_val:.6f}\n")
```

```
            f.write(f"  P50  : {stats.p50:.6f}  P95 : {stats.p95:.6f}\n\n")

        if result.steady_state_latency_stats is not None:
            f.write("LATENCY (STEADY-STATE, EXCL. WARM-UP)\n")
            f.write("-" * 40 + "\n")
            ss = result.steady_state_latency_stats
            f.write(f"  P50  : {ss.p50:.2f}ms  P95 : {ss.p95:.2f}ms  P99 : {ss.p99:.2f}ms\n")
            if result.warmup_latency_stats is not None:
                wu = result.warmup_latency_stats
                f.write(f"  Warm-up P50: {wu.p50:.2f}ms (excluded from above)\n")
            if result.throughput_events_per_sec is not None:
                f.write(f"  Throughput  : {result.throughput_events_per_sec:.2f} events/sec\n")
            f.write("\n")

        if result.attack_wise_metrics:
            f.write("ATTACK-WISE PERFORMANCE\n")
            f.write("-" * 40 + "\n")
            f.write(f"  {'Attack Type':<30} {'Recall':>8} {'N':>6}\n")
            for atype, m in sorted(result.attack_wise_metrics.items()):
                f.write(
                    f"  {atype:<30} {m.recall:>8.4f} {m.support:>6}\n"
                )
            f.write("\n")

        if result.persona_wise_metrics:
            f.write("PERSONA-WISE PERFORMANCE\n")
            f.write("-" * 40 + "\n")
            f.write(f"  {'Persona':<30} {'Prec':>8} {'Rec':>8} {'F1':>8} {'N':>6}\n")
            for etype, m in sorted(result.persona_wise_metrics.items()):
                f.write(
                    f"  {etype:<30} {m.precision:>8.4f} {m.recall:>8.4f} "
                    f"{m.f1:>8.4f} {m.support:>6}\n"
                )
            f.write("\n")

        f.write("=" * 80 + "\n")

    logger.info("Reports saved to %s (.json and .txt)", output_dir)
    return json_path

class LatencyTracker:
    def __init__(self) -> None:
        self._latencies: List[float] = []
        self._start_time: float = 0.0

    def __enter__(self) -> "LatencyTracker":
        self._start_time = time.perf_counter()
        return self

    def __exit__(self, *args: Any) -> None:
        elapsed_ms = (time.perf_counter() - self._start_time) * 1000.0
        self._latencies.append(elapsed_ms)

    def record(self, latency_ms: float) -> None:
        self._latencies.append(latency_ms)

    def get_latencies(self) -> np.ndarray:
        return np.array(self._latencies, dtype=np.float64)

    def get_stats(self) -> DistributionStats:
        return compute_distribution_stats(self.get_latencies())

def find_optimal_threshold(
    y_true: np.ndarray,
    y_scores: np.ndarray,
    metric: str = "f1",
    constraint: float = 0.90,
    thresholds: Optional[np.ndarray] = None,
    attack_labels: Optional[np.ndarray] = None,
    min_attack_recall: float = 0.0,
) -> Tuple[float, float]:
    if thresholds is None:
        thresholds = np.linspace(0.01, 0.99, 100)

    best_threshold = 0.5
    best_value = -2.0

    if len(y_true) == 0:
        return best_threshold, best_value

    for t in thresholds:
```

```
        y_pred = (y_scores >= t).astype(int)

        if metric == "f1":
            val = float(f1_score(y_true, y_pred, zero_division=0))
        elif metric == "balanced_accuracy":
            val = float(balanced_accuracy_score(y_true, y_pred))
        elif metric == "precision_at_recall":
            rec = float(recall_score(y_true, y_pred, zero_division=0))
            if rec >= constraint:
                val = float(precision_score(y_true, y_pred, zero_division=0))
            else:
                val = 0.0
        elif metric == "recall_at_fpr":
            fpr = compute_fpr(y_true, y_pred)
            if fpr <= constraint:
                val = float(recall_score(y_true, y_pred, zero_division=0))
            else:
                val = 0.0
        else:
            val = float(f1_score(y_true, y_pred, zero_division=0))

        if min_attack_recall > 0.0 and attack_labels is not None:
            min_rec = 1.0
            for atype in np.unique(attack_labels):
                if atype == "benign":
                    continue
                mask_atype = (attack_labels == atype)
                if mask_atype.sum() > 0:
                    rec = (y_pred[mask_atype] == 1).sum() / mask_atype.sum()
                    min_rec = min(min_rec, float(rec))
            if min_rec < min_attack_recall:
                val = -1.0

        if val > best_value:
            best_value = val
            best_threshold = float(t)

    if best_value == -1.0:
        best_threshold = float(thresholds[0])
        best_value = 0.0

    return best_threshold, best_value
```

File: pipeline_runner.py

Description: Pipeline CLI Entrypoint & Stage Orchestrator

```
"""Command-line orchestrator for executing individual or end-to-end pipeline stages.

Supported stages: generate, features, train, calibrate, infer, all.
"""

from __future__ import annotations

import argparse
import json
import logging
import pickle
import sys
import time
from pathlib import Path
from typing import Any, Dict, List, Optional
from sklearn.linear_model import LogisticRegression

import numpy as np
import pandas as pd
import torch

from config import (
    ENTITY_TYPES,
    ATTACK_TYPES,
    NUM_ENTITY_TYPES,
    ATTACK_TYPE_LABELS,
    ProjectConfig,
    get_project_config,
)
from data_generator import generate_dataset
from feature_pipeline import run_feature_pipeline
from models import (
    FocalLoss,
    LSTMSequenceClassifier,
    PolicyEngine,
    RiskScorer,
    SharedAutoencoder,
    create_sequences,
    infer_autoencoder,
    save_model,
    save_thresholds,
    set_torch_seed,
    train_attack_classifier,
    train_autoencoder,
    train_lstm,
    AttackTypeClassifier,
    MultiClassFocalLoss,
)
from models import _get_device
from inference_pipeline import InferencePipeline
from evaluation import generate_report, find_optimal_threshold
from signature_rules import tune_signatures

logger = logging.getLogger(__name__)

def run_stage_generate(config: ProjectConfig) -> None:
    logger.info("=" * 60)
    logger.info("STAGE 1: Synthetic Data Generation")
    logger.info("=" * 60)
    start = time.perf_counter()
    generate_dataset(config)
    elapsed = time.perf_counter() - start
    logger.info("Data generation completed in %.2f seconds.", elapsed)

def run_stage_features(config: ProjectConfig) -> Dict[str, Any]:
    logger.info("=" * 60)
    logger.info("STAGE 2: Feature Engineering")
    logger.info("=" * 60)
    start = time.perf_counter()
    result = run_feature_pipeline(config)
    elapsed = time.perf_counter() - start
    logger.info("Feature engineering completed in %.2f seconds.", elapsed)
    return result

def run_stage_train(config: ProjectConfig) -> Dict[str, Any]:
    logger.info("=" * 60)
```

```
    logger.info("STAGE 3: Model Training")
    logger.info("=" * 60)
    start = time.perf_counter()

    paths = config.paths
    proc_dir = paths.processed_data_dir
    models_dir = paths.models_dir
    model_cfg = config.model
    training_cfg = config.training
    feature_cfg = config.features

    set_torch_seed(training_cfg.random_seed)
    device = _get_device(training_cfg.device)

    X_train = np.load(proc_dir / feature_cfg.x_train_file)
    X_val = np.load(proc_dir / feature_cfg.x_val_file)
    y_train = np.load(proc_dir / feature_cfg.y_train_file)
    y_val = np.load(proc_dir / feature_cfg.y_val_file)

    with open(proc_dir / feature_cfg.feature_metadata_file, "r") as f:
        metadata = json.load(f)
    num_features = metadata["num_features"]

    logger.info(
        "Loaded training data: X_train=%s, X_val=%s",
        X_train.shape, X_val.shape,
    )

    logger.info("Training SharedAutoencoder...")
    autoencoder = SharedAutoencoder.from_config(
        input_dim=num_features, cfg=model_cfg.autoencoder
    )
    ae_history = train_autoencoder(
        autoencoder, X_train, X_val, training_cfg
    )
    save_model(
        autoencoder,
        models_dir / "autoencoder.pt",
        metadata={"input_dim": num_features, "config": "autoencoder"},
    )

    df_logs = pd.read_csv(paths.raw_logs_file, parse_dates=["timestamp"])
    df_logs = df_logs.sort_values("timestamp").reset_index(drop=True)
    n_total = len(df_logs)
    train_end = int(n_total * feature_cfg.train_ratio)
    train_entity_types = df_logs.iloc[:train_end]["entity_type"].values

    thresholds = autoencoder.compute_adaptive_thresholds(
        X_train, train_entity_types, model_cfg.autoencoder.threshold_percentile, device
    )
    save_thresholds(thresholds, models_dir / "thresholds.json")

    logger.info("Computing training AE errors for drift reference...")
    train_ae_errors = infer_autoencoder(autoencoder, X_train, device)
    np.savez(
        models_dir / "ae_reference_stats.npz",
        errors=train_ae_errors,
        entity_types=train_entity_types
    )
    logger.info("Saved AE reference stats to ae_reference_stats.npz")

    logger.info("Training LSTMSequenceClassifier...")
    X_train_seq, y_train_seq = create_sequences(
        X_train, y_train, model_cfg.lstm.sequence_length
    )
    X_val_seq, y_val_seq = create_sequences(
        X_val, y_val, model_cfg.lstm.sequence_length
    )

    lstm = LSTMSequenceClassifier.from_config(
        input_dim=num_features, cfg=model_cfg.lstm
    )
    focal_loss = FocalLoss.from_config(model_cfg.focal_loss)
    lstm_history = train_lstm(
        lstm, X_train_seq, y_train_seq, X_val_seq, y_val_seq,
        focal_loss, training_cfg,
    )
    save_model(
        lstm,
        models_dir / "lstm_classifier.pt",
        metadata={"input_dim": num_features, "config": "lstm"},
```

```
    )

    logger.info("Fitting Platt Scaling calibrator on validation logits...")
    lstm.eval()
    val_tensor_seq = torch.tensor(X_val_seq, dtype=torch.float32, device=device)
    with torch.no_grad():
        val_logits = lstm(val_tensor_seq).cpu().numpy()

    platt_scaler = LogisticRegression(solver="lbfgs")
    platt_scaler.fit(val_logits.reshape(-1, 1), y_val_seq)
    with open(paths.platt_scaler_file, "wb") as f:
        pickle.dump(platt_scaler, f)

    val_probs = platt_scaler.predict_proba(val_logits.reshape(-1, 1))[:, 1]

    logger.info("Extracting frozen LSTM features for Attack Type Classifier...")
    df_truth = pd.read_csv(paths.ground_truth_file, parse_dates=["timestamp"])
    df_truth = df_truth.sort_values("timestamp").reset_index(drop=True)
    val_end = train_end + int(n_total * feature_cfg.val_ratio)

    train_attack_types = df_truth.iloc[:train_end]["attack_type"].values
    val_attack_types = df_truth.iloc[train_end:val_end]["attack_type"].values

    train_attack_labels = np.array([ATTACK_TYPE_LABELS.get(at, 0) for at in train_attack_types])
    val_attack_labels = np.array([ATTACK_TYPE_LABELS.get(at, 0) for at in val_attack_types])

    offset = model_cfg.lstm.sequence_length - 1
    train_attack_labels_seq = train_attack_labels[offset:]
    val_attack_labels_seq = val_attack_labels[offset:]

    lstm.eval()
    with torch.no_grad():
        train_hidden = lstm.extract_hidden(torch.tensor(X_train_seq, dtype=torch.float32, device=device)).cpu().numpy()
        val_hidden = lstm.extract_hidden(torch.tensor(X_val_seq, dtype=torch.float32, device=device)).cpu().numpy()

    logger.info("Training AttackTypeClassifier...")
    attack_classifier = AttackTypeClassifier.from_config(
        input_dim=train_hidden.shape[1], cfg=model_cfg.attack_classifier
    )
    class_weights = MultiClassFocalLoss.compute_class_weights(
        train_attack_labels_seq, num_classes=model_cfg.attack_classifier.num_classes, max_ratio=10.0
    ).to(device)
    logger.info("Attack classifier class weights: %s", class_weights.cpu().tolist())
    mc_focal_loss = MultiClassFocalLoss(
        alpha=model_cfg.attack_classifier.focal_alpha,
        gamma=model_cfg.attack_classifier.focal_gamma,
        class_weights=class_weights,
    )
    attack_history = train_attack_classifier(
        attack_classifier, train_hidden, train_attack_labels_seq, val_hidden, val_attack_labels_seq,
        mc_focal_loss, training_cfg,
    )
    save_model(
        attack_classifier,
        models_dir / "attack_classifier.pt",
        metadata={"input_dim": train_hidden.shape[1], "config": "attack_classifier"},
    )

    logger.info("Executing calibration via run_stage_calibrate...")
    calibration_summary = run_stage_calibrate(config)

    elapsed = time.perf_counter() - start
    logger.info("Model training completed in %.2f seconds.", elapsed)

    history = {"autoencoder": ae_history, "lstm": lstm_history}
    history_path = models_dir / "training_history.json"
    with open(history_path, "w") as f:
        json.dump(history, f, indent=2)
    logger.info("Training histories saved to %s", history_path)

    return {
        "autoencoder": autoencoder,
        "lstm": lstm,
        "thresholds": thresholds,
        "calibration_summary": calibration_summary,
        "ae_history": ae_history,
        "lstm_history": lstm_history,
    }

def run_stage_infer(
    config: ProjectConfig,
```

```
    risk_threshold: float = 0.5,
) -> Dict[str, Any]:
    logger.info("=" * 60)
    logger.info("STAGE 4: Inference & Alert Generation")
    logger.info("=" * 60)
    start = time.perf_counter()

    pipeline = InferencePipeline.from_saved_models(config, risk_threshold)
    results = pipeline.run(split="test", risk_threshold=risk_threshold)

    alerts_df = results["alerts_df"]
    if not alerts_df.empty:
        alerts_path = config.paths.outputs_dir / "alerts.csv"
        alerts_df.to_csv(alerts_path, index=False)
        logger.info("Saved %d alerts to %s", len(alerts_df), alerts_path)

    elapsed = time.perf_counter() - start
    logger.info("Inference completed in %.2f seconds.", elapsed)

    return results

def _budgeted_macro_selection(scores: np.ndarray, y: np.ndarray, attack_types: np.ndarray, budget: int) -> tuple[float, float]:
    candidates = np.arange(0.05, 0.81, 0.01)
    best_threshold, best_score = float(candidates[-1]), -1.0
    benign = attack_types == "none"
    for threshold in candidates:
        pred = scores >= threshold
        if int(pred.sum()) > budget:
            continue
        f1s = []
        for attack in ATTACK_TYPES:
            mask = benign | (attack_types == attack)
            if not (attack_types == attack).any():
                continue
            tp = int((pred[mask] & (attack_types[mask] == attack)).sum())
            fp = int((pred[mask] & benign[mask]).sum())
            fn = int((~pred[mask] & (attack_types[mask] == attack)).sum())
            precision = tp / (tp + fp) if tp + fp else 0.0
            recall = tp / (tp + fn) if tp + fn else 0.0
            f1s.append(2 * precision * recall / (precision + recall) if precision + recall else 0.0)
        value = float(np.mean(f1s)) if f1s else -1.0
        if value > best_score or (np.isclose(value, best_score) and threshold > best_threshold):
            best_threshold, best_score = float(threshold), value
    if best_score < 0:
        raise RuntimeError(f"No threshold met alert budget {budget}; extend the threshold grid.")
    return best_threshold, best_score

def run_stage_calibrate(config: ProjectConfig, risk_threshold: float = 0.5) -> Dict[str, Any]:
    logger.info("STAGE: Calibration (saved models only)")
    pipeline = InferencePipeline.from_saved_models(config, risk_threshold)
    results = pipeline.run(split="val")
    _, y, entity_types, _, _, _, _, _, attack_types, _ = pipeline._load_split_data("val", config.paths, config.features)
    offset = pipeline.sequence_length - 1
    y, entity_types, attack_types = y[offset:], np.asarray(entity_types[offset:]), np.asarray(attack_types[offset:])
    ae, lstm, policy = results["ae_errors"], results["lstm_probs"], results["policy_scores"]
    budget = config.training.alert_budget
    best_weights, best_threshold, best_value = None, 0.5, -1.0
    for w_ae in np.arange(0.10, 0.81, 0.10):
        for w_lstm in np.arange(0.10, 0.81, 0.10):
            w_policy = round(1.0 - w_ae - w_lstm, 10)
            if w_policy < 0.10:
                continue
            pipeline.risk_scorer.update_weights(float(w_ae), float(w_lstm), w_policy)
            scores = pipeline.risk_scorer.compute(ae, lstm, policy, pipeline.ae_min, pipeline.ae_max)
            threshold, value = _budgeted_macro_selection(scores, y, attack_types, budget)
            if value > best_value:
                best_weights, best_threshold, best_value = (float(w_ae), float(w_lstm), w_policy), threshold, value
    assert best_weights is not None
    pipeline.risk_scorer.update_weights(*best_weights)
    scores = pipeline.risk_scorer.compute(ae, lstm, policy, pipeline.ae_min, pipeline.ae_max)
    quotas = {etype: max(1, round(budget * float((entity_types == etype).mean()))) for etype in ENTITY_TYPES}
    thresholds = {"global": best_threshold}
    for etype in ENTITY_TYPES:
        mask = entity_types == etype
        thresholds[etype], _ = _budgeted_macro_selection(scores[mask], y[mask], attack_types[mask], quotas[etype])
    unscaled = pipeline.scaler.inverse_transform(np.load(config.paths.processed_data_dir / config.features.x_val_file)[offset:])
    feature_df = pd.DataFrame(unscaled, columns=pipeline.feature_names)
    _, _, _, entity_ids, timestamps, source_ips, _, _, _, _ = pipeline._load_split_data("val", config.paths, config.features)
    feature_df["source_ip"] = np.asarray(source_ips[offset:])
    feature_df["entity_id"] = np.asarray(entity_ids[offset:])
    feature_df["timestamp"] = np.asarray(timestamps[offset:])
```

```
    signature_thresholds, signature_evidence = tune_signatures(feature_df, attack_types)
    with open(config.paths.risk_weights_file, "w") as f:
        json.dump({"w_ae": best_weights[0], "w_lstm": best_weights[1], "w_policy": best_weights[2], "ae_min": pipeline.ae_min, "ae_max":
pipeline.ae_max}, f, indent=2)
    with open(config.paths.risk_thresholds_file, "w") as f:
        json.dump(thresholds, f, indent=2)
    with open(config.paths.models_dir / "signature_thresholds.json", "w") as f:
        json.dump(signature_thresholds, f, indent=2)
     recalls = {attack: float((scores[attack_types == attack] >= best_threshold).mean()) for attack in ATTACK_TYPES if (attack_types ==
attack).any()}
    for attack, recall in recalls.items():
        if recall < 0.15:
            logger.warning("Post-hoc ML-only recall below 0.15 for %s: %.3f; signature union is evaluated separately.", attack, recall)
       summary = {"selected_weights": dict(zip(("w_ae", "w_lstm", "w_policy"), best_weights)), "global_threshold": best_threshold,
"persona_thresholds": thresholds, "alert_budget": budget, "macro_f1": best_value, "ml_only_recalls": recalls, "signature_validation":
signature_evidence, "signature_thresholds": signature_thresholds}
    with open(config.paths.outputs_dir / "optimization_summary.json", "w") as f:
        json.dump(summary, f, indent=2)
    run_stage_infer(config, risk_threshold)
    return summary

def run_full_pipeline(
    config: Optional[ProjectConfig] = None,
    risk_threshold: float = 0.5,
) -> Dict[str, Any]:
    if config is None:
        config = get_project_config()

    total_start = time.perf_counter()
    logger.info("=" * 60)
    logger.info("BEHAVIORAL ANOMALY DETECTION -- FULL PIPELINE")
    logger.info("=" * 60)

    run_stage_generate(config)
    features = run_stage_features(config)
    training = run_stage_train(config)
    inference = run_stage_infer(config, risk_threshold)

    total_elapsed = time.perf_counter() - total_start
    logger.info("=" * 60)
    logger.info("FULL PIPELINE COMPLETED in %.2f seconds.", total_elapsed)
    logger.info("=" * 60)

    return {
        "features": features,
        "training": training,
        "inference": inference,
        "total_time_seconds": total_elapsed,
    }

def main() -> None:
    parser = argparse.ArgumentParser(
        description="Behavioral Anomaly Detection Pipeline Runner",
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog="""
Stages:
  generate  -- Run synthetic data generation only
  features  -- Run feature engineering only
  train     -- Run model training only
  infer     -- Run inference and alert generation only
  all       -- Run the complete end-to-end pipeline

Examples:
  python pipeline_runner.py --stage all
  python pipeline_runner.py --stage train
  python pipeline_runner.py --stage infer --risk-threshold 0.6
        """,
    )
    parser.add_argument(
        "--stage",
        type=str,
        default="all",
        choices=["generate", "features", "train", "calibrate", "infer", "all"],
        help="Pipeline stage to execute (default: all).",
    )
    parser.add_argument(
        "--risk-threshold",
        type=float,
        default=0.5,
        help="Risk score threshold for alert generation (default: 0.5).",
    )
```

```
    parser.add_argument(
        "--log-level",
        type=str,
        default="INFO",
        choices=["DEBUG", "INFO", "WARNING", "ERROR"],
        help="Logging verbosity level (default: INFO).",
    )

    args = parser.parse_args()

    logging.basicConfig(
        level=getattr(logging, args.log_level),
        format="%(asctime)s | %(name)s | %(levelname)s | %(message)s",
    )

    config = get_project_config()

    if args.stage == "generate":
        run_stage_generate(config)
    elif args.stage == "features":
        run_stage_features(config)
    elif args.stage == "train":
        run_stage_train(config)
    elif args.stage == "calibrate":
        run_stage_calibrate(config, args.risk_threshold)
    elif args.stage == "infer":
        run_stage_infer(config, args.risk_threshold)
    elif args.stage == "all":
        run_full_pipeline(config, args.risk_threshold)
    else:
        parser.print_help()
        sys.exit(1)

if __name__ == "__main__":
    main()
```

File: dashboard.py

Description: SOC Interactive Streamlit Dashboard

```
"""Interactive Streamlit SOC Analyst Dashboard.

Renders overview metrics, alerts table, risk distributions, entity timeline, attack chains, MITRE mapping,
Captum IG feature importance, and system health status.
"""

from __future__ import annotations

import json
import logging
import pickle
import re
from pathlib import Path
from typing import Any, Dict, List, Optional, Tuple

import numpy as np
import pandas as pd

try:
    import streamlit as st
except ImportError:
    raise ImportError(
        "Streamlit is required for the dashboard. "
        "Install it with: pip install streamlit"
    )

from config import (
    ATTACK_TYPES,
    ENTITY_TYPES,
    ProjectConfig,
    get_project_config,
)
from alert_engine import (
    MITRE_ATTACK_MAPPING,
    SEVERITY_LEVELS,
    get_severity_distribution,
)

logger = logging.getLogger(__name__)

st.set_page_config(
    page_title="SOC Dashboard -- Behavioral Anomaly Detection",
    page_icon="??",
    layout="wide",
    initial_sidebar_state="expanded",
)

@st.cache_data
def load_raw_data(
    raw_logs_path: str, ground_truth_path: str
) -> Tuple[pd.DataFrame, pd.DataFrame]:
    df_logs = pd.read_csv(raw_logs_path, parse_dates=["timestamp"])
    df_truth = pd.read_csv(ground_truth_path, parse_dates=["timestamp"])
    return df_logs, df_truth

@st.cache_data
def load_alerts(alerts_path: str) -> pd.DataFrame:
    if not Path(alerts_path).exists():
        return pd.DataFrame()
    return pd.read_csv(alerts_path)

@st.cache_data
def load_evaluation_report(report_path: str) -> Dict[str, Any]:
    if not Path(report_path).exists():
        return {}
    with open(report_path, "r") as f:
        return json.load(f)

@st.cache_data
def load_feature_metadata(metadata_path: str) -> Dict[str, Any]:
    if not Path(metadata_path).exists():
        return {}
    with open(metadata_path, "r") as f:
        return json.load(f)

@st.cache_data
```

```
def load_training_history(history_path: str) -> Dict[str, Any]:
    if not Path(history_path).exists():
        return {}
    with open(history_path, "r") as f:
        return json.load(f)

def render_sidebar(
    df_logs: pd.DataFrame,
    alerts_df: pd.DataFrame,
) -> Dict[str, Any]:
    st.sidebar.title("?? SOC Dashboard")
    st.sidebar.markdown("---")

    st.sidebar.header("? Filters")

    entity_types = st.sidebar.multiselect(
        "Entity Type",
        options=list(ENTITY_TYPES),
        default=list(ENTITY_TYPES),
    )

    severity_options = list(SEVERITY_LEVELS.keys())
    severity_filter = st.sidebar.multiselect(
        "Severity",
        options=severity_options,
        default=severity_options,
    )

    risk_range = st.sidebar.slider(
        "Risk Score Range",
        min_value=0.0,
        max_value=1.0,
        value=(0.0, 1.0),
        step=0.05,
    )

    st.sidebar.markdown("---")
    st.sidebar.header("? Search")
    search_entity_id = st.sidebar.text_input(
        "Entity ID", placeholder="e.g. COR-00001"
    )
    search_ip = st.sidebar.text_input(
        "Source IP", placeholder="e.g. 10.1.0.1"
    )

    attack_type_filter = st.sidebar.multiselect(
        "Attack Type",
        options=["all"] + list(ATTACK_TYPES) + ["unknown"],
        default=["all"],
    )

    return {
        "entity_types": entity_types,
        "severity_filter": severity_filter,
        "risk_range": risk_range,
        "search_entity_id": search_entity_id,
        "search_ip": search_ip,
        "attack_type_filter": attack_type_filter,
    }

def apply_filters(
    alerts_df: pd.DataFrame,
    filters: Dict[str, Any],
) -> pd.DataFrame:
    if alerts_df.empty:
        return alerts_df

    filtered = alerts_df.copy()

    if filters["entity_types"]:
        filtered = filtered[filtered["entity_type"].isin(filters["entity_types"])]

    if filters["severity_filter"]:
        filtered = filtered[filtered["severity"].isin(filters["severity_filter"])]

    lo, hi = filters["risk_range"]
    filtered = filtered[
        (filtered["risk_score"] >= lo) & (filtered["risk_score"] <= hi)
    ]

    if filters["search_entity_id"]:
```

```
        filtered = filtered[
            filtered["entity_id"].str.contains(
                filters["search_entity_id"], case=False, na=False
            )
        ]

    if filters["search_ip"]:
        if "source_ip" in filtered.columns:
            filtered = filtered[
                filtered["source_ip"].str.contains(
                    filters["search_ip"], case=False, na=False
                )
            ]

    if "all" not in filters["attack_type_filter"] and filters["attack_type_filter"]:
        filtered = filtered[
            filtered["predicted_attack_type"].isin(filters["attack_type_filter"])
        ]

    return filtered

def render_dataset_overview(
    df_logs: pd.DataFrame,
    df_truth: pd.DataFrame,
) -> None:
    st.header("? Dataset Overview")

    col1, col2, col3, col4 = st.columns(4)
    col1.metric("Total Events", f"{len(df_logs):,}")
    col2.metric("Unique Entities", f"{df_logs['entity_id'].nunique():,}")

    attack_rate = df_truth["is_attack"].mean() * 100
    col3.metric("Attack Rate", f"{attack_rate:.2f}%")
    col4.metric("Total Attacks", f"{df_truth['is_attack'].sum():,}")

    col_left, col_right = st.columns(2)
    with col_left:
        st.subheader("Entity Type Distribution")
        entity_counts = df_logs["entity_type"].value_counts()
        st.bar_chart(entity_counts)

    with col_right:
        st.subheader("Attack Type Distribution")
        attack_counts = df_truth[df_truth["is_attack"] == 1]["attack_type"].value_counts()
        if not attack_counts.empty:
            st.bar_chart(attack_counts)
        else:
            st.info("No attacks found in ground truth.")

    st.subheader("Event Volume Over Time")
    df_logs["date"] = df_logs["timestamp"].dt.date
    daily_counts = df_logs.groupby("date").size()
    st.line_chart(daily_counts)

def render_alert_panel(
    alerts_df: pd.DataFrame,
) -> None:
    st.header("? Live Alert Panel")

    if alerts_df.empty:
        st.info("No alerts match the current filters. Run the inference pipeline first.")
        return

    sev_cols = st.columns(5)
    severity_order = ["critical", "high", "medium", "low", "info"]
    severity_colors = {
        "critical": "?", "high": "?", "medium": "?",
        "low": "?", "info": "?",
    }
    for i, sev in enumerate(severity_order):
        count = (alerts_df["severity"] == sev).sum()
        sev_cols[i].metric(
            f"{severity_colors[sev]} {sev.capitalize()}",
            f"{count:,}",
        )

    st.subheader("Alert Details")
    st.caption(
        "Detection source: `ml_threshold` = hybrid-model threshold, "
        "`signature_rule` = high-precision behavioural signature, "
        "`both` = both detection paths agreed."
```

```
    )
    display_cols = [
        "alert_id", "entity_id", "entity_type", "severity",
        "risk_score", "detection_source", "predicted_attack_type", "mitre_technique_id",
        "classification_confidence",
        "contributing_factors",
        "timestamp",
    ]
    available_cols = [c for c in display_cols if c in alerts_df.columns]
    display_df = alerts_df[available_cols].head(100).copy()
    if "classification_confidence" in display_df.columns:
        display_df["classification_confidence"] = display_df.apply(
            lambda row: (
                f"{float(row['classification_confidence']):.0%}"
                if pd.notna(row["classification_confidence"])
                else (
                    "Rule-based (validated)"
                    if row.get("detection_source") == "signature_rule"
                    else (
                        "Signature override (validated)"
                        if row.get("detection_source") == "both" else "N/A"
                    )
                )
            ),
            axis=1,
        )
        display_df = display_df.rename(
            columns={"classification_confidence": "Confidence"}
        )
    st.dataframe(
        display_df,
        width="stretch",
        hide_index=True,
    )

    alert_options = alerts_df["alert_id"].astype(str).head(100).tolist()
    selected_alert_id = st.selectbox(
        "Inspect an alert",
        alert_options,
        key="alert_detail_selector",
    )
    selected_alert = alerts_df[
        alerts_df["alert_id"].astype(str) == selected_alert_id
    ].iloc[0]
    detail_cols = st.columns(6)
    detail_cols[0].metric("AE Error", f"{selected_alert['ae_error']:.6f}")
    detail_cols[1].metric("LSTM Probability", f"{selected_alert['lstm_prob']:.6f}")
    detail_cols[2].metric("Policy Score", f"{selected_alert['policy_score']:.6f}")
    detail_cols[3].metric("Attack Type", str(selected_alert["predicted_attack_type"]))
    confidence = selected_alert.get("classification_confidence")
    confidence_display = (
        f"{float(confidence):.0%}" if pd.notna(confidence)
        else (
            "Rule-based (validated)"
            if selected_alert.get("detection_source") == "signature_rule"
            else (
                "Signature override (validated)"
                if selected_alert.get("detection_source") == "both" else "N/A"
            )
        )
    )
    detail_cols[4].metric(
        "Classification Confidence",
        confidence_display,
    )
    detail_cols[5].metric("Detection Source", str(selected_alert["detection_source"]))
    st.markdown("**Why this alert fired**")
    st.write(selected_alert.get("contributing_factors", "No explanation recorded."))

def render_risk_score_panel(alerts_df: pd.DataFrame, report: Dict[str, Any]) -> None:
    st.header("? Hybrid Risk Score Distribution")

    if alerts_df.empty or "risk_score" not in alerts_df.columns:
        st.info("No risk scores available.")
        return

    col1, col2, col3, col4 = st.columns(4)
    scores = alerts_df["risk_score"]
    col1.metric("Mean", f"{scores.mean():.4f}")
    col2.metric("Median", f"{scores.median():.4f}")
    col3.metric("Max", f"{scores.max():.4f}")
```

```
    col4.metric("Std Dev", f"{scores.std():.4f}")

    st.subheader("Score Distribution")
    hist_data = pd.cut(scores, bins=20).value_counts().sort_index()
    hist_data.index = hist_data.index.astype(str)
    st.bar_chart(hist_data)

    if "benign_risk_distribution" in report and "attack_risk_distribution" in report:
        st.subheader("Risk Distribution: Benign vs Attack")
        colA, colB = st.columns(2)
        with colA:
            st.markdown("**Benign Traffic**")
            st.json(report["benign_risk_distribution"])
        with colB:
            st.markdown("**Attack Traffic**")
            st.json(report["attack_risk_distribution"])

def render_entity_timeline(
    alerts_df: pd.DataFrame,
) -> None:
    st.header("? Entity Timeline")

    if alerts_df.empty:
        st.info("No alerts to show timeline.")
        return

    entity_ids = sorted(alerts_df["entity_id"].unique().tolist())
    if not entity_ids:
        st.info("No entities with alerts.")
        return

    selected_entity = st.selectbox("Select Entity", entity_ids)

    entity_alerts = alerts_df[alerts_df["entity_id"] == selected_entity].copy()
    if entity_alerts.empty:
        st.info(f"No alerts for {selected_entity}.")
        return

    entity_alerts["timestamp_parsed"] = pd.to_datetime(
        entity_alerts["timestamp"], utc=True, errors="coerce"
    )
    entity_alerts = entity_alerts.sort_values("timestamp_parsed")

    st.subheader(f"Timeline for {selected_entity}")
    if "cold_start" in entity_alerts.columns and entity_alerts["cold_start"].fillna(False).astype(bool).any():
        st.info(
            "? Cold-Start Entity -- limited history, scored via persona-baseline fallback."
        )
    timeline_cols = [
        "timestamp", "severity", "risk_score", "detection_source",
        "predicted_attack_type", "contributing_factors", "cold_start",
        "ae_error", "lstm_prob", "policy_score",
    ]
    st.dataframe(
        entity_alerts[[c for c in timeline_cols if c in entity_alerts.columns]],
        width="stretch",
        hide_index=True,
    )

    if len(entity_alerts) > 1:
        st.subheader("Risk Score Over Time")
        chart_data = entity_alerts.set_index("timestamp_parsed")["risk_score"]
        st.line_chart(chart_data)

def render_attack_chain_panel(alerts_df: pd.DataFrame) -> None:
    st.header("? Attack Chain Reconstruction")

    if alerts_df.empty or "attack_chain_id" not in alerts_df.columns:
        st.info("No attack chains available.")
        return

    chains = alerts_df[alerts_df["attack_chain_id"].notna() & (alerts_df["attack_chain_id"] != "")]
    if chains.empty:
        st.info(
            "No active multi-stage attack chains detected. Attack chains are "
            "reconstructed when multiple related alerts for the same entity occur "
            "within the configured correlation window -- this indicates no "
            "correlated attack sequence exists in the current alert set."
        )
        return
```

```
    chain_ids = sorted(chains["attack_chain_id"].unique().tolist())
    st.metric("Total Chains", len(chain_ids))

    selected_chain = st.selectbox("Select Chain", chain_ids)
    chain_alerts = chains[chains["attack_chain_id"] == selected_chain].copy()

    st.subheader(f"Chain: {selected_chain}")
    col1, col2, col3 = st.columns(3)
    col1.metric("Alerts in Chain", len(chain_alerts))
    col2.metric("Entities", chain_alerts["entity_id"].nunique())
    col3.metric("Max Risk", f"{chain_alerts['risk_score'].max():.4f}")

    st.dataframe(
        chain_alerts[
            ["alert_id", "entity_id", "timestamp", "severity",
             "risk_score", "predicted_attack_type", "mitre_technique_id"]
        ],
        width="stretch",
        hide_index=True,
    )

def render_mitre_panel(alerts_df: pd.DataFrame) -> None:
    st.header("? MITRE ATT&CK Mapping")

    if alerts_df.empty:
        st.info("No alerts for MITRE mapping.")
        return

    if "mitre_tactic" in alerts_df.columns:
        st.subheader("Tactic Distribution")
        tactic_counts = alerts_df["mitre_tactic"].value_counts()
        st.bar_chart(tactic_counts)

    if "mitre_technique_id" in alerts_df.columns:
        st.subheader("Technique Distribution")
        tech_counts = alerts_df["mitre_technique_id"].value_counts()
        st.bar_chart(tech_counts)

    st.subheader("Mapping Reference")
    mapping_rows = []
    for atype, info in MITRE_ATTACK_MAPPING.items():
        if atype == "unknown":
            continue
        mapping_rows.append({
            "Attack Type": atype,
            "Tactic": info["tactic"],
            "Technique ID": info["technique_id"],
            "Technique Name": info["technique_name"],
            "Severity Base": info["severity_base"],
        })
    st.dataframe(pd.DataFrame(mapping_rows), width="stretch", hide_index=True)

def render_explainability_panel(
    alerts_df: pd.DataFrame,
    report: Dict[str, Any],
) -> None:
    st.header("? Explainability Panel")

    if alerts_df.empty:
        st.info("No data for explainability.")
        return

    tab_ae, tab_lstm, tab_policy, tab_features = st.tabs([
        "Reconstruction Error", "LSTM Probability", "Policy Score", "Feature Importance"
    ])

    with tab_ae:
        st.subheader("Autoencoder Reconstruction Error")
        if "ae_error" in alerts_df.columns:
            ae_data = alerts_df["ae_error"].dropna()
            if not ae_data.empty:
                col1, col2, col3 = st.columns(3)
                col1.metric("Mean Error", f"{ae_data.mean():.6f}")
                col2.metric("P95 Error", f"{ae_data.quantile(0.95):.6f}")
                col3.metric("Max Error", f"{ae_data.max():.6f}")
                ae_hist = pd.cut(ae_data, bins=20).value_counts().sort_index()
                ae_hist.index = ae_hist.index.astype(str)
                st.bar_chart(ae_hist)
        if "reconstruction_error_distribution" in report:
            stats = report["reconstruction_error_distribution"]
            st.json(stats)
```

```
    with tab_lstm:
        st.subheader("LSTM Anomaly Probability")
        if "lstm_prob" in alerts_df.columns:
            lstm_data = alerts_df["lstm_prob"].dropna()
            if not lstm_data.empty:
                col1, col2, col3 = st.columns(3)
                col1.metric("Mean Prob", f"{lstm_data.mean():.6f}")
                col2.metric("P95 Prob", f"{lstm_data.quantile(0.95):.6f}")
                col3.metric("Max Prob", f"{lstm_data.max():.6f}")
                lstm_hist = pd.cut(lstm_data, bins=20).value_counts().sort_index()
                lstm_hist.index = lstm_hist.index.astype(str)
                st.bar_chart(lstm_hist)

    with tab_policy:
        st.subheader("Policy Engine Score")
        if "policy_score" in alerts_df.columns:
            policy_data = alerts_df["policy_score"].dropna()
            if not policy_data.empty:
                col1, col2, col3 = st.columns(3)
                col1.metric("Mean Score", f"{policy_data.mean():.6f}")
                col2.metric("P95 Score", f"{policy_data.quantile(0.95):.6f}")
                col3.metric("Max Score", f"{policy_data.max():.6f}")
                policy_hist = pd.cut(policy_data, bins=20).value_counts().sort_index()
                policy_hist.index = policy_hist.index.astype(str)
                st.bar_chart(policy_hist)

    with tab_features:
        st.subheader("Most Common Reasons Alerts Fired")
        if "contributing_factors" not in alerts_df.columns:
            st.info("No contributing-factor data is available for the current alerts.")
        else:
            factor_counts: Dict[str, int] = {}
            for factors in alerts_df["contributing_factors"].dropna():
                for factor in re.split(r"[;,]", str(factors)):
                    normalized_factor = factor.strip()
                    if normalized_factor.startswith("travel_speed_kmh ="):
                        normalized_factor = "physically implausible travel speed"
                    if normalized_factor:
                        factor_counts[normalized_factor] = (
                            factor_counts.get(normalized_factor, 0) + 1
                        )

            if not factor_counts:
                st.info("No contributing factors were recorded for the current alerts.")
            else:
                factor_df = (
                    pd.DataFrame(
                        [{"Contributing factor": factor, "Alerts": count}
                         for factor, count in factor_counts.items()]
                    )
                    .sort_values(["Alerts", "Contributing factor"], ascending=[False, True])
                    .reset_index(drop=True)
                )
                st.caption(
                    "Counts are calculated from the current filtered alert set."
                )
                st.dataframe(factor_df, width="stretch", hide_index=True)

def render_system_health(
    df_logs: pd.DataFrame,
    alerts_df: pd.DataFrame,
    report: Dict[str, Any],
    drift_report: Dict[str, Any],
) -> None:
    st.header("? System Health Summary")

    col1, col2, col3, col4 = st.columns(4)
    col1.metric("Total Events Processed", f"{len(df_logs):,}")
    col2.metric("Total Alerts", f"{len(alerts_df):,}")

    if report:
        col3.metric("Model F1", f"{report.get('f1', 0):.4f}")
        col4.metric("Model ROC-AUC", f"{report.get('roc_auc', 0):.4f}")
    else:
        col3.metric("Model F1", "N/A")
        col4.metric("Model ROC-AUC", "N/A")

    st.subheader("Concept Drift Monitor (PSI)")
    if drift_report:
        cols = st.columns(len(drift_report))
```

```
        for i, (persona, data) in enumerate(drift_report.items()):
            psi = data.get("psi", 0.0)
            level = data.get("level", "normal")
            if level == "alert":
                st_color = "red"
                icon = "?"
            elif level == "warning":
                st_color = "orange"
                icon = "??"
            else:
                st_color = "green"
                icon = "?"
            cols[i].metric(f"{persona} {icon}", f"{psi:.4f}")
            if persona == "corporate_employee" and level == "alert":
                cols[i].caption(
                    "Elevated PSI under investigation -- see README Known Limitations."
                )
    else:
        st.info("No drift report found. Run pipeline with drift monitoring enabled.")

    if report:
        st.subheader("Model Performance Metrics")
        metrics_col1, metrics_col2 = st.columns(2)
        with metrics_col1:
            st.metric("Precision", f"{report.get('precision', 0):.4f}")
            st.metric("Recall", f"{report.get('recall', 0):.4f}")
            st.metric("F1-Score", f"{report.get('f1', 0):.4f}")
        with metrics_col2:
            st.metric("ROC-AUC", f"{report.get('roc_auc', 0):.4f}")
            st.metric("PR-AUC", f"{report.get('pr_auc', 0):.4f}")
            st.metric("FPR", f"{report.get('false_positive_rate', 0):.6f}")

        if "confusion_matrix" in report:
            st.subheader("Global Confusion Matrix")
            cm = report["confusion_matrix"]
            cm_df = pd.DataFrame(
                cm,
                index=["Actual Negative", "Actual Positive"],
                columns=["Predicted Negative", "Predicted Positive"],
            )
            st.dataframe(cm_df, width="stretch")

        if "persona_confusion_matrices" in report:
            st.subheader("Persona-wise Confusion Matrices")
            for etype, cm in report["persona_confusion_matrices"].items():
                st.markdown(f"**{etype}**")
                cm_df = pd.DataFrame(
                    cm,
                    index=["Actual Negative", "Actual Positive"],
                    columns=["Predicted Negative", "Predicted Positive"],
                )
                st.dataframe(cm_df, width="stretch")

        if "roc_curve_data" in report:
            st.subheader("ROC Curve")
            roc = report["roc_curve_data"]
            roc_df = pd.DataFrame({"FPR": roc["fpr"], "TPR": roc["tpr"]})
            st.line_chart(roc_df, x="FPR", y="TPR")

        if "pr_curve_data" in report:
            st.subheader("Precision-Recall Curve")
            pr = report["pr_curve_data"]
            pr_df = pd.DataFrame({"Recall": pr["recall"], "Precision": pr["precision"]})
            st.line_chart(pr_df, x="Recall", y="Precision")

        if "threshold_metrics" in report:
            st.subheader("Threshold Optimisation")
            tm = report["threshold_metrics"]
            tm_df = pd.DataFrame({"Threshold": tm["thresholds"], "F1": tm["f1"], "Precision": tm["precision"], "Recall": tm["recall"]})
            st.line_chart(tm_df.set_index("Threshold"))

        if "persona_wise_metrics" in report:
            st.subheader("Persona-wise Performance")
            persona_data = []
            for etype, metrics in report["persona_wise_metrics"].items():
                persona_data.append({
                    "Persona": etype,
                    "Precision": metrics["precision"],
                    "Recall": metrics["recall"],
                    "F1": metrics["f1"],
                    "Support": metrics["support"],
```

```
                })
            st.dataframe(pd.DataFrame(persona_data), width="stretch", hide_index=True)

        if "latency_stats_ms" in report:
            st.subheader("Latency Statistics (ms)")
            lat = report["latency_stats_ms"]
            st.json(lat)

def main() -> None:
    config = get_project_config()
    paths = config.paths

    try:
        df_logs, df_truth = load_raw_data(
            str(paths.raw_logs_file),
            str(paths.ground_truth_file),
        )
    except FileNotFoundError:
        st.error(
            "Raw data files not found. Run `python pipeline_runner.py --stage generate` first."
        )
        st.stop()

    alerts_df = load_alerts(str(paths.outputs_dir / "alerts.csv"))
    report = load_evaluation_report(
        str(paths.outputs_dir / "evaluation_report.json")
    )

    drift_report_path = paths.outputs_dir / "drift_report.json"
    drift_report = {}
    if drift_report_path.exists():
        with open(drift_report_path, "r", encoding="utf-8") as f:
            drift_report = json.load(f)

    feature_metadata = load_feature_metadata(
        str(paths.processed_data_dir / config.features.feature_metadata_file)
    )

    filters = render_sidebar(df_logs, alerts_df)
    filtered_alerts = apply_filters(alerts_df, filters)

    tab_overview, tab_alerts, tab_risk, tab_timeline, tab_chains, \
        tab_mitre, tab_explain, tab_health = st.tabs([
            "? Overview", "? Alerts", "? Risk Scores",
            "? Timeline", "? Chains", "? MITRE",
            "? Explainability", "? Health",
        ])

    with tab_overview:
        render_dataset_overview(df_logs, df_truth)

    with tab_alerts:
        render_alert_panel(filtered_alerts)

    with tab_risk:
        render_risk_score_panel(filtered_alerts, report)

    with tab_timeline:
        render_entity_timeline(filtered_alerts)

    with tab_chains:
        render_attack_chain_panel(filtered_alerts)

    with tab_mitre:
        render_mitre_panel(filtered_alerts)

    with tab_explain:
        render_explainability_panel(filtered_alerts, report)

    with tab_health:
        render_system_health(df_logs, filtered_alerts, report, drift_report)

if __name__ == "__main__":
    main()
```

File: scripts/validate_data.py

Description: Dataset Integrity Validator

```
"""CLI dataset validation script checking schema integrity, timestamps, and ground truth alignment.
"""

import pandas as pd
import numpy as np

LOG_PATH = "data/raw/raw_logs.csv"
LABEL_PATH = "data/raw/ground_truth.csv"

logs = pd.read_csv(LOG_PATH)
labels = pd.read_csv(LABEL_PATH)

print("=" * 80)
print("DATASET SUMMARY")
print("=" * 80)
print(f"Logs shape      : {logs.shape}")
print(f"Labels shape    : {labels.shape}")

print("\nColumns:")
print(logs.columns.tolist())

print("\nGround Truth Columns:")
print(labels.columns.tolist())

print("\n" + "=" * 80)
print("1. MISSING VALUES")
print("=" * 80)

print("\nLogs:")
print(logs.isna().sum())

print("\nLabels:")
print(labels.isna().sum())

print("\n" + "=" * 80)
print("2. TIMESTAMP CHECK")
print("=" * 80)

logs["timestamp"] = pd.to_datetime(logs["timestamp"])
labels["timestamp"] = pd.to_datetime(labels["timestamp"])

print("Logs sorted chronologically:",
      logs["timestamp"].is_monotonic_increasing)

print("Labels sorted chronologically:",
      labels["timestamp"].is_monotonic_increasing)

print("\n" + "=" * 80)
print("3. LABEL ALIGNMENT")
print("=" * 80)

print("Entity IDs aligned:",
      (logs["entity_id"] == labels["entity_id"]).all())

print("Timestamps aligned:",
      (logs["timestamp"] == labels["timestamp"]).all())

print("\n" + "=" * 80)
print("4. PERSONA DISTRIBUTION")
print("=" * 80)

print(logs["entity_type"].value_counts())

print("\n" + "=" * 80)
print("5. SESSION DURATION BY PERSONA")
print("=" * 80)

print(
    logs.groupby("entity_type")["session_duration"].describe()
)

print("\n" + "=" * 80)
print("6. AUTH METHODS")
print("=" * 80)

print(
```

```
    logs.groupby("entity_type")["auth_method"].value_counts()
)

print("\n" + "=" * 80)
print("7. UNIQUE RESOURCES PER PERSONA")
print("=" * 80)

print(
    logs.groupby("entity_type")["resource_accessed"].nunique()
)

print("\n" + "=" * 80)
print("8. TOP RESOURCES")
print("=" * 80)

print(
    logs.groupby("entity_type")["resource_accessed"]
    .value_counts()
    .head(20)
)

print("\n" + "=" * 80)
print("9. DEVICE FINGERPRINTS")
print("=" * 80)

print(
    logs.groupby("entity_type")["device_fingerprint"]
    .value_counts()
)

print("\n" + "=" * 80)
print("10. ATTACK DISTRIBUTION")
print("=" * 80)

print(labels["attack_type"].value_counts())

print("\nPercentage:")
print(
    labels["attack_type"]
    .value_counts(normalize=True)
    .mul(100)
    .round(3)
)

print("\n" + "=" * 80)
print("11. SAMPLE ATTACK EVENTS")
print("=" * 80)

for attack in labels["attack_type"].unique():
    if attack == "none":
        continue

    print(f"\n----- {attack} -----")

    idx = labels[labels.attack_type == attack].index[:3]

    print(logs.loc[idx][[
        "entity_id",
        "entity_type",
        "timestamp",
        "source_ip",
        "geo_location",
        "resource_accessed",
        "auth_method",
        "device_fingerprint"
    ]])

print("\n" + "=" * 80)
print("12. UNIQUE ENTITIES")
print("=" * 80)

print("Unique entities:", logs["entity_id"].nunique())

print("\nEntities by type:")

print(
    logs.groupby("entity_type")["entity_id"]
    .nunique()
)

print("\n" + "=" * 80)
```

```
print("13. COMMAND SEQUENCE EXAMPLES")
print("=" * 80)

for persona in logs["entity_type"].unique():

    print(f"\n{persona}")

    print(
        logs.loc[
            logs.entity_type == persona,
            "command_sequence"
        ].head(5).tolist()
    )

print("\n" + "=" * 80)
print("14. DUPLICATE ROW CHECK")
print("=" * 80)

print("Duplicate log rows:",
      logs.duplicated().sum())

print("\n" + "=" * 80)
print("15. BASIC VALIDATION")
print("=" * 80)

print("Row counts equal:",
      len(logs) == len(labels))

print("Attack rate:",
      round(labels["is_attack"].mean() * 100, 3), "%")

print("\nValidation Complete.")
```

File: tests/conftest.py

Description: Pytest Shared Fixtures

```
"""Pytest configuration and shared test fixtures.
"""

from pathlib import Path
import sys

PROJECT_ROOT = Path(__file__).resolve().parents[1]
if str(PROJECT_ROOT) not in sys.path:
    sys.path.insert(0, str(PROJECT_ROOT))
```

File: tests/test_smoke.py

Description: Automated Verification Test Suite

```
"""Automated smoke test suite verifying risk scorer monotonicity, weight sums, alert fallback thresholds,
prediction source confidence, temporal leakage, and recall guardrails.
"""

import pytest
import numpy as np

from config import get_project_config, RiskScorerConfig
from models import RiskScorer

def test_risk_scorer_monotonicity():
    cfg = RiskScorerConfig(w_ae=0.4, w_lstm=0.4, w_policy=0.2)
    scorer = RiskScorer(cfg)

    ae_errors = np.array([0.1, 0.5, 0.9])
    lstm_probs = np.array([0.1, 0.5, 0.9])
    policy_scores = np.array([0.1, 0.5, 0.9])

    scores = scorer.compute(ae_errors, lstm_probs, policy_scores, ae_min=0.0, ae_max=1.0)

    assert scores[0] < scores[1] < scores[2], "Risk scores are not monotonically increasing."

def test_risk_scorer_weight_sum():
    cfg = RiskScorerConfig(w_ae=0.5, w_lstm=0.3, w_policy=0.2)
    scorer = RiskScorer(cfg)

    ae_errors = np.array([1.0])
    lstm_probs = np.array([1.0])
    policy_scores = np.array([1.0])

    scores = scorer.compute(ae_errors, lstm_probs, policy_scores, ae_min=0.0, ae_max=1.0)

    assert np.isclose(scores[0], 1.0), "Risk score did not sum correctly."

def test_alert_engine_fallback_threshold(monkeypatch):
    from alert_engine import AlertEngine

    engine = AlertEngine(risk_threshold=0.5)

    risk_scores = np.array([0.6])
    ae_errors = np.array([0.5])
    lstm_probs = np.array([0.5])
    policy_scores = np.array([0.5])
    entity_ids = np.array(["user_1"])
    entity_types = np.array(["unknown_persona"])
    timestamps = np.array(["2025-01-01 12:00:00"])

    risk_thresholds = {"global": 0.55, "corporate_employee": 0.8}

    alerts_df = engine.generate_alerts(
        risk_scores=risk_scores,
        ae_errors=ae_errors,
        lstm_probs=lstm_probs,
        policy_scores=policy_scores,
        entity_ids=entity_ids,
        entity_types=entity_types,
        timestamps=timestamps,
        risk_thresholds=risk_thresholds
    )

    assert len(alerts_df) == 1, "Fallback to global threshold failed."

    risk_scores_below = np.array([0.5])
    alerts_df_below = engine.generate_alerts(
        risk_scores=risk_scores_below,
        ae_errors=ae_errors,
        lstm_probs=lstm_probs,
        policy_scores=policy_scores,
        entity_ids=entity_ids,
        entity_types=entity_types,
        timestamps=timestamps,
        risk_thresholds=risk_thresholds
    )

    assert len(alerts_df_below) == 0, "Fallback to global threshold failed (allowed alert below threshold)."
```

```
def test_alert_confidence_respects_prediction_source():
    from alert_engine import AlertEngine

    engine = AlertEngine(risk_threshold=0.5)
    base_args = dict(
        risk_scores=np.array([0.9, 0.9]),
        ae_errors=np.array([0.5, 0.5]),
        lstm_probs=np.array([0.9, 0.9]),
        policy_scores=np.array([0.5, 0.5]),
        entity_ids=np.array(["signature_user", "classifier_user"]),
        entity_types=np.array(["corporate_employee", "corporate_employee"]),
        timestamps=np.array(["2025-01-01 12:00:00", "2025-01-01 14:00:00"]),
        predicted_attack_types=np.array(["brute_force", "brute_force"]),
        attack_probabilities=np.array([[0.99, 0.001, 0.009], [0.05, 0.9, 0.05]]),
        attack_type_to_index={"brute_force": 1, "device_spoofing": 2},
        signature_matches={"device_spoofing_signature": np.array([True, False])},
    )

    alerts_df = engine.generate_alerts(**base_args)
    signature_alert = alerts_df.loc[alerts_df["entity_id"] == "signature_user"].iloc[0]
    classifier_alert = alerts_df.loc[alerts_df["entity_id"] == "classifier_user"].iloc[0]

    assert signature_alert["predicted_attack_type"] == "device_spoofing"
    assert np.isnan(signature_alert["classification_confidence"])
    assert classifier_alert["classification_confidence"] == pytest.approx(0.9)

def test_temporal_leakage():
    import pandas as pd
    from feature_pipeline import (
        _compute_behavioral_features,
        _compute_network_features,
        _compute_device_resource_features,
        _compute_relationship_features,
        _encode_entity_type,
        _init_geo_coords
    )
    from config import get_project_config

    cfg = get_project_config()
    _init_geo_coords(cfg)

    df1 = pd.DataFrame({
        "timestamp": pd.to_datetime(["2025-01-01 10:00:00", "2025-01-01 10:05:00", "2025-01-01 10:10:00"]),
        "entity_id": ["user1", "user1", "user1"],
        "entity_type": ["corporate_employee"] * 3,
        "source_ip": ["10.0.0.1", "10.0.0.2", "10.0.0.1"],
        "geo_location": ["US", "CA", "US"],
        "resource_accessed": ["res1", "res2", "res1"],
        "action": ["login", "download", "login"],
        "status": ["success", "success", "failure"],
        "device_fingerprint": ["dev1", "dev1", "dev2"],
        "session_duration": [30.0, 45.0, 10.0],
        "command_sequence": ["login", "login;download", "login"],
    })

    df2 = pd.DataFrame({
        "timestamp": pd.to_datetime(["2025-01-01 10:00:00", "2025-01-01 10:05:00", "2025-01-01 10:10:00", "2025-01-01 10:15:00"]),
        "entity_id": ["user1", "user1", "user1", "user1"],
        "entity_type": ["corporate_employee"] * 4,
        "source_ip": ["10.0.0.1", "10.0.0.2", "10.0.0.1", "10.0.0.99"],
        "geo_location": ["US", "CA", "US", "RU"],
        "resource_accessed": ["res1", "res2", "res1", "res99"],
        "action": ["login", "download", "login", "delete"],
        "status": ["success", "success", "failure", "success"],
        "device_fingerprint": ["dev1", "dev1", "dev2", "dev99"],
        "session_duration": [30.0, 45.0, 10.0, 60.0],
        "command_sequence": ["login", "login;download", "login", "delete;exit"],
    })

    def run_feat(df):
        df = df.copy()
        df["hour_of_day"] = df["timestamp"].dt.hour
        df["day_of_week"] = df["timestamp"].dt.dayofweek

        b = _compute_behavioral_features(df)
        n = _compute_network_features(df, cfg.features)
        dr = _compute_device_resource_features(df, cfg.features)
        r = _compute_relationship_features(df)
        return pd.concat([b, n, dr, r], axis=1)

    feat1 = run_feat(df1)
```

```
    feat2 = run_feat(df2)

    for col in feat1.columns:
            if col in ["timestamp", "entity_id", "entity_type", "source_ip", "geo_location", "resource_accessed", "action", "status",
"device_fingerprint"]:
                continue
            np.testing.assert_array_almost_equal(
                feat1[col].values,
                feat2[col].values[:3],
                err_msg=f"Temporal leakage detected in feature '{col}'!"
            )

def test_attack_wise_recall_guardrail():
    import json
    from inference_pipeline import InferencePipeline
    from config import get_project_config

    cfg = get_project_config()
    if not cfg.paths.risk_thresholds_file.exists():
        pytest.skip("Models not yet trained, skipping recall guardrail test.")

    pipeline = InferencePipeline.from_saved_models(cfg)
    results = pipeline.run(split="val")

    evaluation = results.get("evaluation")
    assert evaluation is not None, "Evaluation results missing."

    attack_metrics = evaluation.attack_wise_metrics
    for attack_type, metrics in attack_metrics.items():
        if metrics.support > 0:
            assert metrics.recall >= 0.15, f"Guardrail failed: {attack_type} recall is {metrics.recall:.4f} < 0.15"
```